

Corso Angular

Manuale di studio con esempi pratici

Angular 21 · componenti standalone · signal · zoneless

Dagli appunti del corso, con gli esempi del progetto corso-angular

Settembre 2026

Indice

1. Introduzione e installazione	7
Cos'è Angular	7
Come usare questo manuale	7
Installare gli strumenti	7
Creare e avviare un progetto	8
I file di un progetto	8
Come parte l'applicazione	9
2. TypeScript essenziale	10
Perché TypeScript	10
Tipi di base e inferenza	10
Interfacce e type	10
null, undefined e gli operatori ?. ?? !	11
Funzioni e arrow function	11
Classi	11
Destrutturazione e spread	12
Moduli: import ed export	12
Generics	12
Codice asincrono: Promise, async/await e Observable	12
Decoratori	12
Errori di TypeScript frequenti	13
3. La CLI di Angular	14
Comandi principali	14
Script npm	14
ng generate	14
Opzioni di generate	15
4. Componenti e template	16
Basi da ricordare	16
Un componente	16
Leggere e scrivere nel template	16
Data binding: i quattro modi	17
Stili del componente	17
Il ciclo di vita di un componente	18
Come si aggiorna la pagina: change detection e zoneless	19
5. Control flow e direttive standard	20
Tre tipi di direttive	20
@if, @else	20
@for, track, @empty	20
@switch	20
@defer: caricare più tardi	20
@let: variabili nel template	21
Riepilogo del control flow	21
La sintassi vecchia: *ngIf e *ngFor	21
Direttive di @angular/common	21
Direttive di @angular/forms	22
Variabili di riferimento nel template	22
6. Signal	23
Cos'è un signal	23
Leggere con le parentesi	23
computed: valori derivati	23
Aggiornare oggetti e array: sempre una copia nuova	24
effect: effetti collaterali	24
linkedSignal: un valore modificabile che si riallinea	24
Signal nei componenti e negli altri capitoli	24
Signal o proprietà normale?	25
7. Pipe	26
Cos'è una pipe	26
Pipe standard	26
Esempi: testo	26
Esempi: date (DatePipe)	26
Esempi: numero, percentuale, valuta	27
Formati in italiano: registrare la lingua	27
Frase in base al valore	27

Creare una pipe personalizzata	27
8. Direttive personalizzate	29
Una direttiva di attributo	29
La stessa direttiva in stile moderno	30
Una direttiva che reagisce a un clic fuori	30
Una direttiva strutturale	30
Errori tipici con le direttive	31
9. Comunicazione tra componenti	32
Padre e figlio	32
@Input: un esempio	32
Cosa deve contenere il padre	32
Cosa deve contenere il figlio	32
Dal padre al figlio: @Input	33
Dal figlio al padre: @Output	33
Regole da ricordare	33
@ViewChild: leggere un pezzo del template	33
Quale strumento usare	34
input() e output(): la versione moderna di @Input e @Output	34
model(): legame nei due sensi tra padre e figlio	35
Il padre usa il figlio: variabile di template e viewChild()	36
ng-content: passare HTML al figlio	36
Fratelli e componenti lontani: servizio con signal	36
La stessa cosa con RxJS: BehaviorSubject	37
Da una pagina all'altra	38
10. Servizi e dependency injection	40
Cos'è un servizio	40
Cos'è la dependency injection	40
Dove vive un servizio: root o componente	40
Valori di configurazione: InjectionToken	41
File di ambiente	41
Errori tipici con i servizi	41
11. Observable e RxJS	42
Cos'è un Observable	42
Subscribe e unsubscribe	42
Creare Observable	43
Subject e BehaviorSubject	43
pipe e gli operatori	43
Esempio: ricerca mentre si scrive	44
Esempio: caricare due cose insieme	45
Da Observable a signal e ritorno	45
Errori tipici con RxJS	45
12. Routing	46
Le idee di base	46
Passo 1: accendere il router	46
Passo 2: scrivere le rotte	46
Passo 3: dove compare la pagina	46
Passo 4: leggere il parametro nella pagina	47
Passo 5: cambiare pagina dal codice	47
Il comando ng generate module app-routing --flat	47
Un file di rotte completo	47
Link con evidenziazione della pagina attiva	48
Parametri del percorso: tre modi per leggerli	48
Esempio completo: dettaglio con precedente e successivo	49
Query params: filtri, ricerca, pagine	50
Dati statici della rotta	51
Rotte figlie e router-outlet annidato	51
Navigazione relativa dal codice	52
Guardie: chi può entrare e uscire	52
Resolver: dati pronti prima della pagina	52
Spinner durante il cambio pagina	53
Errori tipici del routing	53
13. Form template-driven	54
Due modi di fare i form	54
Accendere i form	54
Il form più piccolo	54
Riferimenti di template: # con ngForm, ngModel, ngModelGroup	55
I tipi di campo	55

Gruppi di campi: ngModelGroup	55
Il submit	56
Stati del form e dei campi	56
Quando mostrare un errore	56
Validatori pronti.....	57
Validatore personalizzato su un campo	57
Validatore su un gruppo: password uguali	58
Validatore asincrono: controllo sul server	58
ngModelOptions: quando aggiornare.....	59
Prendere il form con viewChild.....	59
L'esempio completo nel progetto	60
Pagina di esempi: tutte le forme di ngModel	60
Errori tipici con i form template-driven	62
14. Form reactive	63
Cosa sono.....	63
Accendere i form reactive.....	63
I mattoni	63
Creare un FormGroup.....	64
Aggiungere un FormControl per ogni campo.....	64
La stessa cosa con FormBuilder	65
Sincronizzare la parte HTML	65
Verificare il submit.....	66
Aggiungere la validazione	67
Accedere ai controlli nell'HTML	68
Leggere e scrivere dal codice.....	69
Liste dinamiche: FormArray.....	69
Stati del form reactive	70
L'esempio completo nel progetto	70
Template-driven e reactive a confronto	70
Errori tipici con i form reactive	70
15. Chiamate HTTP e database Firebase	72
Com'è fatta una chiamata HTTP	72
I metodi a confronto.....	72
Importare il modulo HTTP	73
HttpClient: i metodi e le opzioni	73
Header	74
Body e query params	74
Un servizio REST completo (CRUD)	74
Usare il servizio nel componente: quattro modi.....	75
Interceptor: codice che vede ogni richiesta	76
Provare con un server finto: json-server	77
Creare il database su Firebase	77
Come risponde Firebase: l'API REST	78
Il servizio completo per Firebase	79
La pagina dei corsi: il componente	79
Vedere le chiamate: un interceptor registro	80
Accedere con Firebase Authentication	81
Provare senza Firebase: il server finto	81
L'esempio completo nel progetto	82
Errori tipici con HTTP e Firebase	82
16. Autenticazione.....	83
Come funziona il login in un'app Angular	83
Modelli e lettura del token	83
Il servizio di autenticazione.....	84
L'interceptor che aggiunge il token	85
Le guardie	85
La pagina di login	86
Collegare tutto: rotte, configurazione, intestazione	87
Un server finto per provare subito	88
La pagina protetta che usa il token	89
Login e SSR.....	89
Rinnovare il token (refresh token).....	89
Sicurezza: cosa ricordare	90
17. Angular Material.....	92
Cos'è	92
Installazione e tema.....	92
Usare un componente: import e tag.....	92
Campi di un form: mat-form-field.....	93
Bottoni e icone	93

Card	94
Tabella: mat-table.....	94
Avvisi: snack bar	94
Reset di un form con Material	95
Altri componenti utili.....	95
18. Test	96
Perché e quali test	96
Eseguire i test	96
La struttura di un test	96
Testare una pipe	97
Testare una direttiva	97
Testare un componente con TestBed	97
Testare un servizio HTTP.....	98
Servizi finti nei test di componente	99
Errori tipici nei test	100
19. Build, SSR e pubblicazione	101
ng build	101
Rendering lato server (SSR) e prerender	101
Pubblicare su Firebase Hosting.....	102
Altri modi di pubblicare	102
Prima di pubblicare: controllo finale.....	102
20. Buone pratiche ed errori frequenti	104
Organizzare le cartelle	104
Regole di stile	104
Componenti	104
Stato e dati	104
Prestazioni	105
Accessibilità	105
Sicurezza	105
Errori già visti.....	105
Come cercare la soluzione a un errore	106
Ogni giorno	106
21. Esercizi	108
Basi (capitoli 1-8)	108
Comunicazione e servizi (capitoli 9-11)	108
Routing (capitolo 12)	108
Form (capitoli 13-14)	109
HTTP e autenticazione (capitoli 15-16)	109
Qualità e pubblicazione (capitoli 17-20)	109
Appendice A. Glossario.....	111
Appendice B. Gli esempi nel progetto corso-angular	113

PARTE PRIMA

Le basi

1. Introduzione e installazione

Cos'è Angular

Angular è un framework di Google per costruire applicazioni web che girano nel browser. Si scrive in TypeScript, cioè JavaScript con i tipi, e si organizza in componenti: pezzi di pagina riusabili, ognuno con i suoi dati, il suo HTML e il suo stile.

Un'app Angular è una SPA, Single Page Application. Il browser scarica una sola pagina HTML e il JavaScript dell'app; poi, quando l'utente clicca un link, Angular cambia il contenuto della pagina senza ricaricarla. I dati arrivano dal server con chiamate HTTP, in formato JSON.

Angular dà già pronto tutto quello che serve a un'app vera: componenti e template, router per le pagine, form con validazione, client HTTP, dependency injection per i servizi, strumenti per i test e per la build. Per questo si chiama framework e non libreria: decide la struttura, e tu riempi i pezzi.

Questo corso usa Angular 21, la versione dei progetti corso-angular e tour-degli-eroi. Le versioni recenti hanno cambiato molto rispetto ad Angular 2-15: componenti standalone senza NgModule, signal per lo stato, nuovo control flow @if e @for, app senza zone.js. Online trovi ancora molto materiale vecchio: se vedi NgModule, *ngIf ovunque o constructor pieni di parametri, è codice di qualche anno fa. Funziona ancora, ma non è il modo in cui si scrive oggi.

Come usare questo manuale

Ogni capitolo parte dalla teoria minima, mostra codice completo da copiare, e finisce con gli errori tipici. Gli esempi più grandi sono pagine vere del progetto corso-angular, che puoi aprire mentre leggi:

/iscrizione Form template-driven con validazione e stati. Capitolo 13.

/ngmodel Tutte le forme di ngModel. Capitolo 13.

/iscrizione-reactive Lo stesso form in versione reactive, con FormArray. Capitolo 14.

/corsi-firebase Chiamate HTTP GET, POST, PUT, PATCH, DELETE verso Firebase. Capitolo 15.

/material Componenti Angular Material e una pipe personalizzata. Capitoli 7 e 17.

Prerequisiti: HTML e CSS di base, e un po' di JavaScript (variabili, funzioni, array, oggetti). TypeScript si impara strada facendo; il capitolo 2 riassume quello che serve.

Consiglio di studio: leggi un capitolo, apri la pagina d'esempio, poi fai gli esercizi del capitolo 21 senza guardare il codice. Quando qualcosa non funziona, cerca l'errore nell'ultima sezione del capitolo o nel capitolo 20.

Installare gli strumenti

Servono tre cose: Node.js, la CLI di Angular e un editor.

1) Node.js, versione LTS (quella consigliata). Si scarica da nodejs.org. Porta con sé npm, il gestore dei pacchetti. Controlla l'installazione:

```
node -v
npm -v
```

2) La CLI di Angular, installata una volta per tutto il computer con -g (globale):

```
npm install -g @angular/cli
ng version
```

3) Visual Studio Code con l'estensione Angular Language Service: completa i nomi nel template, segnala gli errori mentre scrivi e permette di saltare dal template alla classe. Utile anche l'estensione del browser Angular DevTools, che mostra l'albero dei componenti e i loro valori.

Su Windows, se PowerShell rifiuta il comando `ng` con un errore sui criteri di esecuzione degli script, usa `npx ng` al posto di `ng` (come negli esempi di questo manuale) oppure il prompt dei comandi.

Creare e avviare un progetto

```
ng new corso-angular
cd corso-angular
ng serve -o
```

`ng new` fa alcune domande: il formato degli stili (scegli SCSS), se attivare il rendering lato server SSR (corso-angular l'ha attivato) e se generare file di configurazione per gli assistenti di intelligenza artificiale. Poi scarica le dipendenze in `node_modules`: ci vuole qualche minuto.

`ng serve` compila l'app e la apre su `http://localhost:4200` (-o apre il browser da solo). Resta in ascolto: ogni volta che salvi un file ricompila e aggiorna la pagina. Si ferma con `Ctrl+C` nel terminale.

Se `ng serve` mostra dati vecchi dopo che hai aggiunto file nuovi, o la pagina non riflette le modifiche, fermalo e riavvialo: la cache di Vite, lo strumento che compila, ogni tanto resta indietro. È successo più volte costruendo gli esempi di questo corso.

I file di un progetto

package.json Le dipendenze (Angular, RxJS...) e gli script: `npm start`, `npm test`, `npm run build`.

node_modules/ Le librerie scaricate da npm. Non si modifica e non si mette in git: si ricrea con `npm install`.

angular.json La configurazione del progetto: dove sono i file, le opzioni di build e di serve, i file di ambiente.

tsconfig.json Le opzioni di TypeScript. `strict: true` attiva i controlli più severi, e conviene tenerlo.

public/ File copiati così come sono: favicon, immagini.

src/index.html L'unica pagina HTML. Contiene solo il tag del componente radice, `<app-root>`.

src/main.ts Il punto di partenza: avvia l'app con `bootstrapApplication`.

src/styles.scss Gli stili globali, validi in tutta l'app. In corso-angular contiene il tema di Angular Material.

src/environments/ Valori diversi per sviluppo e produzione, come l'URL delle API.

src/app/app.ts Il componente radice, App. Parte da qui.

src/app/app.html Il template della radice: il menu e `<router-outlet>`.

src/app/app.config.ts I provider globali: router, HttpClient, hydration.

src/app/app.routes.ts L'elenco delle rotte: quale indirizzo mostra quale componente.

src/main.server.ts, server.ts, app.config.server.ts, app.routes.server.ts I file del rendering lato server (SSR). Capitolo 19.

***.spec.ts** I test. Ogni `ng generate` ne crea uno accanto al file. Capitolo 18.

Come parte l'applicazione

```
<!-- src/index.html -->
<body>
  <app-root></app-root>
</body>

// src/main.ts
import { bootstrapApplication } from '@angular/platform-browser';
import { appConfig } from './app/app.config';
import { App } from './app/app';

bootstrapApplication(App, appConfig).catch((err) => console.error(err));

// src/app/app.ts
@Component({
  selector: 'app-root',
  imports: [RouterLink, RouterOutlet],
  templateUrl: './app.html',
})
export class App {}
```

Il browser carica index.html, che contiene il tag vuoto <app-root>. main.ts avvia Angular con il componente App e la configurazione appConfig. Angular trova <app-root> perché è il selector di App, e ci disegna dentro app.html. In app.html c'è <router-outlet>: lì il router inserisce il componente della pagina corrente, in base all'indirizzo. Da quel momento tutto è un albero di componenti che contengono altri componenti.

2. TypeScript essenziale

Perché TypeScript

TypeScript è JavaScript con i tipi. Scrivi il tipo di una variabile, di un parametro, di un oggetto, e il compilatore ti avvisa prima di eseguire il codice se usi un valore nel modo sbagliato. Il browser non capisce TypeScript: ng serve e ng build lo traducono in JavaScript. In Angular quasi tutti gli errori che vedi in compilazione sono errori di tipo: impararli a leggere fa risparmiare molto tempo.

Tipi di base e inferenza

```
let nome: string = 'Mario';
let eta: number = 30;
let attivo: boolean = true;
let voti: number[] = [7, 8, 9];
let qualsiasi: unknown = 'da controllare prima di usarlo';

let citta = 'Roma';           // inferenza: TypeScript capisce da solo che è string
citta = 42;                   // errore: Type 'number' is not assignable to type 'string'
```

Non serve scrivere il tipo quando è ovvio dal valore iniziale: TypeScript lo deduce. Scrivilo sui parametri delle funzioni, sui valori di ritorno importanti e quando il valore iniziale non basta, per esempio null.

Evita any: spegne i controlli e nasconde gli errori. Se non sai il tipo, usa unknown, che obbliga a controllarlo prima dell'uso.

Interfacce e type

```
export interface Eroe {
  id: number;
  nome: string;
  superpotere: string;
  email?: string;           // ? = facoltativo, può mancare
  readonly creato: Date;    // non si può riassegnare
}

export type Livello = 'base' | 'intermedio' | 'avanzato'; // solo questi tre valori
export type Id = number | string;                       // 1'uno o l'altro

const e: Eroe = { id: 1, nome: 'Superman', superpotere: 'volo', creato: new Date() };
```

Un'interfaccia descrive la forma di un oggetto: quali campi ha e di che tipo. Non produce codice JavaScript, serve solo al compilatore. type fa lo stesso e in più permette unioni (|) e tipi letterali come 'base' | 'avanzato', molto usati in Angular al posto degli enum.

Tipi di utilità che userai spesso:

Partial<Eroe> Tutti i campi facoltativi. Per i dati di un aggiornamento parziale (PATCH).

Omit<Eroe, 'id'> Eroe senza il campo id. Per i dati di un elemento nuovo.

Pick<Eroe, 'id' | 'nome'> Solo i campi indicati.

Record<string, Eroe> Un oggetto con chiavi stringa e valori Eroe. È la forma delle risposte di Firebase.

ReturnType<typeof f> Il tipo del valore restituito dalla funzione f.

null, undefined e gli operatori ?. ?? !

```
let scelto: Eroe | undefined;           // può non esserci ancora

scelto.nome;                           // errore: 'scelto' is possibly 'undefined'
scelto?.nome;                           // optional chaining: undefined se scelto manca
scelto?.nome ?? 'nessuno';              // nullish coalescing: valore di riserva se null o undefined
if (scelto) { scelto.nome; }             // dentro l'if TypeScript sa che c'è (narrowing)

@ViewChild('campo') campo!: ElementRef; // ! = "fidati, arriverà": da usare con parsimonia
```

Con strict attivo TypeScript non ti lascia leggere un campo di un valore che potrebbe mancare. È la causa dell'errore più frequente all'inizio: Object is possibly 'undefined'. La soluzione giusta è quasi sempre controllare (if, @if nel template, ?.), non mettere ! ovunque.

Funzioni e arrow function

```
function somma(a: number, b: number): number {
  return a + b;
}

const doppio = (n: number) => n * 2;           // arrow function, ritorno implicito
const saluta = (nome = 'ospite') => `Ciao ${nome}`; // parametro con valore predefinito e template string

[1, 2, 3].map((n) => n * 2);                    // [2, 4, 6]
[1, 2, 3].filter((n) => n > 1);                 // [2, 3]
[1, 2, 3].reduce((tot, n) => tot + n, 0);       // 6
eroi.find((e) => e.id === 3);                  // il primo che soddisfa la condizione, o undefined
eroi.some((e) => e.eroe);                      // true se almeno uno soddisfa
```

Le arrow function non hanno un proprio this: usano quello del punto in cui sono scritte. Per questo in Angular si usano nei subscribe e nei callback, dove this deve restare il componente. Le template string con i backtick (`) inseriscono valori con \${...}.

Classi

```
export class Contatore {
  private valore = 0;           // visibile solo dentro la classe
  protected passo = 1;         // visibile anche nel template del componente
  readonly nome: string;

  constructor(nome: string) {
    this.nome = nome;
  }

  incrementa(): void {
    this.valore += this.passo;
  }

  get corrente(): number {      // getter: si legge come una proprietà, contatore.corrente
    return this.valore;
  }
}

// Parametri del constructor come proprietà: la forma breve usata nei vecchi servizi Angular
constructor(private servizio: Eroeservizio) {}
```

Ogni componente, servizio, direttiva e pipe di Angular è una classe. private nasconde un membro al resto del codice e anche al template; nei componenti usa protected (o nessun modificatore) per ciò che il template deve leggere.

Destrutturazione e spread

```
const { nome, superpotere } = eroe;           // estrae due campi in due costanti
const { id, ...dati } = eroe;                // id da una parte, tutto il resto in dati
const [primo, secondo] = eroi;               // dagli array per posizione

const copia = { ...eroe, nome: 'Nuovo nome' }; // copia l'oggetto cambiando un campo
const lista2 = [...lista, nuovoEroe];         // nuovo array con un elemento in più
const senza = lista.filter((e) => e.id !== 3); // nuovo array senza un elemento
```

Lo spread (...) è fondamentale con i signal: per aggiornare un oggetto o un array si crea una copia nuova invece di modificare quello esistente. Il perché è nel capitolo 6.

Moduli: import ed export

```
// eroe.ts
export interface Eroe { id: number; nome: string; }
export const EROI_FINTI: Eroe[] = [];

// eroi.ts
import { Eroe, EROI_FINTI } from './eroe';           // file del progetto: percorso relativo
import { Component, signal } from '@angular/core';   // libreria: nome del pacchetto
```

Ogni file è un modulo. Quello che non è esportato con export resta privato del file. Nel percorso relativo non si scrive l'estensione .ts; ./ è la cartella attuale, ../ quella sopra. L'editor aggiunge gli import da solo se accetti il suggerimento: è il modo più sicuro per non sbagliare i percorsi.

Generics

```
const nomi: Array<string> = ['a', 'b'];           // uguale a string[]
const scelto = signal<Eroe | null>(null);           // il signal conterrà un Eroe oppure null
this.http.get<Eroe[]>(url);                        // la risposta sarà un array di Eroe

function primo<T>(lista: T[]): T | undefined {
  return lista[0];
}
primo([1, 2, 3]); // T = number
primo(['a', 'b']); // T = string
```

Le parentesi angolari <T> passano un tipo come parametro. In Angular le vedi soprattutto in signal, input, output, HttpClient, FormControl e Observable: dicono cosa conterrà quell'oggetto.

Codice asincrono: Promise, async/await e Observable

```
async function carica() {
  const risposta = await fetch('/api/eroi'); // aspetta senza bloccare la pagina
  const eroi = await risposta.json();
  return eroi;
}

setTimeout(() => console.log('dopo 1 secondo'), 1000);
```

Molte operazioni non finiscono subito: una richiesta al server, un timer. JavaScript non si ferma ad aspettare: la riga dopo parte subito, e il risultato arriva più tardi in un callback. Una Promise rappresenta un valore che arriverà una volta sola; async/await permette di scriverla come codice in sequenza. Angular usa soprattutto gli Observable di RxJS, che sono simili ma possono emettere più valori nel tempo e si possono annullare: sono spiegati nel capitolo 11.

Decoratori

```
@Component({ selector: 'app-eroi', templateUrl: './eroi.html' })
export class Eroi {}

@Injectable({ providedIn: 'root' })
export class Eroeservizio {}
```

Un decoratore è la @ con un nome, scritta sopra una classe o una proprietà. Aggiunge informazioni che Angular legge: @Component dice che la classe è un componente e qual è il suo template, @Injectable che è un servizio. Sulle proprietà trovi @Input, @Output, @ViewChild, che oggi hanno le alternative a funzione input(), output(), viewChild().

Errori di TypeScript frequenti

Type 'X' is not assignable to type 'Y' Stai mettendo un valore del tipo sbagliato. Leggi quale tipo arriva e quale serve.

Object is possibly 'undefined' / 'null' Il valore potrebbe mancare. Controllalo con if, ?. o @if.

Property 'x' does not exist on type 'Y' Nome sbagliato, oppure l'interfaccia non ha quel campo.

Property 'x' has no initializer and is not definitely assigned in the constructor Una proprietà senza valore iniziale. Dagli un valore, rendila facoltativa con ?, oppure usa ! se arriva davvero dopo (ViewChild).

Cannot find name 'X' Manca l'import, o il nome è scritto male.

Argument of type 'string' is not assignable to parameter of type 'number' Un parametro del tipo sbagliato: spesso un id preso dall'URL, che è sempre una stringa. Convertilo con Number().

3. La CLI di Angular

Comandi principali

La CLI (Command Line Interface) è il comando ng. Crea progetti e file, avvia l'app, esegue test e build. Tra parentesi la forma breve del comando.

- ng new nome (n)** Crea un nuovo progetto.
- ng serve (s)** Avvia l'app in locale e ricompila al salvataggio. -o apre il browser.
- ng build (b)** Compila per la pubblicazione nella cartella dist/.
- ng test (t)** Esegue i test unitari (Vitest in Angular 21). --watch=false li esegue una volta sola.
- ng e2e (e)** Avvia l'app e lancia i test end-to-end, se configurati.
- ng lint** Controlla stile ed errori del codice, se è installato un linter.
- ng generate (g)** Crea file: componenti, servizi, ecc.
- ng add libreria** Aggiunge e configura una libreria, per esempio ng add @angular/material.
- ng update** Aggiorna Angular e le dipendenze alla versione successiva.
- ng run target** Esegue un'operazione definita in angular.json.
- ng config** Legge o modifica angular.json.
- ng cache** Gestisce la cache su disco della CLI: ng cache clean la svuota.
- ng deploy** Pubblica con il builder di deploy, se installato.
- ng extract-i18n** Estrae i testi da tradurre.
- ng analytics** Attiva o disattiva l'invio di dati d'uso anonimi.
- ng completion** Attiva l'autocompletamento dei comandi nel terminale.
- ng version (v)** Mostra le versioni di CLI e Angular.
- ng help** Aiuto. Anche su un comando: ng build --help.

Script npm

Gli script sono in package.json, sezione scripts. Si lanciano con npm run nome (start e test anche senza run).

- npm start** Come ng serve.
- npm run build** Come ng build.
- npm run watch** Ricompila in continuo in modalità sviluppo.
- npm test** Come ng test.
- npm run serve:ssr:corso-angular** Avvia la build con rendering lato server. Prima: npm run build.
- npm run firebase-finto** Solo in corso-angular: avvia il server che imita Firebase (capitolo 15).

ng generate

- ng g component nome (c)** Componente: classe, HTML, stile e test.
- ng g service nome (s)** Servizio: logica riusabile e dati.
- ng g directive nome (d)** Direttiva: estende un elemento HTML.
- ng g pipe nome (p)** Pipe: trasforma un valore nel template.
- ng g guard nome (g)** Guardia: controlla l'accesso a una rotta.

`ng g interceptor nome` Interceptor HTTP: modifica richieste e risposte.

`ng g resolver nome (r)` Resolver: carica i dati prima della rotta.

`ng g module nome (m)` Crea un NgModule. Nei progetti standalone di solito non serve.

`ng g class nome (cl)` Classe TypeScript.

`ng g interface nome (i)` Interfaccia TypeScript.

`ng g enum nome (e)` Enum: insieme di costanti con nome.

`ng g application nome (app)` Seconda app nello stesso workspace.

`ng g library nome (lib)` Libreria riusabile nel workspace.

`ng g environments` File di ambiente per sviluppo e produzione.

`ng g config tipo` File di configurazione di build o test.

`ng g app-shell` App shell statica mostrata al primo caricamento.

`ng g service-worker` Service worker: cache e uso offline.

`ng g web-worker nome` Web worker: codice pesante in background.

`ng g ai-config` Configurazione per gli assistenti di intelligenza artificiale, con le buone pratiche di Angular.

Il nome può contenere un percorso: `ng g component pagine/dettaglio` crea `src/app/pagine/dettaglio/dettaglio.ts`, `.html`, `.scss` e `.spec.ts`. Da Angular 20 i file non hanno più il suffisso: `dettaglio.ts` con la classe `Dettaglio`, non `dettaglio.component.ts` con `DettaglioComponent`. Nei progetti vecchi troverai la forma con il suffisso.

Opzioni di generate

`--dry-run (-d)` Mostra i file che verrebbero creati, senza scriverli.

`--defaults` Usa i valori predefiniti, senza fare domande.

`--force` Sovrascrive i file già esistenti.

`--skip-tests` Non crea il file `.spec.ts`.

`--inline-template (-t) / --inline-style (-s)` Template e stile dentro il file `.ts` invece che in file separati.

4. Componenti e template

Basi da ricordare

Angular mostra una pagina partendo da classi TypeScript. I dati stanno nella classe. Il file HTML li disegna. Lo stile sta nel file scss.

Ordine mentale: i dati stanno nella classe o in un servizio. Il template li mostra. Un click chiama un metodo. Il figlio riceve input e, se deve rispondere, manda un output.

Un componente

classe .ts Dati e funzioni. Esempio: eroi, eroeSelezionato, onSelect.

template .html Quello che vedi nel browser. Legge le proprietà della classe per nome.

stile .scss L'aspetto di quel componente, solo suo.

selector Il nome del tag. selector: 'app-eroi' si usa come <app-eroi> nel padre.

imports Elenco di pipe, direttive e componenti che il template può usare. Se manca una voce, la compilazione si ferma.

```
import { Component, signal } from '@angular/core';
import { UpperCasePipe } from '@angular/common';
import { Eroee } from '../eroe';

@Component({
  selector: 'app-eroi',
  imports: [UpperCasePipe],
  templateUrl: './eroi.html',
  styleUrls: ['./eroi.scss'],
})
export class Eroee {
  titolo = 'I miei eroi';
  eroi = signal<Eroee[]>([
    { id: 1, nome: 'Superman', superpotere: 'volo', eroe: true },
    { id: 2, nome: 'Joker', superpotere: 'caos', eroe: false },
  ]);
  scelto = signal<Eroee | undefined>(undefined);

  onSelect(eroe: Eroee) {
    this.scelto.set(eroe);
  }
}

<!-- eroi.html -->
<h2>{{ titolo | uppercase }}</h2>
<ul>
  @for (eroe of eroi(); track eroe.id) {
    <li [class.selezionato]="scelto()?.id === eroe.id" (click)="onSelect(eroe)">
      {{ eroe.nome }}
    </li>
  }
</ul>
@if (scelto(); as e) {
  <p>Hai scelto {{ e.nome }}: {{ e.superpotere }}</p>
}
```

Per template molto corti si può scrivere il template direttamente nella classe con template: `...` invece di templateUrl, e lo stile con styles: `...`.

Leggere e scrivere nel template

{{ nome }} Mostra un valore. Solo lettura.

[prop]="valore" Dal componente verso il tag. Esempio: [class.selezionato]="eroeSelezionato === eroe".

`(click)="metodo()"` Dal tag verso il componente. Quando succede l'evento, parte la funzione.

`[(ngModel)]="campo"` Legge e scrive insieme. Serve FormsModule dentro imports.

`{{ title() }}` Le parentesi leggono il valore di un signal. Senza parentesi stampi il contenitore.

Data binding: i quattro modi

Il data binding è il collegamento tra i dati della classe e ciò che si vede. Può andare in un senso solo (one-way: dalla logica alla vista, oppure dalla vista alla logica) o in entrambi (two-way).

1) Interpolazione, dal codice all'utente: `{{ espressione }}` mostra un valore come testo.

```
<p>Ciao {{ nome }}, hai {{ eroi().length }} eroi. Totale: {{ prezzo * quantita }}</p>
```

2) Property binding, dal codice all'utente: `[proprietà]="espressione"` imposta una proprietà di un elemento, non il suo testo. Serve per cambiare una classe CSS, un'immagine, disattivare un bottone in base ai dati.

```
<img [src]="eroe.foto" [alt]="eroe.nome">
<button [disabled]="salvataggio()">Salva</button>
<div [class.attivo]="eroe.eroe" [style.color]="eroe.eroe ? 'green' : 'red'">...</div>
<td [attr.colspan]="colonne.length">...</td>
```

3) Event binding, dall'utente al codice: `(evento)="metodo($event)"` chiama un metodo quando succede qualcosa: click, input, submit, keyup.enter.

```
<button (click)="salva()">Salva</button>
<input (input)="onInput($event)">
<input (keyup.enter)="cerca()">
```

`$event` è l'oggetto dell'evento del browser. Per leggere il testo di un input: `($any($event.target)).value`, oppure meglio un riferimento `#campo` e `campo.value`.

4) Two-way binding: `[(ngModel)]="campo"` unisce property binding ed event binding. Il valore va dalla classe al campo e torna indietro quando l'utente scrive. Richiede FormsModule negli imports. Sui tuoi componenti si ottiene con `model()` (capitolo 9).

```
<input [(ngModel)]="nome">
<p>Stai scrivendo: {{ nome }}</p>
```

`[( )]` si chiama "banana in a box": la banana () dentro la scatola []. L'ordine giusto è `[( )]`, non `[ ( )]`.

Stili del componente

Lo stile nel file .scss di un componente vale solo per quel componente: Angular aggiunge agli elementi un attributo nascosto e riscrive i selettori, così una regola `p { color: red }` non tocca i paragrafi degli altri componenti. Si chiama incapsulamento degli stili.

:host Il tag del componente stesso, per esempio `<app-eroi>`. `:host { display: block; padding: 16px; }`

:host(.attivo) Il tag del componente quando ha la classe attivo.

styles.scss Gli stili globali, validi ovunque: font, colori, reset.

::ng-deep Rompe l'incapsulamento e raggiunge i figli. Sconsigliato: preferisci una classe globale.

[class.nome]="cond" Aggiunge o toglie una classe.

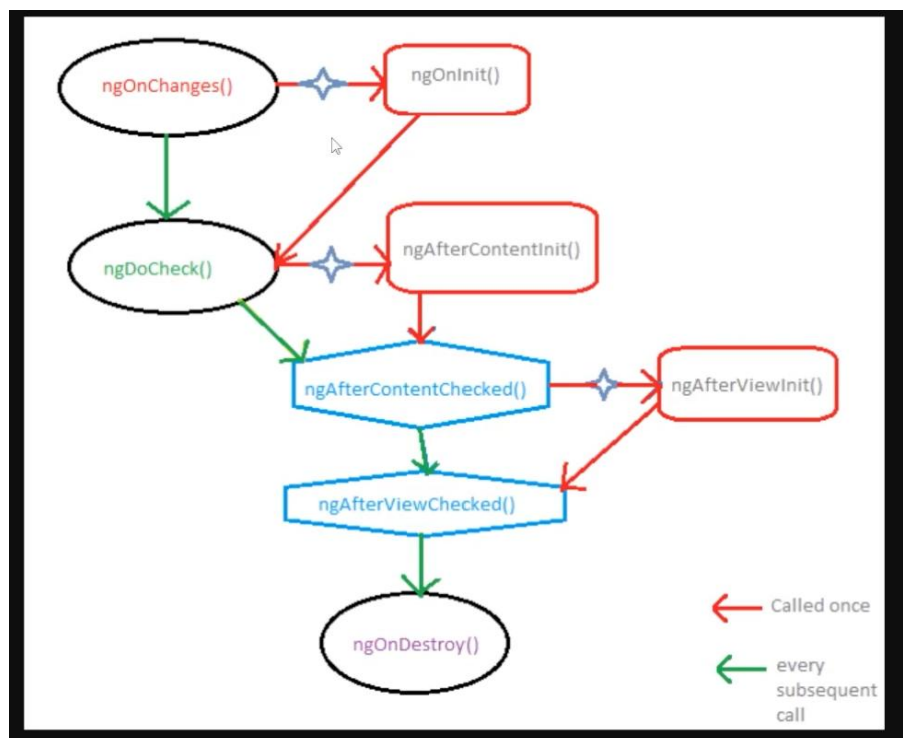
[class]="{ attivo: a, errore: e }" Più classi insieme con un oggetto.

[style.width.px]="larghezza" Uno stile con unità di misura.

Di base un componente si comporta come un elemento inline: se ti servono margini o larghezza, metti `:host { display: block; }`.

Il ciclo di vita di un componente

Angular crea il componente, gli passa gli input, disegna il template, lo aggiorna quando i dati cambiano e infine lo distrugge. In alcuni momenti chiama dei metodi della classe, se li hai scritti: sono gli hook del ciclo di vita.



Le frecce rosse sono chiamate una sola volta, quelle verdi a ogni controllo successivo.

constructor Prima di tutto. Qui si prendono i servizi con inject. Gli input non sono ancora arrivati.

ngOnChanges(changes) Ogni volta che cambia un @Input. changes dice il valore vecchio e nuovo. Con input() signal non serve: usa computed o effect.

ngOnInit Una volta, dopo il primo arrivo degli input. Il posto classico per caricare i dati.

ngDoCheck A ogni controllo dei cambiamenti. Si usa raramente.

ngAfterContentInit / ngAfterContentChecked Dopo che il contenuto proiettato con ng-content è pronto.

ngAfterViewInit Una volta, quando il template e i figli sono disegnati. Da qui @ViewChild e viewChild() hanno un valore.

ngAfterViewChecked Dopo ogni aggiornamento della vista.

ngOnDestroy Prima di togliere il componente dalla pagina. Qui si fermano timer, abbonamenti e ascoltatori.

```

import { AfterViewInit, Component, DestroyRef, OnDestroy, OnInit, inject, signal } from '@angular/core';

export class Orologio implements OnInit, AfterViewInit, OnDestroy {
  private destroyRef = inject(DestroyRef);
  ora = signal(new Date());
  private timer?: ReturnType<typeof setInterval>;

  constructor() {
    console.log('1 constructor');
    // Alternativa moderna a ngOnDestroy, utilizzabile ovunque ci sia inject:
    this.destroyRef.onDestroy(() => console.log('distrutto (DestroyRef)'));
  }

  ngOnInit() {
    console.log('2 ngOnInit: carico i dati, avvio il timer');
    this.timer = setInterval(() => this.ora.set(new Date()), 1000);
  }

  ngAfterViewInit() {
    console.log('3 ngAfterViewInit: il template esiste');
  }
}
  
```

```
}  
  
ngOnDestroy() {  
  console.log('4 ngOnDestroy: fermo il timer');  
  clearInterval(this.timer);  
}  
}
```

implements OnInit, ... non è obbligatorio, ma fa controllare a TypeScript che il nome del metodo sia scritto giusto. Nella pratica si usano quasi sempre solo ngOnInit, ngAfterViewInit e ngOnDestroy (o DestroyRef). Con SSR, il codice che tocca il DOM o window va in afterNextRender(() => ...), che gira solo nel browser.

Come si aggiorna la pagina: change detection e zoneless

Quando un dato cambia, Angular ricontrolla i template e aggiorna solo le parti cambiate: è la change detection. Fino ad Angular 19 una libreria chiamata zone.js si accorgeva di ogni evento, timer e risposta HTTP e faceva ripartire il controllo. Le app nuove di Angular 21, come corso-angular, sono zoneless: senza zone.js. Angular aggiorna la pagina quando:

- cambia un signal letto nel template (set, update, computed);
- succede un evento ascoltato nel template, come (click) o (input);
- arriva un nuovo valore a un input del componente;
- la pipe async riceve un valore.

Conseguenza pratica: se modifichi una proprietà normale dentro un setTimeout o dentro il subscribe di una chiamata HTTP, la pagina potrebbe non aggiornarsi. Usa i signal per ogni dato che cambia dopo un'attesa. È il motivo per cui gli esempi di questo corso usano signal per liste, caricamento ed errori.

5. Control flow e direttive standard

Tre tipi di direttive

Una direttiva è una classe che aggiunge comportamento a un elemento del template. Ce ne sono tre tipi:

- Componenti: direttive con un template proprio. Ogni `<app-...>` è un componente.
- Direttive di attributo: cambiano aspetto o comportamento di un elemento esistente. Esempi: `ngClass`, `ngStyle`, `ngModel`.
- Direttive strutturali: cambiano il DOM aggiungendo o togliendo elementi. Esempi: `*ngIf`, `*ngFor`. Si riconoscono dall'asterisco.

Da Angular 17 le strutturali più usate sono state sostituite dal control flow: `@if`, `@for`, `@switch`. Fanno lo stesso lavoro, sono parte del linguaggio dei template e non vanno importate.

@if, @else

```
@if (utente(); as u) {
  <p>Ciao {{ u.nome }}</p>
} @else if (caricamento()) {
  <p>Caricamento...</p>
} @else {
  <a routerLink="/login">Accedi</a>
}
```

`as u` dà un nome al valore controllato: dentro il blocco `u` esiste di sicuro, senza bisogno di ?.

@for, track, @empty

```
<ul>
  @for (eroe of eroi(); track eroe.id; let i = $index, ultimo = $last) {
    <li>{{ i + 1 }}. {{ eroe.nome }} @if (ultimo) { (ultimo) }</li>
  } @empty {
    <li>Nessun eroe.</li>
  }
</ul>
```

`track` è obbligatorio: dice ad Angular come riconoscere ogni elemento, così quando la lista cambia sposta o aggiorna solo gli elementi giusti invece di ridisegnarli tutti. Usa un id univoco; se non c'è, `track $index`. Variabili disponibili: `$index`, `$first`, `$last`, `$even`, `$odd`, `$count`.

@switch

```
@switch (livello()) {
  @case ('base') { <span>Principiante</span> }
  @case ('avanzato') { <span>Esperto</span> }
  @default { <span>Intermedio</span> }
}
```

@defer: caricare più tardi

```
@defer (on viewport) {
  <app-grafico-pesante />
} @placeholder {
  <p>Il grafico apparirà qui</p>
} @loading (minimum 300ms) {
  <p>Caricamento del grafico...</p>
} @error {
  <p>Impossibile caricare il grafico</p>
}
```

Il codice del componente dentro `@defer` viene scaricato solo quando serve: on viewport (quando entra nello schermo), on idle (predefinito, quando il browser è libero), on interaction (al primo click), on hover, on timer(2s), when condizione. Alleggerisce il primo caricamento della pagina.

@let: variabili nel template

```
@let totale = carrello.totale();
@let cap = form.controls.indirizzo.controls.cap;
<p>Totale: {{ totale | currency:'EUR' }}</p>
```

Riepilogo del control flow

- @if** Mostra un pezzo di pagina se la condizione è vera.
- @else** Pezzo alternativo quando @if è falso.
- @else if** Altra condizione, dopo @if.
- @for** Ripete un pezzo di pagina per ogni elemento di una lista.
- @empty** Contenuto mostrato se la lista di @for è vuota.
- @switch** Sceglie un blocco in base a un valore.
- @case** Uno dei casi di @switch.
- @default** Caso di @switch se nessun @case coincide.
- @defer** Carica un pezzo di pagina più tardi, quando serve.
- @placeholder** Cosa mostrare prima che @defer parta.
- @loading** Cosa mostrare mentre @defer sta caricando.
- @error** Cosa mostrare se il caricamento di @defer fallisce.
- @let** Crea una variabile usabile nel template.
- ngNonBindable** In quel punto Angular non interpreta il template: utile per mostrare {{ }} come testo.

La sintassi vecchia: *ngIf e *ngFor

Nel codice esistente trovi ancora le direttive strutturali. Vanno importate (NgIf, NgFor da @angular/common).

```
<p *ngIf="isOnline()">isVisible, modalità 1</p>

<ng-template [ngIf]="isOnline()" [ngIfElse]="bloccoElse">
  <p>isVisible, modalità 2</p>
</ng-template>
<ng-template #bloccoElse>
  <p>isNotVisible, modalità 2</p>
</ng-template>

<li *ngFor="let persona of persone(); let i = index; trackBy: perId">
  {{ persona.nome }} - {{ persona.cognome }}
</li>
```

L'asterisco è una scorciatoia: `*ngIf="x"` diventa un `<ng-template [ngIf]="x">` intorno all'elemento. `<ng-template>` è un pezzo di HTML che non viene mostrato finché qualcuno non lo usa; `<ng-container>` è un contenitore che non produce nessun tag nella pagina, utile per raggruppare senza aggiungere div.

Per convertire un progetto al nuovo control flow c'è un comando: `ng generate @angular/core:control-flow`.

Direttive di @angular/common

Si importano nel componente e si mettono in imports, come le pipe.

ngClass (NgClass) Aggiunge o toglie classi CSS. `[ngClass]="{ attivo: a, errore: e }"`. Oggi basta `[class]`.

ngStyle (NgStyle) Imposta gli stili CSS. Oggi basta [style].

ngIf (NgIf) Versione vecchia di @if. Mostra o nasconde un elemento.

ngFor (NgFor) Versione vecchia di @for. Ripete un elemento.

ngSwitch (NgSwitch) Versione vecchia di @switch.

ngSwitchCase (NgSwitchCase) Un caso di ngSwitch.

ngSwitchDefault (NgSwitchDefault) Caso predefinito di ngSwitch.

ngPlural (NgPlural) Sceglie un elemento in base a un numero.

ngPluralCase (NgPluralCase) Un caso di ngPlural.

ngTemplateOutlet (NgTemplateOutlet) Inserisce un ng-template in quel punto.

ngComponentOutlet (NgComponentOutlet) Crea un componente in quel punto, scelto da codice.

ngSrc (NgOptimizedImage) Carica un'immagine in modo ottimizzato: dimensioni, lazy loading, priorità.

Direttive di @angular/forms

ngModel (NgModel) Collega un input a un valore del componente. Capitolo 13.

ngForm (NgForm) Fa gestire il form ad Angular.

ngModelGroup (NgModelGroup) Raggruppa più campi con ngModel.

formGroup (FormGroupDirective) Collega un form a un FormGroup. Capitolo 14.

formControlName (FormControlName) Collega un input a un controllo del gruppo.

formControl (FormControlDirective) Collega un input a un FormControl.

formArrayName (FormArrayName) Collega un gruppo di campi a un FormArray.

Variabili di riferimento nel template

```
<input #campo placeholder="Nome">
<button (click)="saluta(campo.value)">Saluta</button>
<p>Hai scritto {{ campo.value.length }} caratteri</p>
```

#campo dà un nome a un elemento: lo puoi usare ovunque nello stesso template. Senza valore è l'elemento HTML; con un valore (#f="ngForm", #m="ngModel") è la direttiva indicata. Per usarlo nella classe si prende con viewChild (capitolo 9).

6. Signal

Cos'è un signal

Un signal è un contenitore di un valore che avvisa quando cambia. Il template che lo legge si aggiorna da solo, e i valori calcolati da lui si ricalcolano da soli. Nelle app zoneless di Angular 21 i signal sono il modo principale per tenere lo stato.

signal(valore) Contenitore. Quando cambia, la pagina si aggiorna da sola.

nome() Legge il valore. Nel template: `{{ nome() }}`.

.set(nuovo) Sostituisce il valore.

.update(n => n + 1) Calcola il valore nuovo partendo da quello attuale.

computed(() => ...) Valore calcolato da altri signal. Si ricalcola da solo. Non si può fare set.

effect(() => ...) Esegue codice ogni volta che cambiano i signal che legge. Per effetti collaterali.

.asReadonly() Versione di sola lettura, da esporre fuori da un servizio.

DestroyRef.onDestroy Quando il componente viene tolto, spegni timer e abbonamenti.

```
import { Component, computed, effect, signal } from '@angular/core';

export class Carrello {
  prezzo = signal(10);
  quantita = signal(1);
  totale = computed(() => this.prezzo() * this.quantita());
  costoso = computed(() => this.totale() > 100);

  constructor() {
    effect(() => console.log('Totale cambiato:', this.totale()));
  }

  aggiungi() { this.quantita.update((q) => q + 1); }
  azzera() { this.quantita.set(0); }
}

<p>{{ quantita() }} x {{ prezzo() }} € = {{ totale() }} €</p>
@if (costoso()) { <p>Spedizione gratuita!</p> }
<button (click)="aggiungi()">+1</button>
```

Leggere con le parentesi

`{{ title() }}` stampa il testo; `{{ title }}` stampa il contenitore, cioè qualcosa come `[Signal: ...]`. È uno degli errori più frequenti. L'unica eccezione è il two-way binding: `[(ngModel)]="nome"` e `[(valore)]="quantita"` vogliono il signal senza parentesi, perché Angular deve poterci scrivere.

computed: valori derivati

`computed` ricorda il risultato e lo ricalcola solo quando cambia uno dei signal che ha letto l'ultima volta. È pigro: se nessuno lo legge, non calcola. Usalo per tutto ciò che si ottiene da altri dati: totali, filtri, liste ordinate, messaggi. Evita di mettere logica complicata nel template: spostala in un `computed` con un nome chiaro.

Attenzione: `computed` si accorge solo dei signal. Se dentro usi `Date.now()` o una proprietà normale, non si ricalcola quando quelle cambiano. Per questo nel capitolo 16 `isLoggato()` è un metodo e non un `computed`.

Aggiornare oggetti e array: sempre una copia nuova

```
lista = signal<Eroe[]>([]);

// SBAGLIATO: modifica lo stesso array, il signal non vede il cambio
this.lista().push(nuovo);
this.lista.update((l) => { l.push(nuovo); return l; });

// GIUSTO: un array nuovo
this.lista.update((l) => [...l, nuovo]); // aggiungi
this.lista.update((l) => l.filter((e) => e.id !== id)); // toglie
this.lista.update((l) => l.map((e) => (e.id === id ? { ...e, nome } : e))); // cambia uno

utente = signal({ nome: 'Mario', eta: 30 });
this.utente.update((u) => ({ ...u, eta: u.eta + 1 }));
```

Un signal confronta il valore vecchio e il nuovo con `Object.is`: se è lo stesso oggetto, per lui non è cambiato niente e non avvisa nessuno. Per questo si crea sempre un oggetto o un array nuovo con `spread`, `map` e `filter`.

effect: effetti collaterali

```
constructor() {
  effect(() => {
    localStorage.setItem('carrello', JSON.stringify(this.articoli()));
  });
}
```

`effect` gira una volta subito e poi ogni volta che cambia un signal che legge. Serve per sincronizzare con il mondo esterno: `localStorage`, `log`, una libreria grafica. Non usarlo per calcolare un valore da mostrare (quello è `computed`) né per copiare un signal in un altro. Si crea in un contesto di iniezione, di solito nel constructor, e si ferma da solo quando il componente viene distrutto.

`untracked(() => ...)` legge un signal dentro un `effect` o un `computed` senza diventarne dipendente: utile quando vuoi usare un valore senza che i suoi cambi facciano ripartire l'effetto.

linkedSignal: un valore modificabile che si riallinea

```
import { linkedSignal } from '@angular/core';

opzioni = signal(['Base', 'Intermedio', 'Avanzato']);
scelta = linkedSignal(() => this.opzioni()[0]); // parte dalla prima opzione

this.scelta.set('Avanzato'); // l'utente sceglie
this.opzioni.set(['Uno', 'Due']); // le opzioni cambiano: scelta torna a 'Uno'
```

`linkedSignal` è come `computed`, ma si può anche fare `set`. Quando cambiano i signal da cui dipende, torna al valore calcolato. Serve per le selezioni che devono ripartire quando cambia la lista.

Signal nei componenti e negli altri capitoli

Molte API di Angular restituiscono signal:

- `input()`, `input.required()` Gli input di un componente. Capitolo 9.
- `model()` Input modificabile, per il two-way binding. Capitolo 9.
- `viewChild()`, `contentChild()` Riferimenti a elementi del template. Capitolo 9.
- `toSignal(observable)` Trasforma un Observable in un signal. Capitolo 11.
- `httpResource()` Una chiamata HTTP con i signal `value`, `isLoading`, `error`. Capitolo 15.

Signal o proprietà normale?

Una proprietà normale va bene per valori che non cambiano (una lista di opzioni fissa, un titolo). Usa un signal per tutto ciò che cambia mentre la pagina è aperta, soprattutto se cambia dopo un'attesa: dati arrivati dal server, stato di caricamento, messaggi d'errore, selezioni. Nelle app zoneless è l'unico modo sicuro perché la pagina si aggiorni.

7. Pipe

Cos'è una pipe

Una pipe trasforma un valore solo nel template. Il dato nel componente non cambia. Sintassi: valore | nomePipe. I parametri vanno dopo i due punti. Si possono concatenare: {{ nome | slice:0:10 | uppercase }}. In un componente standalone la classe della pipe va nell'array imports.

Pipe standard

Sono in @angular/common. Nel template si usa il nome corto. Nel componente si importa la classe e la si mette in imports.

- async** (**AsyncPipe**) Attende un Observable o una Promise e mostra il valore arrivato.
- uppercase** (**UpperCasePipe**) Mette il testo tutto in maiuscolo.
- lowercase** (**LowerCasePipe**) Mette il testo tutto in minuscolo.
- titlecase** (**TitleCasePipe**) Mette la maiuscola all'inizio di ogni parola.
- date** (**DatePipe**) Formatta una data.
- number** (**DecimalPipe**) Formatta un numero con i decimali.
- percent** (**PercentPipe**) Mostra un numero come percentuale.
- currency** (**CurrencyPipe**) Formatta un importo in valuta.
- slice** (**SlicePipe**) Prende una porzione di stringa o di array.
- json** (**JsonPipe**) Stampa un oggetto in formato JSON. Utilissima per il debug.
- keyvalue** (**KeyValuePipe**) Trasforma un oggetto in elenco di chiave e valore, da usare con @for.
- i18nPlural** (**I18nPluralPipe**) Sceglie la frase in base al numero, singolare o plurale.
- i18nSelect** (**I18nSelectPipe**) Sceglie la frase in base a un valore.

Esempi: testo

I risultati delle date qui sotto usano la lingua predefinita en-US. Il valore di oggi è il 28 settembre 2026 alle 15:45:30.

{{ 'testo da trasformare' uppercase }}	TESTO DA TRASFORMARE
{{ 'testo da trasformare' lowercase }}	testo da trasformare
{{ 'testo da trasformare' titlecase }}	Testo Da Trasformare
{{ 'testo da trasformare' slice:0:5 }}	testo (dall'indice 0 fino al 5 escluso)
{{ 'testo da trasformare' slice:5 }}	da trasformare (dall'indice 5 alla fine)

Esempi: date (DatePipe)

dd giorno, MM mese, yyyy anno, HH ore da 00 a 23, mm minuti, ss secondi, EEEE nome del giorno, MMMM nome del mese. I nomi short, medium, long e full sono formati già pronti; con Date e Time mostrano solo la data o solo l'ora.

```

{{ oggi | date:'dd/MM/yyyy HH:mm' }} 28/09/2026 15:45
{{ oggi | date:'short' }} 9/28/26, 3:45 PM
{{ oggi | date:'medium' }} Sep 28, 2026, 3:45:30 PM
{{ oggi | date:'long' }} September 28, 2026 at 3:45:30 PM GMT+2
{{ oggi | date:'full' }} Monday, September 28, 2026 at 3:45:30 PM (fuso per esteso)
{{ oggi | date:'shortDate' }} 9/28/26
{{ oggi | date:'mediumDate' }} Sep 28, 2026
{{ oggi | date:'longDate' }} September 28, 2026
{{ oggi | date:'fullDate' }} Monday, September 28, 2026
{{ oggi | date:'shortTime' }} 3:45 PM
{{ oggi | date:'mediumTime' }} 3:45:30 PM
{{ oggi | date:'longTime' }} 3:45:30 PM GMT+2
{{ oggi | date:'fullTime' }} 3:45:30 PM (fuso per esteso)

```

Esempi: numero, percentuale, valuta

importo vale 1234.5, quota vale 0.25.

```

{{ importo | number:'1.2-2' }} 1,234.50
{{ quota | percent:'1.0-0' }} 25%
{{ importo | currency:'EUR':'symbol':'1.2-2':'it' }} 1.234,50 €

```

'1.2-2' significa almeno 1 cifra intera, da 2 a 2 decimali. PercentPipe moltiplica per 100. I parametri di currency sono: codice valuta, come mostrarla (symbol, code), decimali, lingua.

Formati in italiano: registrare la lingua

Per usare 'it' come lingua, nel parametro delle pipe o per tutta l'app, Angular deve avere i dati della lingua italiana (nomi dei mesi, separatori). Senza, l'errore è NG0701: Missing locale data for the locale "it". Nel progetto è successo davvero con la pagina delle pipe, scoperto dal suo test.

```

// app.config.ts: italiano per tutta l'app
import { LOCALE_ID } from '@angular/core';
import { registerLocaleData } from '@angular/common';
import localeIt from '@angular/common/locales/it';

registerLocaleData(localeIt);

export const appConfig: ApplicationConfig = {
  providers: [
    { provide: LOCALE_ID, useValue: 'it' },
    // ...
  ],
};

```

Con LOCALE_ID 'it' le pipe usano l'italiano senza parametri: {{ oggi | date:'fullDate' }} diventa lunedì 28 settembre 2026, {{ importo | currency:'EUR' }} diventa 1.234,50 €.

Frasi in base al valore

Non caricano un file di lingua. Nel componente c'è una mappa: a ogni valore corrisponde una frase. other è il caso di riserva.

```

genere = 'donna';
frasiGenere = { uomo: 'è un uomo', donna: 'è una donna', other: 'non è indicato' };
quantita = 1;
frasiQuantita = { '=0': 'nessun elemento', '=1': 'un elemento', other: '# elementi' };

{{ genere | i18nSelect:frasiGenere }} è una donna
{{ quantita | i18nPlural:frasiQuantita }} un elemento

```

=0 e =1 sono i casi esatti, other gli altri. # diventa il numero.

Creare una pipe personalizzata

```
ng g pipe pipes/troncato
```

src/app/pipes/troncato.ts nel progetto corso-angular:

```
import { Pipe, PipeTransform } from '@angular/core';

// Uso nel template: {{ descrizione | troncato }} oppure {{ descrizione | troncato:30:'...' }}
@Pipe({ name: 'troncato' })
export class Troncato implements PipeTransform {
  transform(valore: string | null | undefined, massimo = 20, fine = '...'): string {
    if (!valore) return '';
    if (valore.length <= massimo) return valore;
    return valore.slice(0, massimo).trimEnd() + fine;
  }
}

imports: [Troncato],

<td [title]="l.descrizione">{{ l.descrizione | troncato:30 }}</td>
```

name Il nome usato nel template dopo la barra.

transform Riceve il valore a sinistra della barra e, in ordine, i parametri dopo i due punti. Restituisce il valore da mostrare.

parametri con valore predefinito massimo = 20 permette di scrivere solo | troncato.

pura Di base una pipe è pura: Angular la richiama solo quando cambia il valore in ingresso o un parametro. È veloce. Con pure: false verrebbe richiamata a ogni controllo: da evitare.

La pipe è usata nella pagina /material, nella colonna Descrizione. Il suo test, troncato.spec.ts, è nel capitolo 18.

Pipe o computed? Se la trasformazione serve in molti template diversi, fai una pipe. Se serve in un componente solo, un computed nella classe è più semplice.

8. Direttive personalizzate

Una direttiva di attributo

Una direttiva di attributo si mette su un elemento esistente come un attributo e ne cambia aspetto o comportamento. Esempio classico: evidenziare un paragrafo quando il mouse ci passa sopra.

```
ng generate directive direttive/highlight
```

Questa è la direttiva scritta nel progetto corso-angular, src/app/direttive/highlight.ts:

```
import { Directive, ElementRef, HostListener, Input } from '@angular/core';

@Directive({
  selector: '[appHighlight]',
})
export class Highlight {
  @Input() defaultColor = '';
  private scelto = '';

  @Input() set appHighlight(colore: string) {
    this.scelto = colore;
    this.cambiaColore(this.coloreAttivo());
  }

  ngOnInit(): void {
    this.cambiaColore(this.coloreAttivo());
  }

  @HostListener('mouseenter') onMouseEnter() {
    this.cambiaColore(this.coloreAttivo());
  }
  @HostListener('mouseleave') onMouseLeave() {
    this.cambiaColore(this.defaultColor);
  }

  private coloreAttivo(): string {
    return this.scelto || this.defaultColor;
  }

  cambiaColore(colore: string) {
    this.el.nativeElement.style.backgroundColor = colore;
  }

  constructor(private el: ElementRef<HTMLElement>) {}
}
```

E il componente che la usa, direttiva-esempio.html:

```
imports: [Highlight],

<input type="radio" name="colore" value="red" (click)="cambiaColoreEvidenziatore('red')"> red
<input type="radio" name="colore" value="green" (click)="cambiaColoreEvidenziatore('green')"> green

<p [appHighlight]="colore()" defaultColor="red">Testo con cui uso la direttiva</p>
<p [appHighlight]="colore()" defaultColor="green">Testo con cui uso la direttiva</p>
```

selector: '[appHighlight]' Le parentesi quadre nel selector vogliono dire "un attributo". La direttiva si attiva su ogni elemento che ha l'attributo appHighlight. Il prefisso app evita conflitti con attributi HTML veri.

ElementRef L'elemento su cui è messa la direttiva. nativeElement è l'elemento vero del DOM.

@Input con lo stesso nome del selector [appHighlight]="colore()" attiva la direttiva e le passa il colore insieme.

@Input() set Un setter: il codice gira ogni volta che arriva un valore nuovo, per esempio quando cambi radio.

defaultColor="red" Un secondo input. Senza parentesi quadre passa la stringa così com'è.

@HostListener('mouseenter') Ascolta un evento sull'elemento che ospita la direttiva.

La stessa direttiva in stile moderno

Con `input()` e la proprietà `host` del decoratore si scrive senza decoratori sulle proprietà e senza setter. Il comportamento è lo stesso.

```
import { Directive, ElementRef, computed, inject, input, signal } from '@angular/core';

@Directive({
  selector: '[appHighlight]',
  host: {
    '(mouseenter)': 'sopra.set(true)',
    '(mouseleave)': 'sopra.set(false)',
    '[style.backgroundColor]': 'colore()',
  },
})
export class Highlight {
  appHighlight = input('');
  defaultColor = input('');

  protected sopra = signal(false);
  protected colore = computed(() =>
    this.sopra() ? this.appHighlight() || this.defaultColor() : this.defaultColor(),
  );
}
```

host: { '(evento)': '...' } Come `@HostListener`: ascolta un evento sull'elemento ospite.

host: { '[proprietà]': '...' } Come `@HostBinding`: collega una proprietà dell'elemento ospite a un valore della direttiva.

Nessun ElementRef Lo stile lo mette Angular tramite il binding: non si tocca il DOM a mano, e funziona anche con SSR.

Toccare il DOM con `nativeElement.style` funziona nel browser, ma non sul server e salta i controlli di sicurezza di Angular. Quando puoi, preferisci `host binding`; se devi davvero modificare il DOM, usa `Renderer2` (`inject(Renderer2)`) invece di `nativeElement`.

Una direttiva che reagisce a un clic fuori

Un caso pratico: chiudere un menu quando l'utente clicca fuori.

```
import { Directive, ElementRef, inject, output } from '@angular/core';

@Directive({
  selector: '[appClickFuori]',
  host: { '(document:click)': 'controlla($event)' },
})
export class ClickFuori {
  private el = inject(ElementRef<HTMLElement>);
  appClickFuori = output<void>();

  controlla(evento: MouseEvent) {
    if (!this.el.nativeElement.contains(evento.target as Node)) {
      this.appClickFuori.emit();
    }
  }
}
```

`<div class="menu" (appClickFuori)="menuAperto.set(false)">...</div>`

`document:click` ascolta i clic su tutta la pagina. La direttiva emette un output quando il clic non è dentro l'elemento. Anche una direttiva può avere input e output, come un componente.

Una direttiva strutturale

Una direttiva strutturale riceve un pezzo di template (`TemplateRef`) e decide quante volte e dove disegnarlo (`ViewContainerRef`). Oggi servono raramente, perché `@if` e `@for` coprono quasi tutto, ma è utile capire come funzionano `*ngIf` e `*ngFor`.

```
import { Directive, TemplateRef, ViewContainerRef, effect, inject, input } from '@angular/core';

@Directive({ selector: '[appRipeti]' })
export class Ripeti {
  private template = inject(TemplateRef);
  private contenitore = inject(ViewContainerRef);
  appRipeti = input(1);

  constructor() {
    effect(() => {
      this.contenitore.clear();
      for (let i = 0; i < this.appRipeti(); i++) {
        this.contenitore.createEmbeddedView(this.template, { $implicit: i + 1 });
      }
    });
  }
}

<p *appRipeti="3; let n">Riga numero {{ n }}</p>
```

L'asterisco trasforma il `<p>` in un ng-template e lo passa alla direttiva. `createEmbeddedView` lo disegna; il secondo argomento è il contesto: `$implicit` è il valore di `let n`.

Errori tipici con le direttive

Can't bind to 'appHighlight' since it isn't a known property of 'p' La direttiva non è negli imports del componente.

La direttiva non fa nulla Il selector non corrisponde: `[appHighlight]` vuole l'attributo con quel nome esatto, maiuscole comprese.

Expected 1 arguments, but got 0 nel test Il test crea la direttiva con `new Highlight()` ma il constructor vuole `ElementRef`. Nel progetto è successo davvero: vedi il capitolo 18.

Lo stile non compare con SSR Hai toccato `nativeElement` nel constructor o in `ngOnInit` sul server. Usa `host binding`.

9. Comunicazione tra componenti

Padre e figlio

Il padre è il componente che nel proprio template contiene il tag del figlio. Il figlio è il componente usato dentro quel tag. I dati scendono con `@Input`. Gli avvisi salgono con `@Output`. Padre e figlio non si leggono le variabili a vicenda: si scambiano solo input e output.

imports: [Figlio] Il padre deve dichiarare il componente figlio, poi può scrivere il suo tag.

@Input() Il figlio riceve un dato. Nel padre: `[eroeSelezionato]="eroeSelezionato"`.

@Output() Il figlio avvisa il padre. Si usa un `EventEmitter`. Nel padre: `(salvato)="onSalva($event)"`.

@Input: un esempio

Una proprietà del figlio che riceve un valore dal padre. Il nome deve essere uguale da entrambe le parti.

[eroeSelezionato]="eroeSelezionato" Nel tag del padre: le quadre sono il collegamento. A destra c'è il valore del padre.

@Input() eroeSelezionato Nel figlio: la proprietà che riceve quel valore.

Il tipo `Eroe | undefined` vuol dire che può mancare, finché non clicchi una card. Quando il padre cambia il valore, Angular lo ricopia nel figlio e il template può usare `eroeSelezionato.nome`, dentro un `@if` che controlla che ci sia.

Cosa deve contenere il padre

1) L'import della classe del figlio. 2) Quella classe nell'array imports del `@Component`: senza, il tag non è un componente. 3) Nel template il tag, che è il selector del figlio, non il nome della classe: se il selector è `app-figlio`, il tag è `<app-figlio>`. 4) Il dato da consegnare e il metodo che riceve la risposta.

```
import { Component, signal } from '@angular/core';
import { Figlio } from './figlio';

@Component({
  selector: 'app-padre',
  imports: [Figlio],
  template: `
    <app-figlio [testo]="valore()" (inviato)="ricevi($event)"></app-figlio>
    <p>{{ ricevuto() }}</p>
  `,
})
export class Padre {
  valore = signal('ciao');
  ricevuto = signal('');
  ricevi(messaggio: string) {
    this.ricevuto.set(messaggio);
  }
}
```

Cosa deve contenere il figlio

1) selector uguale al tag usato dal padre. 2) Una proprietà `@Input` per ogni dato che arriva. 3) Una proprietà `@Output` per ogni avviso che manda su. 4) Nel template legge l'input e, su un'azione, chiama `emit`.

```
import { Component, EventEmitter, Input, Output } from '@angular/core';

@Component({
  selector: 'app-figlio',
  template: `
    <p>{{ testo }}</p>
    <button type="button" (click)="invia()">invia</button>
  `,
})
export class Figlio {
  @Input() testo = '';
  @Output() inviato = new EventEmitter<string>();

  invia() {
    this.inviato.emit(this.testo);
  }
}
```

Dal padre al figlio: @Input

Nel tag, le parentesi quadre consegnano un valore. [testo]="valore()" significa: la proprietà testo del figlio vale il risultato di valore(). Se valore è un signal servono le parentesi, altrimenti passi il contenitore e non il contenuto. Nel figlio la proprietà si dichiara con @Input() testo. Il nome dentro le quadre e il nome dopo @Input devono essere identici. Se il padre fa valore.set('altro'), la vista del figlio si aggiorna perché il binding è continuo.

Nel constructor del figlio l'input non è ancora stato assegnato: this.testo è ancora il valore iniziale. Da ngOnInit in poi è il valore arrivato dal padre.

Dal figlio al padre: @Output

Il figlio non assegna le proprietà del padre. Dichiara un evento con @Output() inviato = new EventEmitter. Quando vuole avvisare, chiama this.inviato.emit(this.testo). L'argomento di emit è il dato che sale. Nel tag del padre le parentesi tonde ascoltano quello stesso nome: (inviato)="ricevi(\$event)". \$event è proprio l'argomento passato a emit. ricevi lo salva, per esempio in un signal, e il template del padre lo mostra con ricevuto(). Il nome dentro le tonde e il nome dopo @Output devono essere identici.

Se il bottone sta dentro un form, metti type="button": altrimenti il browser invia il form e ricarica la pagina.

Regole da ricordare

[nome] scende, è un input del figlio. (nome) sale, è un output del figlio. I due nomi li scegli tu, ma devono combaciare tra tag e classe. Un figlio può avere più input e più output: ognuno ha il suo nome. Il padre può usare più volte lo stesso figlio, passando valori diversi.

@ViewChild: leggere un pezzo del template

ViewChild legge un pezzo del template di questo stesso componente: un elemento HTML oppure un componente figlio. Non passa dati: quello si fa con input e output. È una maniglia sulla vista, dopo che Angular l'ha disegnata.

Leggere un elemento del template: nel template dai un nome all'elemento con il cancelletto. In classe chiedi quello stesso nome, tra apici. Il tipo è ElementRef. I due nomi devono coincidere.

```
import { AfterViewInit, Component, ElementRef, ViewChild } from '@angular/core';

<input #campo value="ciao">

@ViewChild('campo') campo!: ElementRef<HTMLInputElement>;

ngAfterViewInit() {
  console.log(this.campo.nativeElement.value);
}
```

Il punto esclamativo dice a TypeScript che la proprietà arriverà, anche se all'inizio non c'è. `campo.nativeElement` è il vero elemento del DOM; `campo.nativeElement.value` è il testo dell'input.

Quando il valore c'è: nel constructor e in `ngOnInit` la vista non è ancora stata creata, quindi `campo` è `undefined`. Il primo momento sicuro è `ngAfterViewInit`. Se l'elemento sta dentro `@if` e la condizione è falsa, `ViewChild` resta `undefined`. Con `static: true` lo leggi già in `ngOnInit`, ma solo se l'elemento c'è sempre, non dentro un `if`.

```
@ViewChild('campo', { static: true }) campo!: ElementRef<HTMLInputElement>;
```

Leggere un componente figlio: se nel template c'è il tag di un figlio, puoi prendere la sua istanza. Non serve il cancelletto: si passa la classe. Anche questa lettura è pronta in `ngAfterViewInit`.

```
<app-figlio></app-figlio>
@ViewChild(Figlio) figlio!: Figlio;
```

Se i figli con lo stesso tag sono più di uno, `ViewChild` ne prende uno solo, il primo. Per averli tutti si usa `ViewChildren`.

Quale strumento usare

Prima di scrivere codice, chiediti dove stanno i due componenti rispetto all'altro. La risposta sceglie lo strumento.

Padre verso figlio `input()`. Il padre scrive `[nome]="valore"` nel tag del figlio.

Figlio verso padre `output()`. Il figlio chiama `emit`, il padre ascolta con `(nome)="metodo($event)"`.

Padre e figlio insieme `model()`. Legame nei due sensi: `[(nome)]="segnale"`.

Il padre usa un metodo del figlio Variabile di template `#figlio` oppure `ViewChild()`.

Il padre passa HTML al figlio `ng-content`, la proiezione di contenuto.

Fratelli o componenti lontani Un servizio condiviso con un `signal` dentro.

Da una pagina a un'altra Parametri nell'indirizzo, `query params`, oppure un servizio. In casi rari lo state della navigazione.

Regola pratica: se il dato deve restare dopo un ricaricamento o essere condivisibile con un link, va nell'indirizzo. Se deve essere visto da molti componenti, va in un servizio. Se riguarda solo un padre e il suo figlio diretto, `input` e `output`.

`input()` e `output()`: la versione moderna di `@Input` e `@Output`

Sopra hai visto `@Input()` e `@Output()` con `EventEmitter`. Funzionano ancora. Dalla versione 17 esistono le funzioni `input()` e `output()`, che sono l'approccio consigliato: l'input è un `signal`, quindi si usa con `computed` ed `effect` senza `ngOnChanges`.

```
import { Component, booleanAttribute, computed, input, output } from '@angular/core';
import { Eroe } from '../eroe';

@Component({
  selector: 'app-card-eroe',
  template: `
    <div class="card" [class.evidenziata]="evidenziata()" (click)="seleziona()">
      <h3>{{ titolo() }}</h3>
      <small>Potere: {{ eroe().superpotere }}</small>
      <button type="button" (click)="elimina($event)">Elimina</button>
    </div>
  `,
})
export class CardEroe {
  eroe = input.required<Eroe>();
  evidenziata = input(false, { transform: booleanAttribute });
  prefisso = input('Eroe', { alias: 'etichetta' });
```

```

selezionato = output<Eroe>();
eliminato = output<number>();

titolo = computed(() => `${this.prefisso()}: ${this.eroe().nome.toUpperCase()}`);

seleziona() {
  this.selezionato.emit(this.eroe());
}
elimina(evento: Event) {
  evento.stopPropagation();
  this.eliminato.emit(this.eroe().id);
}
}

```

Il padre:

```

imports: [CardEroe],

eroi = signal<Eroe[]>([...]);
scelto = signal<Eroe | undefined>(undefined);
rimuovi(id: number) { this.eroi.update((l) => l.filter((e) => e.id !== id)); }

@for (e of eroi(); track e.id) {
  <app-card-eroe
    [eroe]="e"
    [evidenziata]="scelto()?.id === e.id"
    etichetta="Supereroe"
    (selezionato)="scelto.set($event)"
    (eliminato)="rimuovi($event)" />
}
@if (scelto(); as s) { <p>Hai scelto {{ s.nome }}</p> }

```

input.required<T>() Obbligatorio. Se il padre non lo passa, errore di compilazione.

input(valore) Facoltativo, con valore iniziale.

transform: booleanAttribute Permette di scrivere solo evidenziata nel tag, senza ="true".

transform: numberAttribute Converte la stringa in numero.

alias Nome diverso nel tag rispetto alla classe. Da usare poco.

output<T>() Evento verso il padre. Il tipo T è quello di \$event.

this.eroe() Un input si legge con le parentesi, perché è un signal. Il figlio non può fare set: l'input appartiene al padre.

stopPropagation Senza, il click su Elimina arriverebbe anche al div e selezionerebbe la card.

Reagire a un input che cambia: con i decorator serviva ngOnChanges. Con input() basta computed per un valore derivato, oppure effect(() => ...) per un effetto collaterale, per esempio scrivere nel localStorage.

model(): legame nei due sensi tra padre e figlio

Quando il figlio deve sia ricevere sia modificare un valore del padre, per esempio un contatore o un selettore personalizzato, si usa model(). È come ngModel, ma per i tuoi componenti.

```

import { Component, model } from '@angular/core';

@Component({
  selector: 'app-contatore',
  template: `
    <button type="button" (click)="meno()">-</button>
    <span>{{ valore() }}</span>
    <button type="button" (click)="piu()">+</button>
  `,
})
export class Contatore {
  valore = model(0);
  meno() { this.valore.update((v) => Math.max(0, v - 1)); }
  piu() { this.valore.update((v) => v + 1); }
}

```

```

quantita = signal(1);

<app-contatore [(valore)]= "quantita" />
<p>Nel padre: {{ quantita() }}</p>

```

Attenzione: dentro [()] si passa il signal senza parentesi, quantita e non quantita(). Angular deve poterci scrivere. Qui, al contrario degli input, il figlio può fare set e update: il padre vede subito il cambio. Dietro le quinte model crea un input valore e un output valoreChange.

Il padre usa il figlio: variabile di template e viewChild()

Per chiamare un metodo del figlio, per esempio avvia o reset, gli dai un nome con # nel template del padre.

```

<app-cronometro #crono />
<button type="button" (click)="crono.avvia()">Avvia</button>
<button type="button" (click)="crono.azzera()">Azzera</button>
<p>Secondi: {{ crono.secondi() }}</p>

```

Se serve nel codice della classe, la versione signal di ViewChild è viewChild():

```

import { ViewChild } from '@angular/core';

crono = ViewChild.required(Cronometro);
campo = ViewChild<ElementRef<HTMLInputElement>>('campo');

ngAfterViewInit() {
  this.crono().avvia();
  this.campo()?.nativeElement.focus();
}

```

Da preferire input e output quando possibile: il padre che comanda il figlio lega molto i due componenti.

ng-content: passare HTML al figlio

Il padre può mettere contenuto tra i tag del figlio. Il figlio decide dove mostrarlo. È il modo per costruire contenitori riutilizzabili: card, pannelli, finestre modali.

```

@Component({
  selector: 'app-pannello',
  template: `
    <section class="pannello">
      <header><ng-content select="[titolo]" /></header>
      <div class="corpo"><ng-content /></div>
      <footer><ng-content select="[azioni]" /></footer>
    </section>
  `,
})
export class Pannello {}

<app-pannello>
  <h3 titolo>Eroi preferiti</h3>
  <p>Qualsiasi contenuto va nel corpo.</p>
  <button azioni type="button">Chiudi</button>
</app-pannello>

```

select="[titolo]" prende gli elementi con l'attributo titolo. ng-content senza select prende tutto il resto. Il contenuto appartiene al padre: i binding {{ }} dentro leggono le proprietà del padre, non del figlio.

Fratelli e componenti lontani: servizio con signal

Due componenti che non sono padre e figlio non possono scambiarsi input e output. La soluzione è un servizio condiviso. Il servizio tiene lo stato in un signal, i componenti lo leggono e chiamano i suoi metodi per cambiarlo. Esempio: una lista prodotti aggiunge al carrello, un'icona nell'intestazione mostra il numero.

carrello-servizio.ts

```

import { Injectable, computed, signal } from '@angular/core';

```

```

export interface Prodotto { id: number; nome: string; prezzo: number; }
export interface Articolo extends Prodotto { quantita: number; }

@Injectable({ providedIn: 'root' })
export class CarrelloServizio {
  private readonly _articoli = signal<Articolo[]>([]);

  readonly articoli = this._articoli.asReadonly();
  readonly numero = computed(() => this._articoli().reduce((n, a) => n + a.quantita, 0));
  readonly totale = computed(() =>
    this._articoli().reduce((t, a) => t + a.prezzo * a.quantita, 0));

  aggiungi(p: Prodotto) {
    this._articoli.update((lista) => {
      const presente = lista.find((a) => a.id === p.id);
      return presente
        ? lista.map((a) => (a.id === p.id ? { ...a, quantita: a.quantita + 1 } : a))
        : [...lista, { ...p, quantita: 1 }];
    });
  }
  rimuovi(id: number) {
    this._articoli.update((lista) => lista.filter((a) => a.id !== id));
  }
  svuota() {
    this._articoli.set([]);
  }
}

```

lista-prodotti.ts

```

export class ListaProdotti {
  protected carrello = inject(CarrelloServizio);
  prodotti: Prodotto[] = [
    { id: 1, nome: 'Mantello', prezzo: 49.9 },
    { id: 2, nome: 'Scudo', prezzo: 120 },
  ];

  @for (p of prodotti; track p.id) {
    <p>{{ p.nome }} - {{ p.prezzo | currency:'EUR' }}
    <button type="button" (click)="carrello.aggiungi(p)">Aggiungi</button></p>
  }
}

```

icona-carrello.ts

```

export class IconaCarrello {
  protected carrello = inject(CarrelloServizio);
}

<span>Carrello: {{ carrello.numero() }} ({{ carrello.totale() | currency:'EUR' }})</span>

```

_articoli privato Solo il servizio può modificarlo. Nessun componente può fare set per sbaglio.

asReadonly() Versione di sola lettura dello stesso signal, da esporre fuori.

computed Valori derivati: si ricalcolano da soli quando cambia _articoli.

Nuovo array a ogni modifica Un signal si accorge del cambio solo se il valore è un oggetto diverso.

lista.push(x) modifica lo stesso array e la pagina non si aggiorna. Per questo si usano [...lista, x], map e filter.

providedIn: 'root' Una sola istanza per tutta l'app: tutti i componenti vedono lo stesso carrello.

La stessa cosa con RxJS: BehaviorSubject

Nel codice esistente trovi spesso questa forma. Un BehaviorSubject è un Observable che ricorda l'ultimo valore e lo dà subito a chi si iscrive.

```
import { BehaviorSubject } from 'rxjs';

private readonly _articoli = new BehaviorSubject<Articolo[]>([]);
readonly articoli$ = this._articoli.asObservable();

aggiungi(p: Prodotto) {
  this._articoli.next([...this._articoli.value, { ...p, quantita: 1 }]);
}

<!-- nel template -->
@for (a of carrello.articoli$ | async; track a.id) { ... }
```

Nei progetti nuovi preferisci i signal per lo stato. Tieni gli Observable per gli eventi nel tempo: HTTP, WebSocket, input dell'utente con debounceTime.

Da una pagina all'altra

Parametro di percorso /eroi/3. Identifica la risorsa. Il dettaglio poi carica i dati dal servizio.

Query params /eroi?cerca=man. Stato facoltativo, condivisibile con un link.

Servizio La pagina A salva il dato nel servizio, la pagina B lo legge. Si perde se ricarichi il browser, a meno che il servizio non lo salvi in localStorage.

state della navigazione Un oggetto passato con navigate, non visibile nell'indirizzo.

```
// pagina di partenza
this.router.navigate(['/conferma'], { state: { numeroOrdine: 1042 } });

// pagina di arrivo
numeroOrdine = history.state?.numeroOrdine as number | undefined;
```

Lo state si perde se l'utente ricarica la pagina o apre il link in una nuova scheda. Usalo solo per informazioni di passaggio, come un messaggio di conferma. history esiste solo nel browser: con SSR proteggi la lettura con isPlatformBrowser.

PARTE SECONDA

Costruire un'applicazione

10. Servizi e dependency injection

Cos'è un servizio

Un servizio è una classe condivisa. Ci metti dati o chiamate usati da più componenti. Si crea con `ng g service nome` e si ottiene con `inject(NomeServizio)`. La lista eroi può stare lì, e lista e dettaglio la usano entrambi. Un servizio con stato condiviso tra componenti, il carrello, è nel capitolo 9; i servizi che fanno chiamate HTTP sono nel capitolo 15.

Cos'è la dependency injection

Un componente non crea da solo i servizi con `new`. Li chiede ad Angular con `inject()`. Angular li crea una volta, li conserva e li consegna a chi li chiede. Si chiama *dependency injection*. I vantaggi: tutti condividono la stessa istanza, e nei test si può sostituire il servizio vero con uno finto senza toccare il componente.

```
ng g service servizi/eroi-api

import { Injectable, inject } from '@angular/core';
import { HttpClient } from '@angular/common/http';

@Injectable({ providedIn: 'root' })
export class EroiApi {
  private http = inject(HttpClient);
}
```

@Injectable Rende la classe un servizio che Angular sa creare e che può a sua volta ricevere altri servizi.

providedIn: 'root' Una sola istanza per tutta l'app, creata la prima volta che qualcuno la chiede. Se nessuno la usa, non finisce nel bundle.

inject(Classe) Chiede il servizio. Va scritto nella dichiarazione di un campo o nel constructor.

constructor(private s: Servizio) Il modo classico. Equivalente a `inject`. In *tour-degli-eroi* trovi entrambi.

`inject()` funziona solo in un contesto di iniezione: inizializzazione dei campi, constructor, guardie, resolver, interceptor, funzioni factory. Dentro `ngOnInit` o in un metodo chiamato da un click dà l'errore NG0203. Prendi il servizio in un campo e usalo dopo.

Dove vive un servizio: root o componente

```
@Component({
  selector: 'app-editor-eroe',
  providers: [BozzaServizio],
  templateUrl: './editor-eroe.html',
})
export class EditorEroe {
  private bozza = inject(BozzaServizio);
}
```

Mettendo il servizio in `providers` del componente, ogni istanza di `EditorEroe` riceve un `BozzaServizio` nuovo, condiviso solo con i suoi figli. Quando il componente viene distrutto, anche il servizio sparisce. Utile se in pagina ci sono due editor aperti che non devono mescolare i dati. In quel caso nel servizio metti `@Injectable()` senza `providedIn`.

providedIn: 'root' Singleton globale. Stato dell'app, chiamate HTTP, autenticazione.

providers nel componente Un'istanza per componente e figli. Stato locale di un widget.

providers in una rotta { path: 'admin', providers: [AdminServizio], ... } Un'istanza per quella sezione.

Valori di configurazione: InjectionToken

Non si inietta solo una classe. Con un InjectionToken si inietta anche una stringa o un oggetto, per esempio l'indirizzo delle API.

```
import { InjectionToken } from '@angular/core';
export const API_URL = new InjectionToken<string>('API_URL');

// app.config.ts
providers: [{ provide: API_URL, useValue: 'http://localhost:3000' }]

// nel servizio
private apiUrl = inject(API_URL);
```

useValue Consegna un valore fisso.

useClass { provide: EroApi, useClass: EroApiFinta } Chi chiede EroApi riceve la versione finta. Ottimo per sviluppare senza server e per i test.

useFactory Calcola il valore con una funzione, che può usare inject.

File di ambiente

```
ng g environments
```

Crea src/environments/environment.ts (produzione) ed environment.development.ts (sviluppo), e in angular.json aggiunge la sostituzione automatica: ng serve usa il file development, ng build quello di produzione.

```
// environment.development.ts
export const environment = { production: false, apiUrl: 'http://localhost:3000' };

// environment.ts
export const environment = { production: true, apiUrl: 'https://api.miosito.it' };

// nel servizio
import { environment } from '../environments/environment';
private url = `${environment.apiUrl}/eroi`;
```

Importa sempre environment.ts, mai il file development: la sostituzione la fa la build. Non mettere segreti in questi file: finiscono nel JavaScript scaricato dal browser e chiunque può leggerli.

Errori tipici con i servizi

NullInjectorError: No provider for X Il servizio non ha providedIn: 'root' e non è in nessun providers.

NG0203 inject() must be called from an injection context inject chiamato dentro un metodo o in ngOnInit. Spostalo in un campo.

Due componenti vedono dati diversi Il servizio è in providers dei componenti: ognuno ne ha una copia.

11. Observable e RxJS

Cos'è un Observable

Un Observable è un flusso di valori che arrivano nel tempo. Non è il dato già pronto: è il biglietto per riceverlo dopo. Sta nella libreria rxjs, che Angular usa per HTTP, router, form ed eventi. Il nome della variabile spesso finisce con \$: eroi\$.

`getEroi(): Eroei[]` consegna l'array subito. `getEroi(): Observable<Eroe[]>` consegna il flusso: i dati arrivano quando qualcuno ascolta.

subscribe Inizia ad ascoltare. Senza subscribe (o senza async, o toSignal) il flusso non parte.

next Funzione chiamata ogni volta che arriva un valore.

error Funzione chiamata se il flusso fallisce, per esempio una HTTP andata male.

complete Il flusso è finito e non manderà altri valori.

unsubscribe Smetti di ascoltare. Va fatto quando il componente viene distrutto, altrimenti il lavoro continua.

takeUntilDestroyed Alternativa a unsubscribe: si stacca da solo alla distruzione del componente.

of(valore) Crea un Observable che emette subito quel valore e poi finisce. Serve per i dati finti.

HttpClient.get La chiamata HTTP restituisce un Observable. La risposta arriva dopo, non nella riga successiva.

async Pipe del template. Scrivi `eroi$ | async`: Angular fa subscribe, mostra il valore e si stacca da solo. Serve AsyncPipe negli imports.

map Trasforma ogni valore che passa nel flusso, senza cambiare gli altri.

```
this.servizio.getEroi().subscribe({
  next: (eroi) => this.eroi.set(eroi),
  error: (err) => this.errore.set('Caricamento fallito'),
  complete: () => console.log('finito'),
});
```

Nel componente ci sono due modi. Nel codice: `this.servizio.getEroi().subscribe(eroi => this.eroi.set(eroi))`. Nel template: `eroi$ = this.servizio.getEroi()` e poi `@for (e of eroi$ | async; track e.id)`. Il secondo evita di gestire unsubscribe a mano. Un terzo modo, il più comodo con i signal, è toSignal, più sotto.

Un Observable non è un signal. Il signal tiene un valore attuale e la pagina lo legge con le parentesi. L'Observable avvisa quando arriva qualcosa. Si possono usare insieme: nel subscribe chiami `signal.set` con il valore arrivato.

Subscribe e unsubscribe

Ogni subscribe che non finisce da solo va chiuso. Se non lo fai, Angular continua ad ascoltare anche dopo che il componente è stato distrutto: la risposta arriva tardi e scrive su un componente che non c'è più. Restano ascolti appesi, memoria occupata e aggiornamenti su pagine già uscite.

subscribe Parte l'ascolto. Senza subscribe la chiamata HTTP non parte.

richiesta = ...subscribe(...) Tieni da parte l'oggetto Subscription.

ngOnDestroy Angular lo chiama quando toglie il componente dalla pagina.

richiesta.unsubscribe() Chiude l'ascolto. Va messo in ngOnDestroy.

```
private richiesta?: Subscription;

ngOnInit() {
  this.richiesta = this.eroeservizio.getEroi().subscribe((eroi) => this.eroi.set(eroi));
}

ngOnDestroy() {
  this.richiesta?.unsubscribe();
}
```

Oggi si preferisce `takeUntilDestroyed`, che fa lo stesso senza `ngOnDestroy`:

```
private destroyRef = inject(DestroyRef);

ngOnInit() {
  this.servizio.getEroi()
    .pipe(takeUntilDestroyed(this.destroyRef))
    .subscribe((eroi) => this.eroi.set(eroi));
}
```

Nel constructor o nell'inizializzazione di un campo basta `takeUntilDestroyed()` senza argomenti. Una chiamata HTTP finisce da sola dopo la risposta, quindi non resta appesa per sempre; ma se l'utente cambia pagina prima della risposta il callback gira lo stesso. Flussi che non finiscono mai, come `interval`, `valueChanges` di un form, eventi del router, vanno chiusi sempre.

Creare Observable

```
import { from, fromEvent, interval, of, throwError, timer } from 'rxjs';

of(1, 2, 3); // emette 1, 2, 3 e finisce
from([10, 20]); // da un array (o da una Promise)
interval(1000); // 0, 1, 2... ogni secondo, per sempre
timer(500); // un solo valore dopo mezzo secondo
fromEvent(document, 'click'); // ogni clic sulla pagina
throwError(() => new Error('ko')); // un flusso che fallisce subito
```

Subject e BehaviorSubject

Un Subject è un Observable in cui puoi spingere valori tu, con `next`. Serve per far comunicare parti dell'app attraverso un servizio.

Subject Chi si iscrive riceve solo i valori emessi da quel momento in poi.

BehaviorSubject(iniziale) Ricorda l'ultimo valore e lo dà subito a chi si iscrive. `.value` lo legge in modo sincrono. Era il modo classico per lo stato nei servizi, oggi sostituito dai signal.

ReplaySubject(n) Ricorda gli ultimi n valori e li ripete a chi arriva dopo.

```
private readonly _messaggi = new Subject<string>();
readonly messaggi$ = this._messaggi.asObservable(); // fuori si può solo ascoltare

invia(testo: string) {
  this._messaggi.next(testo);
}
```

pipe e gli operatori

Gli operatori si mettono dentro `.pipe(...)` e trasformano il flusso, uno dopo l'altro, come una catena di montaggio. Ognuno restituisce un Observable nuovo.

map(x => ...) Trasforma ogni valore.

filter(x => ...) Lascia passare solo i valori che soddisfano la condizione.

tap(x => ...) Esegue un'azione (un log, salvare un token) senza cambiare il valore.

take(n) Prende solo i primi n valori, poi finisce.

startWith(v) Emette v per primo.

debounceTime(300) Aspetta 300 ms di pausa prima di far passare l'ultimo valore. Per la ricerca mentre si scrive.

distinctUntilChanged() Scarta un valore uguale al precedente.

catchError(err => ...) Intercetta l'errore: restituisci un valore di riserva con `of(...)` o rilancialo con `throwError`.

retry(2) Se fallisce, riprova fino a 2 volte.

finalize(() => ...) Alla fine, sia con successo sia con errore. Per spegnere uno spinner.

shareReplay(1) Condivide un'unica esecuzione tra più iscritti e ripete l'ultimo valore. Evita richieste HTTP doppie.

Operatori che passano a un altro Observable: si usano quando per ogni valore bisogna fare una nuova chiamata.

switchMap Annulla la chiamata precedente se ne arriva una nuova. Per ricerca e dettaglio da parametro dell'URL: conta solo l'ultima.

mergeMap Fa tutte le chiamate in parallelo, senza annullare.

concatMap Le fa una dopo l'altra, in ordine. Per salvataggi che devono rispettare la sequenza.

exhaustMap Ignora i nuovi valori finché la chiamata in corso non finisce. Per un bottone Salva cliccato più volte.

Combinare più flussi:

forkJoin([a\$, b\$]) Aspetta che tutti finiscano e dà i risultati insieme. Per caricare più cose all'avvio di una pagina.

combineLatest([a\$, b\$]) A ogni valore di uno qualsiasi, emette l'ultimo valore di tutti. Per filtri che dipendono da più campi.

Esempio: ricerca mentre si scrive

```
import { FormControl, ReactiveFormsModule } from '@angular/forms';
import { toSignal } from '@angular/core/rxjs-interop';
import { catchError, debounceTime, distinctUntilChanged, filter, of, switchMap } from 'rxjs';

export class Ricerca {
  private api = inject(EroiApi);
  cerca = new FormControl('', { nullable: true });

  risultati = toSignal(
    this.cerca.valueChanges.pipe(
      debounceTime(300), // aspetta che l'utente smetta di scrivere
      distinctUntilChanged(), // niente richiesta se il testo non è cambiato
      filter((t) => t.length === 0 || t.length >= 2),
      switchMap((t) => this.api.lista(t).pipe(catchError(() => of([])))), // solo l'ultima conta
    ),
    { initialValue: [] },
  );
}

<input [formControl]="cerca" placeholder="Cerca un eroe">
@for (e of risultati(); track e.id) { <p>{{ e.nome }}</p> }
```

Il `catchError` sta dentro `switchMap`: così un errore di una ricerca non chiude il flusso principale, e la ricerca successiva funziona ancora. Se fosse fuori, dopo il primo errore la casella smetterebbe di cercare.

Esempio: caricare due cose insieme

```
forkJoin({
  eroi: this.api.lista(),
  livelli: this.api.livelli(),
}).subscribe(({ eroi, livelli }) => {
  this.eroi.set(eroi);
  this.livelli.set(livelli);
});
```

Da Observable a signal e ritorno

```
import { toObservable, toSignal } from '@angular/core/rxjs-interop';

eroi = toSignal(this.api.lista(), { initialValue: [] }); // signal, si stacca da solo
id = input.required<number>();
id$ = toObservable(this.id); // Observable dai cambi di un signal
```

Regola pratica: signal per lo stato (cosa c'è adesso), Observable per gli eventi nel tempo (richieste, digitazione, WebSocket). Al confine si passa dall'uno all'altro con toSignal e toObservable. Entrambi vanno chiamati in un contesto di iniezione: campo della classe o constructor.

Errori tipici con RxJS

Non succede niente Nessun subscribe, async o toSignal: l'Observable non parte.

La richiesta parte due volte Due subscribe allo stesso Observable HTTP, o async usato due volte nel template. Usa toSignal o shareReplay(1).

Dopo un errore la ricerca smette di funzionare catchError fuori da switchMap: l'errore ha chiuso il flusso.

NG0203 toSignal() can only be used within an injection context toSignal chiamato in un metodo o in ngOnInit. Spostalo in un campo.

Il valore è undefined subito dopo subscribe La riga dopo subscribe gira prima della risposta. Usa il valore dentro next.

12. Routing

Le idee di base

Il routing cambia il componente mostrato in base all'indirizzo, senza ricaricare la pagina. In corso-angular è già acceso: `provideRouter(routes)` sta in `app.config.ts` e legge l'elenco da `app.routes.ts`.

app.routes.ts Dice quale indirizzo mostra quale componente.

router-outlet Punto del template dove appare la pagina della rotta attuale.

routerLink Link che cambia pagina senza ricaricare il browser.

/eroi/3 Una rotta può avere un parametro, per aprire un eroe preciso.

Passo 1: accendere il router

```
import { provideRouter, withComponentInputBinding } from '@angular/router';

providers: [
  provideRouter(routes, withComponentInputBinding()),
]
```

Va nell'array `providers` di `app.config.ts`. `routes` è l'elenco esportato da `app.routes.ts`. Senza questa riga i path non fanno nulla. `withComponentInputBinding()` è facoltativo ma consigliato: fa arrivare i parametri dell'indirizzo direttamente negli input del componente (passo 4). In corso-angular oggi non c'è: per questo Dettaglio legge il parametro con `route.snapshot`.

Passo 2: scrivere le rotte

```
import { Routes } from '@angular/router';
import { Lista } from '../pagine/lista/lista';
import { Dettaglio } from '../pagine/dettaglio/dettaglio';

export const routes: Routes = [
  { path: '', pathMatch: 'full', redirectTo: 'lista' },
  { path: 'lista', component: Lista },
  { path: 'dettaglio/:id', component: Dettaglio },
];
```

`path` è la parte di indirizzo dopo il dominio. `component` è la classe della pagina. Il path vuoto è la home: `pathMatch full` evita che divori anche gli altri indirizzi. `redirectTo` manda alla lista. I due punti in `:id` sono un parametro: `dettaglio/3` mette 3 in `id`. Le classi `Lista` e `Dettaglio` vanno importate nel file delle rotte.

Passo 3: dove compare la pagina

```
imports: [RouterOutlet, RouterLink],

<a routerLink="/lista">Lista</a>
<a [routerLink]="['/dettaglio', id]">Apri</a>
<router-outlet />
```

`router-outlet` è il buco in cui Angular mette il componente della rotta corrente. I link usano `routerLink`, non `href`: `href` ricaricherebbe la pagina. `RouterOutlet` e `RouterLink` si importano da `@angular/router` e vanno nell'array `imports` del componente che li scrive nel template, di solito `App`.

Passo 4: leggere il parametro nella pagina

```
// con withComponentInputBinding() in provideRouter
id = input<string>();

// senza: con ActivatedRoute
private route = inject(ActivatedRoute);
ngOnInit() {
  const id = this.route.snapshot.paramMap.get('id');
}
```

Nel componente Dettaglio, `input()` con lo stesso nome del parametro (`:id`) riceve il pezzo di indirizzo, ma solo se in `provideRouter` c'è `withComponentInputBinding()`: senza, l'`input` resta undefined. Nel template si legge `id()`. Con l'indirizzo `/dettaglio/3`, `id()` vale '3', ed è una stringa: per un numero usa `input.required({ transform: numberAttribute })`. I tre modi di leggere un parametro, e quando usarli, sono spiegati più avanti in questo capitolo.

Passo 5: cambiare pagina dal codice

```
private router = inject(Router);

apri(id: number) {
  this.router.navigate(['/dettaglio', id]);
}
```

`inject(Router)` prende il router. `navigate` riceve gli stessi pezzi di `routerLink`. Si usa quando il cambio pagina avviene dopo un'azione, per esempio un click che prima salva un dato.

Il comando `ng generate module app-routing --flat`

```
ng generate module app-routing --flat
```

È il modo vecchio, quello con i `NgModule`. Crea il file `app-routing.module.ts` con una classe `AppRoutingModule`. Dentro si dichiarano le rotte e si importa `RouterModule.forRoot(routes)`. `--flat` dice di non creare una cartella: il file finisce in `src/app`, accanto agli altri, invece che in `src/app/app-routing/`.

Nei progetti standalone non serve: le rotte sono l'array in `app.routes.ts` e si accendono con `provideRouter(routes)` in `app.config.ts`. In `corso-angular` c'è un file `app-routing-module.ts` rimasto da una prova: non è importato da nessuna parte, quindi non ha effetto, e si può eliminare.

Un file di rotte completo

Questo è un `app.routes.ts` realistico. Contiene tutti i casi che si incontrano davvero: home con redirect, lista, dettaglio con parametro, pagina caricata solo quando serve, area protetta con figlie, login, pagina 404.

```
import { Routes } from '@angular/router';
import { Home } from './pagine/home/home';
import { Eroi } from './pagine/eroi/eroi';
import { DettaglioEroe } from './pagine/dettaglio-eroe/dettaglio-eroe';
import { Login } from './pagine/login/login';
import { NonTrovata } from './pagine/non-trovata/non-trovata';
import { authGuard } from './guardie/auth-guard';
import { eroeResolver } from './resolver/eroe-resolver';

export const routes: Routes = [
  { path: '', pathMatch: 'full', redirectTo: 'home' },
  { path: 'home', component: Home, title: 'Home' },
  { path: 'eroi', component: Eroi, title: 'Elenco eroi' },
  {
    path: 'eroi/:id',
    component: DettaglioEroe,
    resolve: { eroe: eroeResolver },
    title: 'Dettaglio eroe',
  },
],
{
```

```

    path: 'statistiche',
    loadComponent: () =>
      import('./pagine/statistiche/statistiche').then((m) => m.Statistiche),
  },
  {
    path: 'admin',
    canActivate: [AuthGuard],
    loadChildren: () => import('./admin/admin.routes').then((m) => m.ADMIN_ROUTES),
  },
  { path: 'login', component: Login, title: 'Accedi' },
  { path: '**', component: NonTrovata, title: 'Pagina non trovata' },
];

```

title Testo della scheda del browser quando la rotta è attiva. Non serve scrivere codice per cambiarlo.

resolve Carica dati prima di mostrare la pagina. Il risultato arriva nell'input con lo stesso nome della chiave, qui eroe.

loadComponent Il componente viene scaricato solo la prima volta che si apre quell'indirizzo. Il bundle iniziale resta più piccolo.

loadChildren Come loadComponent, ma carica un intero file di rotte figlie.

canActivate Elenco di guardie: se una dice no, la pagina non si apre.

****** Qualsiasi indirizzo. Va sempre per ultimo.

L'ordine conta. Il router prova le rotte dall'alto verso il basso e si ferma alla prima che combacia. Se metti ****** in cima, ogni indirizzo finisce nella pagina 404. Se metti eroi/:id prima di eroi/nuovo, la parola nuovo viene presa come id.

Link con evidenziazione della pagina attiva

```

imports: [RouterLink, RouterLinkActive, RouterOutlet],

<nav>
  <a routerLink="/home" routerLinkActive="attivo">Home</a>
  <a routerLink="/eroi" routerLinkActive="attivo"
    [routerLinkActiveOptions]="{ exact: true }">Eroi</a>
</nav>
<router-outlet />

```

routerLinkActive aggiunge la classe CSS attivo al link quando l'indirizzo corrente corrisponde. Senza exact: true il link /eroi resta attivo anche su /eroi/3. La classe la definisci tu nello scss, per esempio .attivo { font-weight: bold; }.

Parametri del percorso: tre modi per leggerli

Indirizzo /eroi/3. Il pezzo 3 arriva al componente DettaglioEroe. Si può leggere in tre modi. Tutti funzionano; cambia quando usarli.

Modo 1, input() (consigliato, serve withComponentInputBinding):

```

import { Component, input, numberAttribute } from '@angular/core';

export class DettaglioEroe {
  id = input.required({ transform: numberAttribute });
}

```

transform: numberAttribute converte la stringa '3' nel numero 3. Nel template scrivi {{ id() }}. Se l'utente passa da /eroi/3 a /eroi/4 restando nella stessa pagina, Angular riusa il componente e aggiorna solo il signal id.

Modo 2, snapshot (una lettura sola):

```
private route = inject(ActivatedRoute);

ngOnInit() {
  const id = Number(this.route.snapshot.paramMap.get('id'));
  this.eroe = this.servizio.getEroe(id);
}
```

È quello usato oggi in corso-angular. Legge il valore una volta. Se l'indirizzo cambia senza cambiare componente, per esempio con un bottone Eroe successivo, la pagina non si aggiorna: ngOnInit non viene richiamato.

Modo 3, paramMap come Observable (si aggiorna sempre):

```
import { toSignal } from '@angular/core/rxjs-interop';
import { map, switchMap } from 'rxjs';

private route = inject(ActivatedRoute);
private servizio = inject(Eroeservizio);

eroe = toSignal(
  this.route.paramMap.pipe(
    map((p) => Number(p.get('id'))),
    switchMap((id) => this.servizio.getEroe(id)),
  ),
);
```

paramMap emette ogni volta che il parametro cambia. switchMap annulla la richiesta HTTP precedente se ne parte una nuova. toSignal trasforma il risultato in un signal: nel template eroe()?.nome. toSignal si stacca da solo quando il componente viene distrutto, niente unsubscribe.

Con il modo 1, per ricaricare i dati quando l'id cambia:

```
import { toObservable, toSignal } from '@angular/core/rxjs-interop';

id = input.required({ transform: numberAttribute });
eroe = toSignal(toObservable(this.id).pipe(switchMap((id) => this.servizio.getEroe(id))));
```

Esempio completo: dettaglio con precedente e successivo

dettaglio-eroe.ts

```
import { Component, computed, inject, input, numberAttribute } from '@angular/core';
import { toObservable, toSignal } from '@angular/core/rxjs-interop';
import { RouterLink } from '@angular/router';
import { Location } from '@angular/common';
import { switchMap } from 'rxjs';
import { Eroeservizio } from '../servizi/eroeservizio';

@Component({
  selector: 'app-dettaglio-eroe',
  imports: [RouterLink],
  templateUrl: './dettaglio-eroe.html',
})
export class DettaglioEroe {
  private servizio = inject(Eroeservizio);
  private location = inject(Location);

  id = input.required({ transform: numberAttribute });
  eroe = toSignal(toObservable(this.id).pipe(switchMap((id) => this.servizio.getEroe(id))));
  precedente = computed(() => this.id() - 1);
  successivo = computed(() => this.id() + 1);

  indietro() {
    this.location.back();
  }
}
```

dettaglio-eroe.html

```

@if (eroe(); as e) {
  <h2>{{ e.nome }}</h2>
  <p>Id: {{ e.id }}</p>
} @else {
  <p>Caricamento...</p>
}
@if (precedente() > 0) {
  <a [routerLink]='[/eroi, precedente()]>Precedente</a>
}
<a [routerLink]='[/eroi, successivo()]>Successivo</a>
<button type="button" (click)="indietro()">Indietro</button>

```

Location.back() fa come il tasto indietro del browser. [routerLink]='[/eroi, 5]' costruisce /eroi/5: ogni elemento dell'array è un pezzo dell'indirizzo.

Query params: filtri, ricerca, pagine

I query params sono la parte dopo il punto interrogativo: /eroi?cerca=man&pagina=2. Non fanno parte del path: la rotta resta eroi. Si usano per stati facoltativi che devono sopravvivere al ricaricamento e si possono condividere con un link: filtri, ordinamento, numero di pagina.

Scriverli con un link:

```
<a routerLink="/eroi" [queryParams]='{ cerca: 'man', pagina: 2 }'>Cerca man</a>
```

Scriverli dal codice:

```
this.router.navigate(['/eroi'], { queryParams: { pagina: 3 }, queryParamsHandling: 'merge' });
```

queryParamsHandling: 'merge' Tiene i query params già presenti e aggiunge o cambia solo quelli indicati. Senza, tutti gli altri spariscono.

queryParamsHandling: 'preserve' Tiene quelli attuali e ignora i nuovi. Utile nei link di navigazione.

valore null Toglie quel parametro dall'indirizzo: { cerca: null }.

Leggerli: con withComponentInputBinding basta un input con lo stesso nome. Altrimenti route.snapshot.queryParamMap.get('cerca') oppure l'Observable route.queryParamMap.

Esempio completo: ricerca che aggiorna l'indirizzo.

eroi.ts

```

import { Component, computed, inject, input } from '@angular/core';
import { toSignal } from '@angular/core/rxjs-interop';
import { ActivatedRoute, Router, RouterLink } from '@angular/router';
import { Eroeservizio } from '../servizi/eroeservizio';

@Component({
  selector: 'app-eroi',
  imports: [RouterLink],
  templateUrl: './eroi.html',
})
export class Eroi {
  private router = inject(Router);
  private route = inject(ActivatedRoute);
  private servizio = inject(Eroeservizio);

  cerca = input<string>(); // da ?cerca=
  eroi = toSignal(this.servizio.getEroi(), { initialValue: [] });

  eroiFiltrati = computed(() => {
    const testo = (this.cerca() ?? '').toLowerCase();
    return this.eroi().filter((e) => e.nome.toLowerCase().includes(testo));
  });

  aggiornaRicerca(testo: string) {
    this.router.navigate([], {
      relativeTo: this.route,
      queryParams: { cerca: testo || null },
      queryParamsHandling: 'merge',
      replaceUrl: true,
    });
  }
}

```

```

    });
  }
}

```

eroi.html

```




  @for (e of eroiFiltrati(); track e.id) {
    <li><a [routerLink]="['/eroi', e.id]">{{ e.nome }}</a></li>
  } @empty {
    <li>Nessun eroe trovato</li>
  }
</ul>

```

`navigate([])` con `relativeTo` resta sulla stessa pagina e cambia solo i query params. `replaceUrl: true` sostituisce la voce della cronologia invece di aggiungerne una a ogni lettera digitata. Il flusso è circolare ma semplice: digiti, cambia l'indirizzo, cambia l'input cerca, computed ricalcola la lista.

Dati statici della rotta

```
{ path: 'admin', component: Admin, data: { ruoli: ['admin'], sezione: 'Gestione' } }
```

`data` è un oggetto fisso scritto nella rotta. Si legge con `route.snapshot.data['sezione']`, con un `input` `sezione = input<string>()` se l'input binding è attivo, oppure dentro una guardia con `route.data`. Serve per configurare una pagina o una guardia senza scrivere codice nuovo.

Rotte figlie e router-outlet annidato

Le rotte figlie servono quando più pagine condividono una cornice: un menu laterale, un'intestazione di sezione. Il componente padre ha il suo router-outlet dove appaiono le figlie.

admin.routes.ts

```

import { Routes } from '@angular/router';
import { AdminLayout } from './admin-layout';
import { AdminUtenti } from './admin-utenti';
import { AdminImpostazioni } from './admin-impostazioni';

export const ADMIN_ROUTES: Routes = [
  {
    path: '',
    component: AdminLayout,
    children: [
      { path: '', pathMatch: 'full', redirectTo: 'utenti' },
      { path: 'utenti', component: AdminUtenti },
      { path: 'utenti/:id', component: AdminUtenti },
      { path: 'impostazioni', component: AdminImpostazioni },
    ],
  },
];

```

admin-layout.html

```

<aside>
  <a routerLink="utenti" routerLinkActive="attivo">Utenti</a>
  <a routerLink="impostazioni" routerLinkActive="attivo">Impostazioni</a>
</aside>
<main>
  <router-outlet />
</main>

```

Il path del file `admin.routes.ts` è vuoto perché il prefisso `admin` è già nella rotta che fa `loadChildren`. Indirizzi risultanti: `/admin/utenti`, `/admin/impostazioni`. I link senza barra iniziale sono relativi: `routerLink="utenti"` dentro `AdminLayout` porta a `/admin/utenti`. Con la barra, `/utenti`, sarebbe assoluto e cercherebbe una rotta `utenti` in cima.

Navigazione relativa dal codice

```
private router = inject(Router);
private route = inject(ActivatedRoute);

apriFratello(id: number) {
  this.router.navigate(['..', id], { relativeTo: this.route }); // da /eroi/3 a /eroi/7
}
vaiAModifica() {
  this.router.navigate(['modifica'], { relativeTo: this.route }); // da /eroi/3 a /eroi/3/modifica
}
```

navigate(['a', 1]) Array di pezzi, come routerLink. Accetta relativeTo, queryParams, fragment, state, replaceUrl.

navigateByUrl('/a/1?x=2') Una stringa già completa. Sempre assoluta.

'..' Sale di un livello, come nelle cartelle.

fragment: 'commenti' Aggiunge #commenti all'indirizzo, per saltare a una sezione.

navigate restituisce una Promise<boolean>: true se la navigazione è andata a buon fine, false se una guardia l'ha bloccata.

Guardie: chi può entrare e uscire

Una guardia è una funzione che il router chiama prima di cambiare pagina. Restituisce true per lasciar passare, false per bloccare, oppure un UrlTree per mandare altrove. Si crea con ng g guard nome e si sceglie il tipo.

canActivate Posso entrare in questa rotta? Il caso più comune: login.

canActivateChild Posso entrare nelle figlie di questa rotta? Controllata a ogni cambio tra figlie.

canDeactivate Posso uscire da questa pagina? Per le modifiche non salvate.

canMatch Questa rotta esiste per me? Se dice no, il router prova la rotta successiva. Con loadChildren impedisce anche di scaricare il codice.

Guardia di uscita per un form con modifiche non salvate:

```
import { CanDeactivateFn } from '@angular/router';

export interface ConModifiche {
  haModificheNonSalvate(): boolean;
}

export const modificheGuard: CanDeactivateFn<ConModifiche> = (componente) =>
  componente.haModificheNonSalvate()
    ? confirm('Ci sono modifiche non salvate. Vuoi uscire?')
    : true;

{ path: 'eroi/:id/modifica', component: ModificaEroe, canDeactivate: [modificheGuard] }

export class ModificaEroe implements ConModifiche {
  form = inject(FormBuilder).group({ nome: [''] });
  haModificheNonSalvate() { return this.form.dirty; }
}
```

Le guardie di accesso vere, legate al login, sono nel capitolo 16.

Resolver: dati pronti prima della pagina

Un resolver carica i dati prima che la pagina appaia. Il vantaggio: il componente riceve già l'eroe e non deve gestire lo stato di caricamento. Lo svantaggio: finché la richiesta non finisce, l'utente resta sulla pagina precedente. ng g resolver eroe.

```
import { inject } from '@angular/core';
import { ResolveFn } from '@angular/router';
import { Eroee } from '../eroee';
import { Eroeservizio } from '../servizi/eroeservizio';

export const eroeeResolver: ResolveFn<Eroee | undefined> = (route) => {
  const id = Number(route.paramMap.get('id'));
  return inject(Eroeservizio).getEroee(id);
};

{ path: 'eroi/:id', component: DettaglioEroee, resolve: { eroee: eroeeResolver } }

export class DettaglioEroee {
  eroee = input<Eroee>(); // stesso nome della chiave in resolve
}
```

Senza input binding si legge con `route.snapshot.data['eroee']` oppure `route.data.pipe(map(d => d['eroee']))`.

Spinner durante il cambio pagina

Il router emette eventi. Ascoltandoli si sa quando una navigazione inizia e finisce, utile con lazy loading e resolver lenti.

```
import { NavigationCancel, NavigationEnd, NavigationError, NavigationStart, Router } from
  '@angular/router';
import { toSignal } from '@angular/core/rxjs-interop';
import { filter, map } from 'rxjs';

export class App {
  private router = inject(Router);
  caricamento = toSignal(
    this.router.events.pipe(
      filter((e) => e instanceof NavigationStart || e instanceof NavigationEnd
        || e instanceof NavigationCancel || e instanceof NavigationError),
      map((e) => e instanceof NavigationStart),
    ),
    { initialValue: false },
  );
}

@if (caricamento()) { <div class="barra-caricamento"></div> }
```

Errori tipici del routing

La pagina è vuota Manca `<router-outlet />` nel template di App, oppure RouterOutlet non è negli imports.

Il link ricarica tutta la pagina Hai usato href invece di routerLink, o RouterLink non è negli imports.

input() del parametro è undefined Manca `withComponentInputBinding()` in `provideRouter`, oppure il nome dell'input è diverso da quello dopo i due punti.

Ogni indirizzo apre la 404 La rotta `**` non è l'ultima.

Il dettaglio non si aggiorna cambiando id Stai leggendo snapshot in `ngOnInit`. Usa `input()` o `paramMap`.

Il redirect entra in loop Una rotta con path vuoto senza `pathMatch: 'full'`, oppure una guardia che rimanda alla stessa pagina protetta.

Error NG04002: Cannot match any routes Nessuna rotta corrisponde e manca la `**`. Controlla anche le barre nei link relativi.

13. Form template-driven

Due modi di fare i form

Angular ha due sistemi per i form. Usano gli stessi mattoni (FormControl, FormGroup, validatori, stati), ma li costruiscono in posti diversi.

Template-driven Il form si descrive nell'HTML con ngModel e attributi come required. Angular crea da solo i controlli. Poco codice, ideale per form semplici o medi. Modulo: FormsModule.

Reactive Il form si costruisce nella classe con FormBuilder e si collega al template con formGroup e FormControlName. Più codice, ma tutto è esplicito e testabile. Modulo: ReactiveFormsModule. È quello usato nel login del capitolo 16.

Nella versione 21 esistono anche i Signal Forms, ancora sperimentali: un terzo sistema basato sui signal. Per imparare conviene partire da questi due.

In questo capitolo c'è un esempio completo e funzionante nel progetto corso-angular: il componente src/app/pagine/iscrizione, aperto dalla rotta /iscrizione e dal link Iscrizione (form) nel menu.

Accendere i form

```
import { FormsModule } from '@angular/forms';

@Component({
  selector: 'app-iscrizione',
  imports: [FormsModule],
  templateUrl: './iscrizione.html',
})
```

Con FormsModule negli imports succedono due cose da sole: ogni tag <form> del template diventa un NgForm, cioè un FormGroup che contiene tutti i campi, e ogni campo con ngModel e name diventa un FormControl registrato dentro quel form.

Il form più piccolo

```
export class Contatto {
  modello = { nome: '', email: '' };

  invia(f: NgForm) {
    console.log(f.value); // { nome: '...', email: '...' }
    console.log(this.modello); // stesso contenuto, grazie a [(ngModel)]
  }
}

<form #f="ngForm" (ngSubmit)="invia(f)">
  <input name="nome" [(ngModel)]="modello.nome" required>
  <input name="email" type="email" [(ngModel)]="modello.email" required email>
  <button type="submit">Invia</button>
</form>
```

name Obbligatorio dentro un form. È la chiave con cui il campo compare in f.value. Senza, Angular dà errore.

ngModel Registra il campo nel form, senza collegarlo a una proprietà.

[(ngModel)]="x" Un solo senso: dalla classe al campo.

[(ngModel)]="x" Due sensi: il campo scrive nella classe e la classe nel campo.

[(ngModel)]="nome" con nome = signal('') Funziona anche con un signal, senza parentesi. Per un modello a oggetto con campi annidati è più semplice un oggetto normale.

Due fonti per gli stessi dati: this.modello (le tue proprietà, aggiornate da ngModel) e f.value (l'oggetto del form, che segue i name e i gruppi). Di solito si usa il modello.

Riferimenti di template: # con ngForm, ngModel, ngModelGroup

Il cancelletto dà un nome a qualcosa del template. Quello che si ottiene dipende da cosa c'è dopo l'uguale.

#f="ngForm" Il form intero, classe NgForm. Dà valid, submitted, value, resetForm().

#nome="ngModel" Il singolo campo, classe NgModel. Dà valid, touched, errors, hasError().

#indirizzo="ngModelGroup" Un gruppo di campi, classe NgModelGroup. Stati del gruppo e errori dei validatori di gruppo.

#campo senza valore L'elemento HTML, HTMLInputElement. Utile per il focus, non per la validazione.

ngForm, ngModel e ngModelGroup sono gli exportAs delle direttive: dicono ad Angular quale oggetto consegnare. Se scrivi #nome="ngModel" su un input senza ngModel, l'errore è No directive found with exportAs 'ngModel'.

```
<input name="nome" [(ngModel)]="modello.nome" #nome="ngModel" required minlength="2">
<p>Valido: {{ nome.valid }} - Toccato: {{ nome.touched }} - Errori: {{ nome.errors | json }}</p>
```

Un riferimento si può usare ovunque nello stesso template, anche prima della riga dove è dichiarato, e si può passare a un metodo: (ngSubmit)="invia(f)".

I tipi di campo

```
<!-- numero: il valore arriva come number, non come stringa -->
<input type="number" name="eta" [(ngModel)]="modello.eta" min="18" max="99">

<!-- select con stringhe -->
<select name="livello" [(ngModel)]="modello.livello" required>
  <option value="" disabled>Scegli...</option>
  @for (l of livelli; track l) { <option [value]="l">{{ l }}</option> }
</select>

<!-- select con oggetti: [ngValue] invece di [value] -->
<select name="corso" [(ngModel)]="modello.corso">
  @for (c of corsi; track c.id) { <option [ngValue]="c">{{ c.titolo }}</option> }
</select>

<!-- radio: stesso name, value diversi -->
<label><input type="radio" name="modalita" value="presenza" [(ngModel)]="modello.modalita">
  Presenza</label>
<label><input type="radio" name="modalita" value="online" [(ngModel)]="modello.modalita"> Online</label>

<!-- checkbox: il valore è true o false -->
<input type="checkbox" name="privacy" [(ngModel)]="modello.privacy" required>

<!-- textarea -->
<textarea name="note" [(ngModel)]="modello.note" maxlength="200"></textarea>
<small>{{ modello.note.length }}/200</small>
```

[value] mette nel modello una stringa. [ngValue] mette l'oggetto intero: con [value] un oggetto diventerebbe la stringa [object Object].

Gruppi di campi: ngModelGroup

```
<fieldset ngModelGroup="indirizzo" #indirizzo="ngModelGroup">
  <input name="citta" [(ngModel)]="modello.indirizzo.citta" required>
  <input name="cap" [(ngModel)]="modello.indirizzo.cap" required pattern="\d{5}">
  <small>Gruppo valido: {{ indirizzo.valid }}</small>
</fieldset>
```

f.value diventa { ..., indirizzo: { citta: '...', cap: '...' } }. Il gruppo ha i suoi stati: è valido solo se tutti i suoi campi lo sono. È anche il posto dove mettere un validatore che confronta più campi, come le due password.

Il submit

```
<form #f="ngForm" (ngSubmit)="invia(f)" novalidate>
  ...
  <button type="submit" [disabled]="f.pending">Iscriviti</button>
  <button type="button" (click)="svuota()">Svuota</button>
</form>

invia(f: NgForm) {
  if (f.invalid || f.pending) {
    f.control.markAllAsTouched(); // mostra tutti gli errori
    return;
  }
  this.iscrizioni.salva(this.modello).subscribe({
    next: () => this.inviata.set(true),
    error: () => this.errore.set('Salvataggio non riuscito'),
  });
}
```

(ngSubmit) Parte con il bottone `type="submit"` o premendo Invio in un campo. Blocca il ricaricamento della pagina che il browser farebbe con un submit normale.

`type="button"` Per ogni bottone che non deve inviare. Senza `type`, un bottone dentro un form è un submit.

`novalidate` Spegne i fumetti di errore del browser, così li mostra solo Angular. I validatori di Angular funzionano lo stesso.

`f.submitted` Diventa true al primo invio, anche se il form non è valido. Serve per mostrare gli errori dei campi mai toccati.

`[disabled]="f.invalid"` Alternativa comune, ma l'utente non capisce perché il bottone è spento. Meglio lasciarlo attivo e mostrare gli errori all'invio, disattivandolo solo durante il controllo asincrono.

Stati del form e dei campi

Ogni campo, gruppo e form ha gli stessi stati. Sono coppie di opposti.

`valid` / `invalid` Tutti i validatori passano oppure no.

`pending` Un validatore asincrono sta ancora lavorando. In quel momento il campo non è né `valid` né `invalid`.

`disabled` / `enabled` Un campo disabilitato non si valida e non compare in `f.value`.

`pristine` / `dirty` L'utente non ha ancora cambiato il valore, oppure sì. Cambiarlo dal codice non rende `dirty`.

`untouched` / `touched` L'utente non è ancora uscito dal campo, oppure sì (evento `blur`).

`submitted` Solo sul form, `NgForm`: è stato inviato almeno una volta.

`status` Il riassunto in una parola: `'VALID'`, `'INVALID'`, `'PENDING'`, `'DISABLED'`.

`errors` null se valido, altrimenti un oggetto con una chiave per ogni errore: `{ required: true }`.

`value` Il valore attuale.

Angular mette anche delle classi CSS su ogni campo, gruppo e form: `ng-valid`, `ng-invalid`, `ng-pending`, `ng-pristine`, `ng-dirty`, `ng-untouched`, `ng-touched` e, sul form, `ng-submitted`. Si possono usare direttamente nello scss.

```
input.ng-invalid.ng-touched { border-color: #c62828; } /* rosso dopo l'uscita dal campo */
input.ng-valid.ng-dirty    { border-color: #2e7d32; } /* verde se modificato e valido */
input.ng-pending          { border-color: #f9a825; } /* giallo durante il controllo */
form.ng-submitted .ng-invalid:not(form):not(fieldset) { border-color: #c62828; }
```

Quando mostrare un errore

Mostrare gli errori appena si apre la pagina è sgradevole: tutti i campi sono vuoti e quindi tutti rossi. La regola usata nell'esempio: mostra l'errore se il campo non è valido e l'utente ci è già passato, oppure ha già provato a inviare.

```
@if (nome.invalid && (nome.touched || f.submitted)) {
  <div class="errore">
    @if (nome.hasError('required')) { Il nome è obbligatorio. }
    @if (nome.hasError('minlength')) {
      Almeno {{ nome.errors?.['minlength'].requiredLength }} caratteri
      (ora {{ nome.errors?.['minlength'].actualLength }}).
    }
  </div>
}
```

`hasError('x')` è il modo comodo per chiedere un errore preciso. `errors?.['x']` dà anche i dettagli dell'errore.

Validatori pronti

Si scrivono come attributi del campo. Ognuno, se fallisce, aggiunge una chiave in `errors`.

required { required: true }. Campo vuoto. Su una checkbox vuol dire che deve essere spuntata.

minlength="2" { minlength: { requiredLength: 2, actualLength: 1 } }.

maxlength="30" { maxlength: { ... } }. Il browser impedisce già di scrivere oltre.

min="18" / max="99" { min: { min: 18, actual: 15 } }. Per i campi numerici.

email { email: true }. Formato di un indirizzo email.

pattern="\d{5}" { pattern: { requiredPattern: '^\\d{5}'

Falli in ordine nel progetto corso-angular. Ognuno usa quello precedente.

1. Aggiungi `withComponentInputBinding` in `app.config.ts` e trasforma Dettaglio perché legga id con `input.required({ transform: numberAttribute })` al posto di `route.snapshot`.
2. Aggiungi una pagina `NonTrovata` e la rotta `**` in fondo. Prova un indirizzo inventato.
3. Metti `routerLinkActive` sui link di `app.html` con una classe `attivo` nello `scss`.
4. Nella lista persone aggiungi un campo di ricerca che scrive `?cerca=` nell'indirizzo, come nell'esempio Eroi. Ricarica la pagina: il filtro deve restare.
5. Crea un componente `CardPersona` con `input.required<Persona>()` e `output<number>()` per l'eliminazione. Usalo nella lista.
6. Crea un `Contatore` con `model()` e collegalo a un signal del padre con `[(valore)]`.
7. Crea un `PreferitiServizio` con un signal privato, `asReadonly` e `computed` per il numero. Un bottone nella card aggiunge ai preferiti, un badge in `app.html` mostra il totale.
8. Installa `json-server`, crea `db.json` con le persone e scrivi un servizio CRUD con `HttpClient`. Mostra caricamento ed errore. Spegni `json-server` e verifica il messaggio `Server non raggiungibile`.
9. Aggiungi l'autenticazione completa di questo capitolo con il backend finto. Proteggi `pagina2` con `authGuard` e crea una sezione admin con rotte figlie e `ruoloGuard`.
10. Aggiungi `modificheGuard` a un form di modifica e verifica che chieda conferma prima di uscire.

, `actualValue: '12' }` }. Angular aggiunge da solo `^` e `$`: deve combaciare tutto il valore.

[required]="condizione" Validatore acceso solo quando la condizione è vera. Vale anche **[minlength]="n"** e gli altri.

Validatore personalizzato su un campo

Nei form template-driven un validatore nuovo è una direttiva. Si registra nel token `NG_VALIDATORS`, così Angular lo chiama insieme a `required` e agli altri. Esempio: vietare un nome preciso.

```
import { Directive, input } from '@angular/core';
import { AbstractControl, NG_VALIDATORS, ValidationErrors, Validator } from '@angular/forms';

@Directive({
  selector: '[appValoreVietato]',
  providers: [{ provide: NG_VALIDATORS, useExisting: ValoreVietato, multi: true }],
})
export class ValoreVietato implements Validator {
  appValoreVietato = input('');

  validate(control: AbstractControl): ValidationErrors | null {
    const vietato = this.appValoreVietato().toLowerCase();
    const valore = String(control.value ?? '').trim().toLowerCase();
    return vietato && valore === vietato ? { valoreVietato: { vietato } } : null;
  }
}

imports: [FormsModule, ValoreVietato],

<input name="nome" [(ngModel)]="modello.nome" #nome="ngModel" appValoreVietato="admin">
@if (nome.hasError('valoreVietato')) { "{{ nome.errors?.['valoreVietato'].vietato }}" non è permesso. }
```

validate Restituisce null se va bene, altrimenti un oggetto errore. La chiave, qui `valoreVietato`, è quella che leggi con `hasError`.

multi: true `NG_VALIDATORS` è un elenco: aggiungi il tuo senza togliere gli altri.

useExisting Usa l'istanza della direttiva già creata su quel campo, così legge il suo input.

input con lo stesso nome del selettore `appValoreVietato="admin"` è insieme l'attivazione della direttiva e il suo parametro.

Validatore su un gruppo: password uguali

Un validatore messo su `ngModelGroup` riceve il gruppo intero e può confrontare i campi.

```
@Directive({
  selector: '[appPasswordUguali]',
  providers: [{ provide: NG_VALIDATORS, useExisting: PasswordUguali, multi: true }],
})
export class PasswordUguali implements Validator {
  validate(gruppo: AbstractControl): ValidationErrors | null {
    const password = gruppo.get('password')?.value;
    const conferma = gruppo.get('conferma')?.value;
    return password && conferma && password !== conferma ? { passwordDiverse: true } : null;
  }
}

<fieldset ngModelGroup="credenziali" #credenziali="ngModelGroup" appPasswordUguali>
  <input type="password" name="password" [(ngModel)]="modello.credenziali.password" required
  minlength="6">
  <input type="password" name="conferma" [(ngModel)]="modello.credenziali.conferma" #conferma="ngModel"
  required>
  @if (credenziali.hasError('passwordDiverse') && (conferma.touched || f.submitted)) {
    <div class="errore">Le due password non coincidono.</div>
  }
</fieldset>
```

L'errore sta sul gruppo, non sul campo conferma: per questo si legge da `credenziali.hasError`. `gruppo.get('password')` usa il name del campo.

Validatore asincrono: controllo sul server

Alcuni controlli richiedono il server, per esempio un'email già registrata. Un validatore asincrono restituisce un `Observable` e si registra in `NG_ASYNC_VALIDATORS`. Mentre aspetta, il campo e il form sono pending.

```

@Directive({
  selector: '[appEmailLibera]',
  providers: [{ provide: NG_ASYNC_VALIDATORS, useExisting: EmailLibera, multi: true }],
})
export class EmailLibera implements AsyncValidator {
  private http = inject(HttpClient);

  validate(control: AbstractControl): Observable<ValidationErrors | null> {
    return this.http
      .get<{ libera: boolean }>(`/api/email-libera?email=${control.value}`)
      .pipe(map((r) => (r.libera ? null : { emailUsata: true })));
  }
}

<input type="email" name="email" [(ngModel)]="modello.email" #email="ngModel"
      [ngModelOptions]="{ updateOn: 'blur' }" required email appEmailLibera>
@if (email.pending) { <div>Controllo in corso...</div> }
@if (email.hasError('emailUsata')) { <div class="errore">Email già registrata.</div> }

```

Nell'esempio del progetto la chiamata HTTP è simulata con `timer(800)` e un elenco fisso, così funziona senza server. I validatori asincroni partono solo se quelli sincroni sono tutti passati: con un'email malformata la richiesta non parte.

ngModelOptions: quando aggiornare

`{ updateOn: 'change' }` Predefinito: valore e validazione a ogni tasto.

`{ updateOn: 'blur' }` Solo quando esci dal campo. Ideale con un validator asincrono: una richiesta sola invece di una per lettera.

`{ updateOn: 'submit' }` Solo all'invio del form.

`{ standalone: true }` Il campo non entra nel form e non serve name. Per un campo di servizio, per esempio un filtro dentro il form.

Si può mettere anche su tutto il form: `<form [ngFormOptions]="{ updateOn: 'blur' }">`.

Prendere il form con viewChild

Il riferimento `#f` basta nel template. Per usare il form nel codice della classe, fuori dal submit, si prende con `viewChild` usando lo stesso nome.

```

import { AfterViewInit, Component, DestroyRef, inject, viewChild } from '@angular/core';
import { NgForm, NgModel } from '@angular/forms';
import { takeUntilDestroyed } from '@angular/core/rxjs-interop';

export class Iscrizione implements AfterViewInit {
  private destroyRef = inject(DestroyRef);

  form = viewChild.required<NgForm>('iscrizioneForm'); // <form #iscrizioneForm="ngForm">
  campoEmail = viewChild<NgModel>('email'); // <input #email="ngModel">
  modifiche = signal(0);

  ngAfterViewInit() {
    this.form().valueChanges
      ?.pipe(takeUntilDestroyed(this.destroyRef))
      .subscribe(() => this.modifiche.update((n) => n + 1));
  }

  svuota() {
    const vuoto = this.modelloVuoto();
    this.form().resetForm(vuoto);
    this.modello = vuoto;
  }
}

```

Con il decoratore classico la stessa cosa si scrive `@ViewChild('iscrizioneForm') form!: NgForm;` e si legge `this.form` senza parentesi.

Il tipo tra le angulari Decide cosa ricevi. Il nome tra apici deve essere quello scritto dopo #. Il valore dopo l'uguale (ngForm, ngModel) decide la classe.

Quando è pronto Da ngAfterViewInit in poi. Nel constructor la vista non esiste ancora.

Controlli registrati in ritardo ngModel registra i campi nel form un attimo dopo, in modo asincrono. In ngAfterViewInit this.form().value può essere ancora vuoto: iscriversi a valueChanges va bene, leggere i valori subito no.

resetForm(valori) Rimette i valori passati e azzerata touched, dirty e submitted. Senza argomento i campi diventano null.

valueChanges / statusChanges Observable che emettono a ogni cambio di valore o di stato. Si staccano con takeUntilDestroyed.

Perché resetForm con i valori: resetForm() senza argomento mette tutti i campi a null. Se poi riassegna al modello un valore uguale a quello di prima, per esempio il radio modalita che era e resta 'presenza', ngModel non vede cambiamenti e non lo riscrive nel campo: il radio resta deselezionato. Passando il modello vuoto a resetForm, form e modello ripartono uguali.

Perché non patchValue: in un form template-driven la fonte dei dati è il modello. form.patchValue(...) cambierebbe i campi ma non this.modello, e i due andrebbero fuori sincrono. Per precompilare si cambia il modello: this.modello = { ...this.modello, nome: 'Luigi' }.

L'esempio completo nel progetto

File in src/app/pagine/iscrizione:

iscrizione.ts Modello, viewChild del form e del campo email, submit, precompila, svuota, contatore di valueChanges.

iscrizione.html Tutti i tipi di campo, i due gruppi, i messaggi di errore e il pannello degli stati.

iscrizione.scss Le classi ng-* per colorare i campi.

validatori.ts Le tre direttive: ValoreVietato, PasswordUguali, EmailLibera.

Prove da fare con ng serve su http://localhost:4200/iscrizione:

1. Premi Iscriviti con il form vuoto: submitted diventa true e compaiono tutti gli errori. 2. Scrivi admin come nome: errore del validator personalizzato. 3. Scrivi mario@esempio.it nell'email ed esci dal campo: per un attimo pending è true, poi arriva l'errore emailUsata. 4. Scrivi due password diverse: errore sul gruppo credenziali. 5. Premi Precompila, completa password e privacy e invia: compare il riquadro verde. 6. Premi Svuota: valori iniziali, pristine, untouched e submitted false. 7. Guarda il pannello a destra: iscrizioneForm.value e modello cambiano insieme mentre scrivi.

Pagina di esempi: tutte le forme di ngModel

Seconda pagina nel progetto: src/app/pagine/esempi-ngmodel, rotta /ngmodel, link Esempi ngModel nel menu. Ogni riquadro mostra una variante. Fuori da un <form> ngModel non ha bisogno di name.

1. [(ngModel)] su una proprietà normale. Il bottone cambia la proprietà dal codice e il campo si aggiorna: il legame va nei due sensi.

```
nome = 'Mario';

<input [(ngModel)]="nome">
<p>Ciao {{ nome || 'sconosciuto' }}!</p>
<button type="button" (click)="nome = 'Luigi'">Imposta Luigi</button>
```

2. [(ngModel)] su un signal. Si passa il signal senza parentesi; nel resto del template si legge con le parentesi e si può usare in computed.

```

citta = signal('Roma');
lettereCitta = computed(() => this.citta().length);

<input [(ngModel)]="citta">
<p>{{ citta() }} ha {{ lettereCitta() }} lettere.</p>

```

3. La forma lunga: `[(ngModel)]` per scrivere nel campo, `(ngModelChange)` per ricevere il nuovo valore. `[(ngModel)]="x"` è la scorciatoia di `[(ngModel)]="x" (ngModelChange)="x = $event"`. La forma lunga serve quando il valore va trasformato o controllato prima di salvarlo.

```

codice = '';
aggiornaCodice(valore: string) {
  this.codice = valore.toUpperCase().replace(/[^A-Z0-9]/g, '');
}

<input [(ngModel)]="codice" (ngModelChange)="aggiornaCodice($event)">

```

Attenzione: se la trasformazione produce lo stesso valore di prima, per esempio scrivendo un trattino che viene tolto, `ngModel` non vede cambiamenti e il trattino resta visibile nel campo finché non scrivi un'altra lettera.

4. Due campi legati alla stessa proprietà: uno slider e un campo numerico. Muovi uno e si aggiorna l'altro. Con `type="number"` e `type="range"` il valore arriva già come numero.

```

volume = 30;

<input type="range" min="0" max="100" [(ngModel)]="volume">
<input type="number" min="0" max="100" [(ngModel)]="volume">

```

5. Checkbox. Una sola è un booleano. Per scelte multiple non esiste un `ngModel` su array: si usa la forma lunga su ogni casella.

```

interessi: string[] = ['Angular'];
scelto(voce: string) { return this.interessi.includes(voce); }
cambiaInteresse(voce: string, attivo: boolean) {
  this.interessi = attivo ? [...this.interessi, voce] : this.interessi.filter((i) => i !== voce);
}

<input type="checkbox" [(ngModel)]="newsletter">
@for (voce of interessiDisponibili; track voce) {
  <input type="checkbox" [(ngModel)]="scelto(voce)" (ngModelChange)="cambiaInteresse(voce, $event)"> {{ voce }}
}

```

6. Select con oggetti e `compareWith`. `[(ngValue)]` mette nel modello l'oggetto intero. Ma se l'oggetto selezionato arriva da un'altra fonte, per esempio dal server, non è lo stesso oggetto dell'elenco anche se ha lo stesso id: senza `compareWith` il select resterebbe vuoto.

```

corsoScelto: Corso | null = { id: 2, titolo: 'Angular avanzato', ore: 24 };
stessoCorso = (a: Corso | null, b: Corso | null) => a?.id === b?.id;

<select [(ngModel)]="corsoScelto" [compareWith]="stessoCorso">
  <option [(ngValue)]="null">Nessun corso</option>
  @for (c of corsi; track c.id) { <option [(ngValue)]="c">{{ c.titolo }}</option> }
</select>

```

7. Radio: stesso name, `[value]` diverso, stesso `[(ngModel)]`. 8. `[(ngModelOptions)]="{ updateOn: 'blur' }"`: la proprietà cambia solo quando esci dal campo.
9. Dentro un form, `ngModel` senza binding. Il campo è registrato nel form, ma nessuna proprietà della classe lo segue: i valori si leggono da `f.value` al submit. `[(ngModel)]=""` dà solo il valore iniziale. Dopo l'aggiunta, `resetForm` svuota il form e rimette la priorità predefinita.

```

aggiungi(f: NgForm) {
  if (f.invalid) return;
  const { testo, priorit  } = f.value;
  this.promemoria.update((lista) => [...lista, { id: this.prossimoId++, testo, priorit , fatto: false }]);
  f.resetForm({ testo: '', priorit : 'bassa' });
}

<form #f="ngForm" (ngSubmit)="aggiungi(f)">
  <input name="testo" ngModel required minlength="3">
  <select name="priorit " [ngModel]="'bassa'">...</select>
  <button type="submit" [disabled]="f.invalid">Aggiungi</button>
</form>

@for (p of promemoria(); track p.id) {
  <input type="checkbox" [ngModel]="p.fatto" (ngModelChange)="segnaFatto(p.id, $event)"> {{ p.testo }}
}

```

Le caselle della lista stanno fuori dal form, quindi non serve name. Se fossero dentro, ognuna avrebbe bisogno di un name diverso, per esempio [name]='fatto' + p.id', altrimenti si sovrascriverebbero. La lista   un signal, quindi segnaFatto crea un array nuovo con map invece di modificare p.fatto direttamente.

Errori tipici con i form template-driven

Can't bind to 'ngModel' since it isn't a known property of 'input' Manca FormsModule negli imports del componente.

If ngModel is used within a form tag, either the name attribute must be set... Manca name sul campo. Oppure usa [ngModelOptions]='{ standalone: true }'.

No directive found with exportAs 'ngModel' #x="ngModel" su un elemento senza ngModel.

Due campi si sovrascrivono Hanno lo stesso name nello stesso form o gruppo. Vale solo per i radio, che devono dividerlo.

Il select mostra [object Object] Usa [ngValue] per gli oggetti.

Il validatore personalizzato non parte La direttiva non   negli imports del componente, o manca providers con NG_VALIDATORS.

NG0500 During hydration Angular expected <tr> but found <tbody> Con SSR, una <table> senza <tbody> nel template. Il browser aggiunge tbody da solo e il DOM non coincide pi  con quello che Angular si aspetta: l'hydration si ferma e la pagina smette di reagire. Scrivi sempre <tbody>.   successo davvero costruendo il pannello degli stati.

Il submit ricarica la pagina Manca FormsModule, quindi il form non   un NgForm e ngSubmit non esiste.

14. Form reactive

Cosa sono

Nei form reactive il form si costruisce nella classe TypeScript: ogni campo è un oggetto `FormControl`, i campi si raggruppano in un `FormGroup`, le regole di validazione sono funzioni passate al momento della creazione. Il template non crea niente: collega i tag HTML ai controlli già esistenti, per nome.

La differenza con i template-driven è dove sta la verità. Là la fonte dei dati è il tuo modello e `ngModel` lo sincronizza; il form viene dedotto dall'HTML. Qui la fonte dei dati è il form stesso: leggi e scrivi i valori direttamente dai controlli, in modo sincrono, senza un modello separato.

Template-driven Form descritto nell'HTML. Poco codice. Validatori personalizzati come direttive. Ideale per form semplici.

Reactive Form descritto nella classe. Tutto esplicito e tipizzato. Validatori come funzioni. Facile aggiungere o togliere campi a runtime, reagire ai cambi con `RxJS`, scrivere test senza template. Ideale per form medi e grandi, e per campi dinamici.

Quando scegliere reactive: il form ha campi che compaiono o cambiano regole in base ad altri campi, liste di lunghezza variabile, validazioni tra più campi, oppure vuoi testare la logica senza il browser. Nei progetti di lavoro è la scelta più diffusa.

Esempio completo nel progetto: `src/app/pagine/iscrizione-reactive`, rotta `/iscrizione-reactive`, link `Iscrizione (reactive)` nel menu. È lo stesso form della pagina `/iscrizione`, rifatto in versione reactive, così puoi confrontarli campo per campo. In più ha una lista di telefoni (`FormArray`), un campo che si abilita solo in certi casi e un `FormControl` fuori dal form.

Accendere i form reactive

```
import { ReactiveFormsModule } from '@angular/forms';

@Component({
  selector: 'app-iscrizione-reactive',
  imports: [ReactiveFormsModule],
  templateUrl: './iscrizione-reactive.html',
})
```

`ReactiveFormsModule` dà le direttive `[formGroup]`, `formControlName`, `formGroupName`, `formArrayName` e `[formControl]`. Nello stesso campo non mescolare `formControlName` con `ngModel`: Angular lo sconsiglia e stampa un avviso. Scegli un sistema per ogni form.

I mattoni

FormControl Un singolo campo: valore, stato, errori.

FormGroup Un oggetto di controlli con un nome ciascuno. Il suo valore è un oggetto `{ nome: ..., email: ... }`. Può contenere altri `FormGroup`.

FormArray Una lista di controlli senza nome, indicizzati da 0. Il suo valore è un array. Serve per elenchi di lunghezza variabile: telefoni, righe d'ordine, allegati.

AbstractControl La classe madre dei tre. Ha tutto ciò che hanno in comune: `value`, `status`, `errors`, `valid`, `touched`, `reset()`, `valueChanges`...

Tutti e tre hanno gli stessi stati e gli stessi metodi di base: una volta imparato il `FormControl`, sai già usare gruppi e liste.

Creare un FormGroup

Il form più piccolo possibile, per vedere la struttura.

```
import { Component } from '@angular/core';
import { FormControl, FormGroup, ReactiveFormsModule } from '@angular/forms';

@Component({
  selector: 'app-contatto',
  imports: [ReactiveFormsModule],
  template: `
    <form [formGroup]="form" (ngSubmit)="invia()">
      <input formControlName="nome">
      <input formControlName="email">
      <button type="submit">Invia</button>
    </form>
    <p>{{ form.value | json }}</p>
  `,
})
export class Contatto {
  form = new FormGroup({
    nome: new FormControl(''),
    email: new FormControl(''),
  });

  invia() {
    console.log(this.form.value); // { nome: '...', email: '...' }
  }
}
```

Le chiavi dell'oggetto passato a new FormGroup (nome, email) sono i nomi con cui il template collega i campi: formControlName="nome". Il valore iniziale è il primo argomento di new FormControl.

Il FormGroup si crea come campo della classe, non in ngOnInit: così esiste già quando Angular disegna il template, e TypeScript ne conosce la forma esatta.

Aggiungere un FormControl per ogni campo

Il costruttore di FormControl accetta il valore iniziale e un oggetto di opzioni.

```
nome: new FormControl('', {
  nonNullable: true,
  validators: [Validators.required, Validators.minLength(2)],
}),
email: new FormControl('', {
  nonNullable: true,
  validators: [Validators.required, Validators.email],
  asyncValidators: [emailLibera()],
  updateOn: 'blur',
}),
eta: new FormControl<number | null>(null, [Validators.required, Validators.min(18)]),
azienda: new FormControl({ value: '', disabled: true }, { nonNullable: true }),
```

nonNullable: true Il tipo del valore è string e non string | null, e reset() riporta al valore iniziale invece di mettere null. Usalo quasi sempre.

validators Elenco di validatori sincroni. Si può passare anche come secondo argomento diretto, come per eta.

asyncValidators Validatori asincroni: partono solo se quelli sincroni sono passati.

updateOn 'change' (predefinito), 'blur' o 'submit': quando aggiornare valore e validazione.

FormControl<number> null | Tipo esplicito. Serve quando il valore iniziale (null) non basta a TypeScript per capire il tipo.

{ value: '', disabled: true } Controllo che nasce disabilitato.

I form sono tipizzati: form.value.nome è string, form.controls.eta.value è number | null. Se sbagli il nome di un campo nel codice, l'errore arriva in compilazione.

Aggiungere un gruppo annidato e una lista è la stessa cosa, un livello più giù:

```
credenziali: new FormGroup({
  password: new FormControl('', { nullable: true, validators: [Validators.required,
    Validators.minLength(6)] }),
  conferma: new FormControl('', { nullable: true, validators: [Validators.required] }),
}),
{ validators: passwordUguali }, // validatore del gruppo
),
telefoni: new FormArray([new FormControl('', { nullable: true })]),
```

Si possono anche aggiungere e togliere controlli dopo la creazione: `form.addControl('coupon', new FormControl(''))`, `form.removeControl('coupon')`. Con i form tipizzati si preferisce dichiararli tutti subito e disabilitare quelli che non servono.

La stessa cosa con FormBuilder

FormBuilder è una scorciatoia per scrivere meno new. Il risultato è identico.

```
private fb = inject(FormBuilder).nonNullable;

form = this.fb.group({
  nome: ['', [Validators.required, Validators.minLength(2)]],
  email: ['', { validators: [Validators.required, Validators.email], updateOn: 'blur' }],
  credenziali: this.fb.group({
    password: ['', Validators.required], conferma: [''] },
    { validators: passwordUguali },
  ),
  telefoni: this.fb.array([this.fb.control('')]),
});
```

Ogni campo è un array: [valore iniziale, validatori, validatori asincroni]. `.nonNullable` rende `nonNullable` tutti i controlli creati. Il login del capitolo 16 usa FormBuilder. Per imparare, `new FormGroup` e `new FormControl` mostrano meglio cosa succede.

Sincronizzare la parte HTML

Il template collega ogni tag al controllo con lo stesso nome. Le direttive seguono la struttura del FormGroup, livello per livello.

[formGroup]="form" Sul tag `<form>`. Collega il FormGroup principale. Le parentesi quadre perché passi un oggetto della classe.

formControlName="nome" Su un campo. Cerca il controllo nome nel gruppo del livello attuale. Senza quadre: è una stringa.

formGroupName="credenziali" Su un contenitore (fieldset, div). Da qui in giù i `formControlName` sono relativi a quel gruppo.

formArrayName="telefoni" Su un contenitore. Da qui in giù i campi si collegano con l'indice.

[formControlName]="i" Dentro `formArrayName`: l'elemento numero `i` della lista. Quadre perché `i` è un numero.

[formControl]="ricerca" Collega un FormControl singolo, anche fuori da qualsiasi form.

```
<form [formGroup]="form" (ngSubmit)="invia()" novalidate>
  <input formControlName="nome">
  <input type="email" formControlName="email">
  <input type="number" formControlName="eta">

  <fieldset formGroupName="credenziali">
    <input type="password" formControlName="password">
    <input type="password" formControlName="conferma">
  </fieldset>

  <select formControlName="livello">
    <option value="" disabled>Scegli...</option>
```

```

    @for (l of livelli; track l) { <option [value]="l">{{ l }}</option> }
  </select>

  <label><input type="radio" formControlName="modalita" value="presenza"> In presenza</label>
  <label><input type="radio" formControlName="modalita" value="online"> Online</label>

  <input type="checkbox" formControlName="privacy">

  <fieldset formArrayName="telefoni">
    @for (tel of telefoni.controls; track tel; let i = $index) {
      <input [formControlName]="i">
      <button type="button" (click)="rimuoviTelefono(i)">Rimuovi</button>
    }
    <button type="button" (click)="aggiungiTelefono()">Aggiungi telefono</button>
  </fieldset>

  <button type="submit">Iscriviti</button>
</form>

```

La sincronizzazione va nei due sensi e parte subito: quello che l'utente scrive finisce nel controllo, e se il codice fa `setValue` il campo si aggiorna. Nel template non servono name, né `[(ngModel)]`, né variabili con `#`: tutto si legge dalla classe.

Per i select con oggetti vale `[ngValue]` come nei template-driven, e `[compareWith]` sul select per riconoscere un oggetto uguale per id.

Il FormControl fuori da un form, qui un campo di ricerca che filtra dopo 300 millisecondi di pausa:

```

ricerca = new FormControl('', { nullable: true });
corsiTrovati = toSignal(
  this.ricerca.valueChanges.pipe(
    debounceTime(300),
    distinctUntilChanged(),
    map((testo) => CORSI.filter((c) => c.toLowerCase().includes(testo.toLowerCase()))),
  ),
  { initialValue: CORSI },
);

<input [formControl]="ricerca" placeholder="Cerca un corso">
@for (c of corsiTrovati(); track c) { <li>{{ c }}</li> }

```

Questo è il punto forte dei form reactive: `valueChanges` è un Observable, quindi con `debounceTime`, `distinctUntilChanged` e `switchMap` si costruisce una ricerca che non fa una richiesta al server a ogni tasto.

Verificare il submit

```

invia() {
  if (this.form.invalid || this.form.pending) {
    this.form.markAllAsTouched(); // tutti i campi "toccati": compaiono gli errori
    return;
  }
  const dati = this.form.getRawValue();
  this.salvataggio.set(true);
  this.form.disable(); // blocca i campi durante l'invio
  this.iscrizioni.salva(dati).subscribe({
    next: () => { this.inviata.set(true); this.form.reset(); this.form.enable(); },
    error: () => { this.error.set('Salvataggio non riuscito'); this.form.enable(); },
    complete: () => this.salvataggio.set(false),
  });
}

```

`(ngSubmit)="invia()"` Senza argomenti: il form è già in `this.form`.

`form.invalid` Almeno un validatore non passa, in qualunque livello.

`form.pending` Un validatore asincrono è ancora in corso: non inviare, il risultato non è ancora noto.

`markAllAsTouched()` Segna come toccati tutti i controlli, anche annidati. Con la regola "mostra l'errore se invalid e touched" compaiono tutti insieme.

`form.value` Il valore senza i controlli disabilitati.

`form.getRawValue()` Il valore completo, disabilitati compresi. Tipizzato: `dati.nome` è string.

form.reset() Valori iniziali (con nonNullable), pristine, untouched. Si può passare un oggetto con nuovi valori.

form.disable() / **enable()** Durante l'invio evita modifiche e doppi click.

Il FormGroup non ha lo stato submitted. Se ti serve, prendi la direttiva: `<form [formGroup]="form" #fgd="ngForm">` e poi `fgd.submitted`, oppure tieni un tuo signal tentato = `signal(false)` impostato in `invia()`. Con `markAllAsTouched` di solito non serve.

Aggiungere la validazione

Validatori pronti, tutti in Validators:

Validators.required Valore vuoto. Errore { required: true }.

Validators.requiredTrue Per le checkbox: deve essere true.

Validators.minLength(2) / **maxLength(30)** Lunghezza del testo. Errore { minlength: { requiredLength, actualLength } }.

Validators.min(18) / **max(99)** Valore numerico. Errore { min: { min, actual } }.

Validators.email Formato email.

Validators.pattern(/^\d{5}\$/) Espressione regolare. Con una regex scrivi tu ^ e \$; con una stringa li aggiunge Angular.

Un validator personalizzato è una funzione: riceve il controllo e restituisce null se va bene, altrimenti un oggetto errore. Niente direttive, niente NG_VALIDATORS.

```
import { AbstractControl, ValidationErrors, ValidatorFn } from '@angular/forms';

// Con parametro: una funzione che crea il validator
export function valoreVietato(vietato: string): ValidatorFn {
  return (control: AbstractControl): ValidationErrors | null => {
    const valore = String(control.value ?? '').trim().toLowerCase();
    return valore === vietato.toLowerCase() ? { valoreVietato: { vietato } } : null;
  };
}

// Di gruppo: confronta due campi figli
export const passwordUguali: ValidatorFn = (gruppo) => {
  const password = gruppo.get('password')?.value;
  const conferma = gruppo.get('conferma')?.value;
  return password && conferma && password !== conferma ? { passwordDiverse: true } : null;
};

// Asincrono: restituisce un Observable
export function emailLibera(): AsyncValidatorFn {
  return (control) => timer(800).pipe(
    map(() => (EMAIL_GIA_REGISTRATE.includes(control.value) ? { emailUsata: true } : null)),
  );
}
```

Si usano passandoli al controllo: `validators: [Validators.required, valoreVietato('admin')]`, oppure al gruppo: `new FormGroup(..., { validators: passwordUguali })`. In un'app vera il validator asincrono chiama il server; per iniettare `HttpClient` crealo dentro il componente o un servizio, dove `inject()` è permesso, e passa la funzione al controllo.

Cambiare le regole a runtime:

```
const azienda = this.form.controls.azienda;
azienda.addValidators(Validators.required);
azienda.removeValidators(Validators.required);
azienda.setValidators([Validators.required, Validators.maxLength(80)]); // sostituisce tutti
azienda.updateValueAndValidity(); // ricalcola subito
```

Dopo `addValidators`, `removeValidators` o `setValidators` la validazione non si ricalcola da sola: serve `updateValueAndValidity()`. Per rimuovere un validatore bisogna passare lo stesso riferimento usato per aggiungerlo: con `valoreVietato('admin')` crea la funzione una volta, salvala in una costante e usa quella.

Spesso è più semplice abilitare e disabilitare il campo. Un controllo disabilitato non viene validato e non compare in `value`. Esempio del progetto: la ragione sociale serve solo se la fattura è per un'azienda.

```
constructor() {
  this.form.controls.fattura.valueChanges.pipe(takeUntilDestroyed()).subscribe((tipo) => {
    const azienda = this.form.controls.azienda;
    azienda.reset(); // vuoto e untouched: niente errore appena compare
    if (tipo === 'azienda') azienda.enable();
    else azienda.disable();
  });
}

@if (form.controls.azienda.enabled) {
  <input formControlName="azienda">
}
```

Perché `reset` prima di `enable`: `markAllAsTouched` dopo un invio fallito aveva segnato come toccato anche il campo disabilitato. Senza `reset`, appena abilitato mostrerebbe subito l'errore.

Accedere ai controlli nell'HTML

Per mostrare errori e stati serve arrivare al controllo giusto dal template. Ci sono cinque modi, tutti usati nell'esempio.

```
<!-- 1. form.controls: tipizzato, sempre disponibile -->
@if (form.controls.eta.hasError('min')) { Devi avere almeno 18 anni. }

<!-- 2. getter nella classe: il template resta corto -->
get nome() { return this.form.controls.nome; }
@if (nome.invalid && nome.touched) { ... }

<!-- 3. form.get con il percorso: comodo per i gruppi, ma restituisce AbstractControl | null -->
@if (form.get('indirizzo.citta')?.invalid && form.get('indirizzo.citta')?.touched) { ... }

<!-- 4. @let: una variabile di template per un controllo annidato -->
@let cap = form.controls.indirizzo.controls.cap;
@if (cap.hasError('pattern')) { Il CAP ha 5 cifre. }

<!-- 5. FormArray: i controlli sono in .controls, si scorrono con @for -->
@for (tel of telefoni.controls; track tel; let i = $index) {
  <input [formControlName]="i">
  @if (tel.invalid && tel.touched) { Numero non valido. }
}
```

`hasError('minlength')` true se c'è quell'errore. Con un percorso: `form.hasError('required', 'indirizzo.cap')`.

`getError('minlength')` L'oggetto dell'errore: `getError('minlength').requiredLength`.

`errors` Tutti gli errori: `{{ nome.errors | json }}` è utile mentre sviluppi.

`errore sul gruppo` `form.controls.credenziali.hasError('passwordDiverse')`: l'errore di un validatore di gruppo sta sul gruppo, non sui campi.

Per non ripetere la stessa condizione ovunque, un metodo nella classe:

```
mostraErrore(control: AbstractControl) {
  return control.invalid && control.touched;
}

@if (mostraErrore(nome)) {
  <div class="errore">
    @if (nome.hasError('required')) { Il nome è obbligatorio. }
    @if (nome.hasError('minlength')) {
      Almeno {{ nome.getError('minlength').requiredLength }} caratteri
      (ora {{ nome.getError('minlength').actualLength }}).
    }
  </div>
}
```

Preferisci `form.controls`, i getter o `@let a form.get('...')`: il percorso è una stringa, quindi un errore di battitura non viene segnalato dal compilatore e il risultato è sempre da controllare con `?`.

Leggere e scrivere dal codice

`this.form.value` / `getRawValue()` Il valore attuale, in qualsiasi momento, in modo sincrono.

`setValue({...})` Sostituisce tutto: devi passare ogni campo, altrimenti errore `Must supply a value for form control with name`.

`patchValue({...})` Cambia solo i campi indicati. È quello da usare per precompilare.

`reset(valori?)` Valori iniziali o quelli passati; `pristine` e `untouched`.

`valueChanges` Observable dei valori, a ogni modifica. Con `toSignal` diventa un signal.

`statusChanges` Observable dello stato: `'VALID'`, `'INVALID'`, `'PENDING'`, `'DISABLED'`.

`markAsTouched()` / `markAsDirty()` / `markAsPristine()` Cambiano gli stati a mano.

```
precompila() {
  this.form.patchValue({
    nome: 'Luigi',
    email: 'luigi@esempio.it',
    indirizzo: { città: 'Torino', cap: '10100' },
  });
}

stato = toSignal(this.form.statusChanges, { initialValue: this.form.status });
```

Nei template-driven `patchValue` creava problemi perché il modello separato non si aggiornava. Qui il form è la fonte dei dati: `patchValue` è il modo giusto.

Liste dinamiche: `FormArray`

```
function nuovoTelefono() {
  return new FormControl('', { nullable: true, validators: [Validators.pattern(/^\+?\d{6,15}$/)] });
}

telefoni: new FormArray([nuovoTelefono()]),

get telefoni() { return this.form.controls.telefoni; }
aggiungiTelefono() { this.telefoni.push(nuovoTelefono()); }
rimuoviTelefono(i: number) { this.telefoni.removeAt(i); }

svuota() {
  this.form.reset();
  this.telefoni.clear(); // reset non cambia il numero di elementi
  this.telefoni.push(nuovoTelefono());
}
```

`push(controllo)` Aggiunge in fondo. `insert(i, controllo)` in una posizione.

`removeAt(i)` Toglie l'elemento `i`.

`clear()` Svuota la lista.

`at(i)` Il controllo numero `i`.

`length` Quanti elementi.

`track tel` Nel `@for` si traccia il controllo stesso, non l'indice: rimuovendo un elemento in mezzo, Angular sposta i campi giusti.

Un `FormArray` può contenere anche `FormGroup`, per righe con più campi: `new FormArray([new FormGroup({ prodotto: ..., quantità: ... })])`. Nel template: `formArrayName`, poi `[formGroupName]="i"` e dentro `i` `formControlName`.

Stati del form reactive

Sono gli stessi dei template-driven e valgono per ogni controllo, gruppo e lista: valid / invalid, pending, disabled / enabled, pristine / dirty, untouched / touched, status, errors, value. Le classi CSS ng-valid, ng-invalid, ng-untouched, ng-dirty, ng-pending vengono messe allo stesso modo, quindi lo scss della pagina /iscrizione funziona uguale. Unica differenza: submitted esiste solo sulla direttiva (FormGroupDirective), non sul FormGroup.

L'esempio completo nel progetto

File in src/app/pagine/iscrizione-reactive:

iscrizione-reactive.ts Il FormGroup con tutti i controlli, i getter, il campo condizionale, il FormArray, il FormControl di ricerca, submit, precompila, svuota.

iscrizione-reactive.html Le direttive di collegamento, i cinque modi di accedere ai controlli, il pannello degli stati con value e getRawValue.

validatori-reactive.ts valoreVietato, passwordUguali, emailLibera come funzioni.

Prove da fare su http://localhost:4200/iscrizione-reactive:

1. Premi Iscriviti a vuoto: markAllAsTouched mostra tutti gli errori. 2. Scrivi mario@esempio.it nell'email ed esci: pending, poi emailUsata. 3. Scegli Fattura Azienda: compare la ragione sociale ed è obbligatoria; in form.value non c'era finché era disabilitata. 4. Aggiungi due telefoni, scrivi lettere in uno: errore solo su quello. 5. Scrivi angular nella ricerca: la lista si filtra dopo una breve pausa. 6. Precompila (patchValue), completa e invia. 7. Svuota: valori iniziali, un solo telefono, ragione sociale di nuovo nascosta.

Template-driven e reactive a confronto

Dove si definisce il form Template-driven: nell'HTML, con ngModel e name. Reactive: nella classe, con FormGroup e FormControl.

Fonte dei dati Template-driven: il tuo modello. Reactive: il form stesso.

Collegamento nell'HTML Template-driven: [(ngModel)] e name. Reactive: [formGroup], formControlName.

Validatori Template-driven: attributi e direttive. Reactive: funzioni in Validators o scritte da te.

Accesso a un campo Template-driven: #nome="ngModel" o viewChild. Reactive: form.controls.nome, getter, @let.

Valori nel codice Template-driven: disponibili dopo un ciclo, controlli registrati in modo asincrono. Reactive: sincroni, subito.

Campi dinamici Template-driven: @if e @for nel template. Reactive: FormArray, enable/disable, addControl.

Precompilare Template-driven: cambi il modello. Reactive: patchValue.

Modulo Template-driven: FormsModule. Reactive: ReactiveFormsModule.

Errori tipici con i form reactive

Can't bind to 'formGroup' since it isn't a known property of 'form' Manca ReactiveFormsModule negli imports.

Cannot find control with name: 'x' formControlName="x" non corrisponde a nessuna chiave del gruppo di quel livello. Controlla formGroupName intorno.

formControlName must be used with a parent formGroup directive Il campo non sta dentro un tag con [formGroup], o hai dimenticato le quadre: formGroup="form" passa la stringa 'form'.

It looks like you're using the disabled attribute with a reactive form directive Non usare [disabled] su un campo reactive: usa control.disable() o { value, disabled: true }.

Dopo reset() i campi valgono null Il controllo non è nonNullable.

reset() non toglie le righe aggiunte È normale per i FormArray: usa clear() e poi push().

Il campo disabilitato manca nei dati value lo esclude. Usa getRawValue().

Must supply a value for form control with name 'x' setValue vuole tutti i campi. Usa patchValue.

Il validatore aggiunto non ha effetto Dopo addValidators o setValidators serve updateValueAndValidity().

Il form viene ricreato e perde i dati Il FormGroup è creato in un metodo chiamato più volte, o in un getter. Crealo una volta come campo della classe.

15. Chiamate HTTP e database Firebase

Com'è fatta una chiamata HTTP

Un'app Angular non ha un database: i dati stanno su un server, e l'app li chiede e li manda con richieste HTTP. Ogni richiesta ha quattro parti, e ogni risposta tre.

Metodo Cosa vuoi fare: GET leggere, POST creare, PUT sostituire, PATCH modificare in parte, DELETE eliminare.

Endpoint L'indirizzo della risorsa: `https://server.it/api/corsi/42`. È formato da indirizzo base, percorso e, se servono, query params dopo il punto interrogativo.

Header Informazioni sulla richiesta, coppie nome: valore. Content-Type: application/json dice che il body è JSON; Authorization: Bearer token dice chi sei.

Body I dati inviati, di solito un oggetto JSON. Solo POST, PUT e PATCH lo hanno.

Risposta: status Un numero che dice com'è andata. 2xx bene, 4xx errore tuo, 5xx errore del server.

Risposta: header Informazioni sulla risposta, per esempio Content-Type.

Risposta: body I dati restituiti: l'elenco, l'oggetto creato, un messaggio d'errore.

```
POST https://server.it/api/corsi      <- metodo + endpoint
Content-Type: application/json        <- header
Authorization: Bearer eyJhbGciOi...

{ "titolo": "Angular base", "ore": 16 } <- body

HTTP/1.1 201 Created                  <- status della risposta
Content-Type: application/json

{ "id": 42, "titolo": "Angular base", "ore": 16 } <- body della risposta
```

Gli status più comuni:

200 OK Tutto bene, il body contiene il risultato.

201 Created Risorsa creata (dopo un POST).

204 No Content Tutto bene, ma senza body (spesso dopo DELETE).

400 Bad Request Dati inviati sbagliati o incompleti.

401 Unauthorized Non sei autenticato: token mancante o scaduto.

403 Forbidden Sei autenticato ma non hai il permesso. Firebase usa 401 per Permission denied.

404 Not Found L'endpoint non esiste.

500 Internal Server Error Errore del server.

0 Nessuna risposta: server spento, rete assente o blocco CORS. Non è uno status HTTP vero, lo mette Angular.

I metodi a confronto

GET /corsi Legge l'elenco. Nessun body. Si può ripetere senza effetti: è sicuro.

GET /corsi/42 Legge un elemento.

POST /corsi Crea un elemento nuovo. Body: i dati. Il server assegna l'id. Ripeterlo crea un doppiante.

PUT /corsi/42 Sostituisce tutto l'elemento 42. Body: l'oggetto completo. I campi non inviati spariscono.

PATCH /corsi/42 Cambia solo i campi inviati. Body: { attivo: false }. Gli altri campi restano.

DELETE /corsi/42 Elimina l'elemento. Nessun body.

Il percorso indica la risorsa, il metodo indica l'azione. Per questo si scrive POST /corsi e non GET /creaCorso: è lo stile REST, usato dalla maggior parte delle API.

Importare il modulo HTTP

Nelle app standalone non si importa più HttpClientModule: si registra HttpClient con provideHttpClient in app.config.ts. Una volta sola, per tutta l'app.

```
// src/app/app.config.ts
import { ApplicationConfig, provideBrowserGlobalErrorListeners } from '@angular/core';
import { provideRouter } from '@angular/router';
import { provideHttpClient, withFetch, withInterceptors } from '@angular/common/http';
import { provideClientHydration, withEventReplay } from '@angular/platform-browser';
import { routes } from './app.routes';
import { registroHttpInterceptor } from './servizi/registro-http';

export const appConfig: ApplicationConfig = {
  providers: [
    provideBrowserGlobalErrorListeners(),
    provideRouter(routes),
    provideHttpClient(withFetch(), withInterceptors([registroHttpInterceptor])),
    provideClientHydration(withEventReplay()),
  ],
};
```

provideHttpClient() Rende disponibile HttpClient a servizi e componenti. Senza, inject(HttpClient) dà NullInjectorError: No provider for HttpClient.

withFetch() Usa l'API fetch del browser. Consigliato con SSR, come in corso-angular.

withInterceptors(...) Funzioni che vedono passare ogni richiesta: token, log, errori.

HttpClientModule Il modo vecchio, con i NgModule: imports: [HttpClientModule] nell'AppModule. È deprecato; lo trovi nei progetti datati.

Nei componenti non serve importare niente negli imports: HttpClient non si usa nel template. Si prende con inject(HttpClient), di solito dentro un servizio.

HttpClient: i metodi e le opzioni

```
private http = inject(HttpClient);

this.http.get<T>(url, opzioni)
this.http.post<T>(url, body, opzioni)
this.http.put<T>(url, body, opzioni)
this.http.patch<T>(url, body, opzioni)
this.http.delete<T>(url, opzioni)
```

<T> Il tipo della risposta: get<Corso[]>. Serve a TypeScript, Angular non lo verifica.

url L'endpoint completo, di solito costruito da environment più percorso.

body Un oggetto: HttpClient lo converte in JSON e mette da solo Content-Type: application/json.

opzioni.headers Header in più: new HttpHeaders({...}) oppure un oggetto semplice { 'X-Lingua': 'it' }.

opzioni.params Query params: new HttpParams().set('pagina', 2) oppure { pagina: 2, cerca: 'ang' }.

opzioni.observe: 'response' Invece del solo body ricevi la risposta intera: status, headers, body.

opzioni.responseType 'json' (predefinito), 'text' per testo semplice, 'blob' per file da scaricare.

Ogni metodo restituisce un Observable: la richiesta parte solo al subscribe (o con async, toSignal) e ogni subscribe ne manda una nuova.

Header

```
import { HttpHeaders } from '@angular/common/http';

// Per una sola richiesta
const headers = new HttpHeaders({
  'Content-Type': 'application/json',
  'Accept-Language': 'it',
});
this.http.post<Corso>(url, corso, { headers });

// HttpHeaders è immutabile: set e append restituiscono un oggetto nuovo
let h = new HttpHeaders();
h = h.set('Authorization', `Bearer ${token}`);

// Leggere gli header della risposta
this.http.get<Corso[]>(url, { observe: 'response' }).subscribe((risposta) => {
  console.log(risposta.status); // 200
  console.log(risposta.headers.get('Content-Type')); // application/json
  console.log(risposta.body); // i dati
});
```

Per un header che serve a tutte le richieste, come il token, non ripeterlo in ogni servizio: usa un interceptor, come `authInterceptor` nel capitolo 16. Attenzione ai server di altri: un header personalizzato fa partire una richiesta preliminare CORS (OPTIONS), e se il server non lo permette la chiamata fallisce con status 0.

Body e query params

```
// Body: un oggetto, convertito in JSON
this.http.post<{ name: string }>(`${base}/corsi.json`, { titolo: 'RxJS', ore: 8, attivo: true });

// PATCH: body con i soli campi da cambiare
this.http.patch(`${base}/corsi/${id}.json`, { attivo: false });

// Query params: ?orderBy="$key"&limitToFirst=10
const params = new HttpParams().set('orderBy', "$key").set('limitToFirst', 10);
this.http.get(`${base}/corsi.json`, { params });

// Stessa cosa con un oggetto
this.http.get(`${base}/corsi.json`, { params: { orderBy: "$key", limitToFirst: 10 } });

// Caricare un file: FormData, Content-Type lo mette il browser
const dati = new FormData();
dati.append('file', fileScelto);
this.http.post('/api/allegati', dati);
```

Non costruire i query params a mano concatenando stringhe: `HttpParams` codifica i caratteri speciali (spazi, &, virgolette) al posto tuo.

Un servizio REST completo (CRUD)

CRUD vuol dire le quattro operazioni base: creare, leggere, aggiornare, cancellare. Con un'API REST corrispondono a POST, GET, PUT, DELETE.

```
import { HttpClient, HttpResponse, HttpParams } from '@angular/common/http';
import { Injectable, inject } from '@angular/core';
import { Observable, catchError, retry, throwError } from 'rxjs';
import { environment } from '../environments/environment';
import { Eroee } from '../eroee';

@Injectable({ providedIn: 'root' })
export class EroeeApi {
  private http = inject(HttpClient);
  private url = `${environment.apiUrl}/eroee`;

  lista(cerca?: string): Observable<Eroee[]> {
    let params = new HttpParams();
    if (cerca) params = params.set('nome_like', cerca);
    return this.http.get<Eroee[]>(this.url, { params }).pipe(
      retry(2),
      catchError(this.gestisciErrore),
    );
  }
}
```

```

    });
  }

  uno(id: number): Observable<Eroe> {
    return this.http.get<Eroe>(`${this.url}/${id}`).pipe(catchError(this.gestisciErrore));
  }

  crea(eroe: Omit<Eroe, 'id'>): Observable<Eroe> {
    return this.http.post<Eroe>(this.url, eroe).pipe(catchError(this.gestisciErrore));
  }

  aggiorna(eroe: Eroe): Observable<Eroe> {
    return this.http.put<Eroe>(`${this.url}/${eroe.id}`, eroe).pipe(catchError(this.gestisciErrore));
  }

  elimina(id: number): Observable<void> {
    return this.http.delete<void>(`${this.url}/${id}`).pipe(catchError(this.gestisciErrore));
  }

  private gestisciErrore(err: HttpErrorResponse) {
    const messaggio =
      err.status === 0 ? 'Server non raggiungibile'
      : err.status === 404 ? 'Eroe non trovato'
      : `Errore ${err.status}`;
    return throwError(() => new Error(messaggio));
  }
}

```

get<Eroe[]> Il tipo tra le angulari dice a TypeScript cosa arriverà. Angular non lo controlla davvero: se il server manda altro, lo scopri a runtime.

HttpParams Costruisce i query params della richiesta: ?nome_like=man. È immutabile: set restituisce un oggetto nuovo, per questo params = params.set(...).

Omit<Eroe, 'id'> Un Eroe senza id: l'id lo assegna il server.

retry(2) Se fallisce, riprova fino a due volte. Utile per GET, sconsigliato per POST: rischi di creare due volte.

catchError Intercetta l'errore e lo trasforma in un messaggio leggibile. throwError lo passa avanti al componente.

status 0 Nessuna risposta: server spento, rete assente o blocco CORS.

Una chiamata HttpClient non parte finché qualcuno non fa subscribe (o usa async o toSignal). Ogni subscribe fa una richiesta nuova. La risposta arriva una volta e poi l'Observable si chiude da solo: per HTTP dimenticare unsubscribe non lascia ascolti infiniti, ma se l'utente esce dalla pagina prima della risposta il callback scrive comunque su un componente distrutto. takeUntilDestroyed risolve.

Usare il servizio nel componente: quattro modi

Modo 1, subscribe con takeUntilDestroyed e stati di caricamento ed errore. È il più esplicito, utile per le azioni: salva, elimina.

```

export class ListaEroi {
  private api = inject(EroiApi);
  private destroyRef = inject(DestroyRef);

  eroi = signal<Eroe[]>([]);
  caricamento = signal(false);
  errore = signal('');

  ngOnInit() {
    this.carica();
  }

  carica() {
    this.caricamento.set(true);
    this.errore.set('');
    this.api.lista().pipe(takeUntilDestroyed(this.destroyRef)).subscribe({
      next: (eroi) => this.eroi.set(eroi),
      error: (e: Error) => { this.errore.set(e.message); this.caricamento.set(false); },
      complete: () => this.caricamento.set(false),
    });
  }
}

```

```

    });
  }

  elimina(id: number) {
    this.api.elimina(id).subscribe({
      next: () => this.eroi.update((l) => l.filter((e) => e.id !== id)),
      error: (e: Error) => this.errorre.set(e.message),
    });
  }
}

@if (caricamento()) {
  <p>Caricamento...</p>
} @else if (errorre()) {
  <p class="errorre">{{ errorre() }} <button type="button" (click)="carica()">Riprova</button></p>
} @else {
  @for (e of eroi(); track e.id) {
    <p>{{ e.nome }} <button type="button" (click)="elimina(e.id)">Elimina</button></p>
  }
}

```

`takeUntilDestroyed()` senza argomenti funziona solo in un contesto di iniezione, per esempio in un campo. In `ngOnInit` o in un metodo passagli `destroyRef`, come sopra.

Modo 2, pipe `async`: `eroi$ = this.api.lista()`; nel template `@for (e of eroi$ | async; track e.id)`. Serve `AsyncPipe` negli imports. Iscrizione e disiscrizione automatiche.

Modo 3, `toSignal`: `eroi = toSignal(this.api.lista(), { initialValue: [] })`. Diventa un signal normale, quindi puoi usarlo in `computed`.

Modo 4, `httpResource`: nelle versioni recenti di Angular c'è una funzione che fa la richiesta e dà già i signal di valore, caricamento ed errore, e rifà la chiamata quando cambia un signal letto nell'indirizzo.

```

import { httpResource } from '@angular/common/http';

cerca = signal('');
eroiRes = httpResource<Eroe[]>(() => `${environment.apiUrl}/eroi?nome_like=${this.cerca()}`);

@if (eroiRes.isLoading()) { <p>Caricamento...</p> }
@if (eroiRes.error()) { <p>Errore</p> }
@for (e of eroiRes.value() ?? []; track e.id) { <p>{{ e.nome }}</p> }

```

`httpResource` serve solo per leggere (GET). Per creare, modificare ed eliminare si usano i metodi del servizio con `subscribe`. Controlla sulla documentazione della tua versione se è ancora segnata come sperimentale.

Interceptor: codice che vede ogni richiesta

Un interceptor è una funzione tra `HttpClient` e il server. Riceve ogni richiesta, può modificarla, poi la passa avanti con `next` e può guardare la risposta. Si usa per aggiungere il token, misurare i tempi, gestire gli errori in un punto solo. `ng g interceptor nome`.

```

import { HttpInterceptorFn } from '@angular/common/http';
import { finalize } from 'rxjs';

export const logInterceptor: HttpInterceptorFn = (req, next) => {
  const inizio = Date.now();
  return next(req).pipe(
    finalize(() => console.log(`${req.method} ${req.url} ${Date.now() - inizio} ms`)),
  );
};

```

Esempio: un contatore globale di richieste in corso, per mostrare uno spinner unico.

```

@Injectable({ providedIn: 'root' })
export class CaricamentoServizio {
  private inCorso = signal(0);
  readonly attivo = computed(() => this.inCorso() > 0);
  inizia() { this.inCorso.update((n) => n + 1); }
  finisci() { this.inCorso.update((n) => n - 1); }
}

export const caricamentoInterceptor: HttpInterceptorFn = (req, next) => {
  const stato = inject(CaricamentoServizio);
  stato.inizia();
  return next(req).pipe(finally(() => stato.finisci()));
};

```

req.clone({...}) Una richiesta è immutabile. Per cambiarla se ne crea una copia modificata.

next(req) Passa la richiesta all'interceptor successivo o al server. Senza next la richiesta non parte.

Ordine Gli interceptor girano nell'ordine dell'array di withInterceptors per la richiesta, al contrario per la risposta.

Provare con un server finto: json-server

Per esercitarsi serve un'API. json-server ne crea una da un file JSON, con GET, POST, PUT e DELETE veri.

```
npm i -D json-server@0.17.4
```

db.json nella cartella del progetto:

```

{
  "eroi": [
    { "id": 1, "nome": "Superman", "superpotere": "volo", "eroe": true },
    { "id": 2, "nome": "Joker", "superpotere": "caos", "eroe": false }
  ]
}

npx json-server --watch db.json --port 3000

```

Ora `http://localhost:3000/eroi` risponde con l'elenco, e le modifiche fatte con POST, PUT e DELETE vengono scritte nel file. Tieni aperti due terminali: uno con ng serve, uno con json-server. Usiamo la versione 0.17 perché supporta i filtri come `?nome_like=man`, usati nel servizio qui sopra; la versione 1 ha una sintassi diversa.

Creare il database su Firebase

Firebase è un servizio di Google che offre, tra le altre cose, un database raggiungibile con semplici chiamate HTTP: il Realtime Database. Non serve scrivere un backend: è perfetto per esercitarsi con HttpClient. Il piano gratuito basta per studiare. Il progetto va creato con il tuo account Google:

1. Apri `https://console.firebase.google.com` e accedi con il tuo account Google.
2. Crea un progetto nuovo (Aggiungi progetto o Crea un progetto), dagli un nome, per esempio `corso-angular`. Google Analytics non serve: puoi disattivarlo.
3. Nel menu a sinistra apri Build (Crea), poi Realtime Database, e premi Crea database.
4. Scegli la posizione: per l'Italia `eu-west1` (Belgio).
5. Scegli Avvia in modalità test: chiunque conosca l'URL può leggere e scrivere per 30 giorni. Va bene per studiare, mai per dati veri.
6. In alto nella scheda Dati trovi l'URL del database, per esempio `https://corso-angular-12345-default-rtdb.eu-west1.firebaseio.com`. Copialo.
7. Incollalo in `src/environments/environment.development.ts` nel campo `firebaseUrl`, senza barra finale, e riavvia ng serve.

Per usare il database con HttpClient non serve registrare un'app web né installare librerie: basta l'URL.

Le regole di sicurezza, nella scheda Regole, decidono chi può leggere e scrivere:

```
// Modalità test: aperto a tutti fino a una data (in millisecondi)
{
  "rules": {
    ".read": "now < 1762000000000",
    ".write": "now < 1762000000000"
  }
}

// Solo utenti autenticati con Firebase Authentication
{
  "rules": {
    ".read": "auth != null",
    ".write": "auth != null"
  }
}

// Indice per poter ordinare i corsi per titolo con orderBy="titolo"
{
  "rules": {
    ".read": "now < 1762000000000",
    ".write": "now < 1762000000000",
    "corsi": { ".indexOn": ["titolo"] }
  }
}
```

Quando la modalità test scade, ogni chiamata risponde 401 Permission denied: si allunga la data o si passa alle regole con auth.

Realtime Database o Firestore: Firebase ha anche Firestore, più potente, ma la sua API REST è più complicata e di solito si usa con la libreria `@angular/fire`. Per imparare le chiamate HTTP il Realtime Database è la scelta più semplice.

Come risponde Firebase: l'API REST

Il Realtime Database è un grande oggetto JSON. Ogni percorso è un nodo di quell'oggetto, e per leggerlo o scriverlo si aggiunge `.json` in fondo all'indirizzo.

GET {url}/corsi.json Tutto il nodo corsi. Risponde `{ "-Nx1": {...}, "-Nx2": {...} }`, oppure null se non c'è niente.

GET {url}/corsi/-Nx1.json Un solo corso.

POST {url}/corsi.json Aggiunge un figlio con un id generato. Risponde `{ "name": "-Nx3" }`: name è l'id nuovo.

PUT {url}/corsi/-Nx1.json Scrive esattamente quell'oggetto in quel punto. Risponde con i dati scritti.

PATCH {url}/corsi/-Nx1.json Aggiorna solo i campi inviati. Risponde con i campi inviati.

DELETE {url}/corsi/-Nx1.json Elimina il nodo. Risponde null.

?orderBy="\$key" Ordina per chiave. Le virgolette fanno parte del valore. Con `orderBy="titolo"` serve `.indexOn` nelle regole.

?limitToFirst=10 / limitToLast=10 Solo i primi o gli ultimi N.

?equalTo="Rossi" Con `orderBy="docente"`: solo i corsi di quel docente.

?auth=ID_TOKEN Il token di un utente di Firebase Authentication, se le regole chiedono `auth != null`.

Firebase non restituisce un array ma un oggetto con gli id come chiavi. Il servizio lo trasforma in un array di oggetti con dentro l'id, che è più comodo nel template:

```
lista(): Observable<Corso[]> {
  return this.http
    .get<Record<string, DatiCorso> | null>(`${this.base}.json`, { params })
    .pipe(map((dati) => Object.entries(dati ?? {}).map(([id, corso]) => ({ id, ...corso })))));
}
```

`Object.entries` trasforma `{ "-Nx1": { titolo: 'A' } }` in `[["-Nx1", { titolo: 'A' }]]`. `dati ?? {}` gestisce il null del nodo vuoto.

Il servizio completo per Firebase

`src/app/servizi/corsi-firebase.ts`

```
import { HttpClient, HttpResponse, HttpHeaders, HttpParams } from '@angular/common/http';
import { Injectable, inject } from '@angular/core';
import { Observable, catchError, map, throwError } from 'rxjs';
import { environment } from '../../environments/environment';

export interface Corso { id: string; titolo: string; docente: string; ore: number; attivo: boolean; }
export type DatiCorso = Omit<Corso, 'id'>; // sul database l'id è la chiave, non un campo

@Injectable({ providedIn: 'root' })
export class CorsiFirebase {
  private http = inject(HttpClient);
  private base = `${environment.firebaseUrl}/corsi`;
  private headers = new HttpHeaders({ 'Content-Type': 'application/json' });

  lista(): Observable<Corso[]> {
    const params = new HttpParams().set('orderBy', '"$key"');
    return this.http.get<Record<string, DatiCorso> | null>(`${this.base}.json`, { params }).pipe(
      map((dati) => Object.entries(dati ?? {}).map(([id, corso]) => ({ id, ...corso }))),
      catchError(this.gestisciErrore),
    );
  }

  crea(corso: DatiCorso): Observable<Corso> {
    return this.http.post<{ name: string }>(`${this.base}.json`, corso, { headers: this.headers }).pipe(
      map((risposta) => ({ id: risposta.name, ...corso })),
      catchError(this.gestisciErrore),
    );
  }

  sostituisci(corso: Corso): Observable<Corso> {
    const { id, ...dati } = corso; // l'id va nell'URL, non nel body
    return this.http.put<DatiCorso>(`${this.base}/${id}.json`, dati, { headers: this.headers }).pipe(
      map((salvato) => ({ id, ...salvato })),
      catchError(this.gestisciErrore),
    );
  }

  aggiorna(id: string, campi: Partial<DatiCorso>): Observable<Partial<DatiCorso>> {
    return this.http.patch<Partial<DatiCorso>>(`${this.base}/${id}.json`, campi, { headers: this.headers })
      .pipe(catchError(this.gestisciErrore));
  }

  elimina(id: string): Observable<null> {
    return this.http.delete<null>(`${this.base}/${id}.json`).pipe(catchError(this.gestisciErrore));
  }

  private gestisciErrore(err: HttpResponse) {
    const messaggio =
      err.status === 0 ? 'Server non raggiungibile: controlla firebaseUrl'
      : err.status === 401 || err.status === 403 ? 'Permesso negato: controlla le regole del database'
      : `Errore ${err.status}: ${err.error?.error ?? err.message}`;
    return throwError(() => new Error(messaggio));
  }
}
```

Partial<DatiCorso> Tutti i campi facoltativi: il tipo giusto per il body di un PATCH.

const { id, ...dati } Separa l'id dal resto: l'id finisce nell'endpoint, il resto nel body.

err.error?.error Firebase spiega l'errore nel body: `{ "error": "Permission denied" }`.

La pagina dei corsi: il componente

```
export class CorsiFirebasePagina implements OnInit {
  private api = inject(CorsiFirebase);
```

```

private destroyRef = inject(DestroyRef);

corsi = signal<Corso[]>([]);
caricamento = signal(false);
errore = signal('');
inModifica = signal<string | null>(null);

form = new FormGroup({
  titolo: new FormControl('', { nonNullable: true, validators: [Validators.required] }),
  docente: new FormControl('', { nonNullable: true, validators: [Validators.required] }),
  ore: new FormControl(8, { nonNullable: true, validators: [Validators.min(1)] }),
  attivo: new FormControl(true, { nonNullable: true }),
});

ngOnInit() { this.carica(); }

carica() { // GET
  this.caricamento.set(true);
  this.api.lista().pipe(takeUntilDestroyed(this.destroyRef)).subscribe({
    next: (corsi) => { this.corsi.set(corsi); this.caricamento.set(false); },
    error: (e: Error) => { this.errore.set(e.message); this.caricamento.set(false); },
  });
}

salva() { // POST oppure PUT
  if (this.form.invalid) { this.form.markAllAsTouched(); return; }
  const dati = this.form.getRawValue();
  const id = this.inModifica();
  const richiesta = id ? this.api.sostituisci({ id, ...dati }) : this.api.crea(dati);
  richiesta.subscribe({
    next: (salvato) => {
      this.corsi.update((l) => (id ? l.map((c) => (c.id === id ? salvato : c)) : [...l, salvato]));
      this annulla();
    },
    error: (e: Error) => this.errore.set(e.message),
  });
}

cambiaAttivo(corso: Corso) { // PATCH
  const attivo = !corso.attivo;
  this.api.aggiorna(corso.id, { attivo }).subscribe(() =>
    this.corsi.update((l) => l.map((c) => (c.id === corso.id ? { ...c, attivo } : c))));
}

elimina(corso: Corso) { // DELETE
  this.api.elimina(corso.id).subscribe(() =>
    this.corsi.update((l) => l.filter((c) => c.id !== corso.id)));
}
}

```

Dopo ogni scrittura riuscita il componente aggiorna la lista in memoria invece di rifare il GET: la pagina risponde subito e si risparmia una richiesta. Il bottone Ricarica rifà il GET e mostra i dati come sono davvero sul server.

Con SSR la pagina che chiama il database va disegnata solo nel browser, altrimenti ng build proverebbe a chiamare Firebase durante il prerender:

```

// app.routes.server.ts
{ path: 'corsi-firebase', renderMode: RenderMode.Client },

```

Vedere le chiamate: un interceptor registro

Per studiare è utile vedere ogni richiesta: metodo, endpoint, header, body, status, risposta. Lo fa un interceptor che scrive in un servizio con un signal, mostrato in fondo alla pagina. Le stesse informazioni le trovi negli strumenti del browser (F12), scheda Rete.

```
export const registroHttpInterceptor: HttpInterceptorFn = (req, next) => {
  const registro = inject(RegistroHttp);
  const inizio = Date.now();
  const base = {
    metodo: req.method,
    url: req.urlWithParams,
    headers: req.headers.keys().map((k) => `${k}: ${req.headers.get(k)}`),
    body: req.body,
  };
  return next(req).pipe(
    tap({
      next: (evento) => {
        if (evento instanceof HttpResponse) {
          registro.aggiungi({ ...base, stato: evento.status, risposta: evento.body, ms: Date.now() -
            inizio });
        }
      },
      error: (err: HttpErrorResponse) =>
        registro.aggiungi({ ...base, stato: err.status, risposta: err.error, ms: Date.now() - inizio }),
    }),
  );
};
```

`next(req)` non emette solo la risposta ma anche altri eventi (per esempio il progresso di un upload): per questo si controlla `evento instanceof HttpResponse`.

Accedere con Firebase Authentication

Quando le regole diventano `auth != null`, ogni chiamata deve portare il token di un utente. Il collegamento con il capitolo 16 è diretto:

1. Nella console Firebase apri Build, Authentication, Inizia, e attiva il metodo Email/Password. Aggiungi un utente di prova dalla scheda Utenti.
2. In Impostazioni progetto, Generali, copia la Chiave API web e mettila in environment come `firebaseApiKey`.
3. Il login è una POST all'API di Google, che risponde con `idToken`.
4. Un interceptor aggiunge `?auth=idToken` alle richieste verso il database.

```
// login: dentro AuthService
login(email: string, password: string) {
  const url =
    `https://identitytoolkit.googleapis.com/v1/accounts:signInWithPassword?key=${environment.firebaseApiKey}`;
  return this.http.post<{ idToken: string; email: string; expiresIn: string }>(
    url, { email, password, returnSecureToken: true },
  );
}

// interceptor: il token va nei query params, non in un header
export const firebaseAuthInterceptor: HttpInterceptorFn = (req, next) => {
  const token = inject(AuthServizio).token();
  if (!token || !req.url.startsWith(environment.firebaseUrl)) return next(req);
  return next(req.clone({ setParams: { auth: token } }));
};
```

L'`idToken` dura un'ora (`expiresIn: '3600'`). La chiave API web non è un segreto: identifica il progetto, e la protezione vera la danno le regole del database.

Provare senza Firebase: il server finto

Nel progetto c'è `firebase-finto.mjs`: un piccolo server Node che risponde come il Realtime Database (.json in fondo, POST che restituisce name, PUT, PATCH, DELETE) e salva i dati in `firebase-finto.json`. Serve per esercitarsi senza account e senza rete.

```
npm run firebase-finto
```

Ascolta su `http://localhost:9123`, che è il valore di `firebaseUrl` in `environment.development.ts`. La porta 9000, la prima scelta, sul tuo PC era già occupata da un altro programma: se una porta è occupata, il server si ferma con

l'errore EADDRINUSE. Per passare a Firebase vero basta cambiare firebaseUrl: il resto del codice non cambia, perché l'API è la stessa. Tieni due terminali aperti: uno con ng serve, uno con il server finto.

L'esempio completo nel progetto

src/app/servizi/corsi-firebase.ts Il servizio con GET, POST, PUT, PATCH, DELETE, header, query params e gestione errori.

src/app/servizi/registro-http.ts Il servizio RegistroHttp e l'interceptor che registra le chiamate.

src/app/pagine/corsi-firebase La pagina: form reactive, tabella, registro delle chiamate.

src/environments firebaseUrl per sviluppo e produzione.

firebase-finto.mjs Il server finto, avviato con npm run firebase-finto.

app.config.ts provideHttpClient(withFetch()), withInterceptors([registroHttpInterceptor])).

Prove da fare su <http://localhost:4200/corsi-firebase>:

1. Aggiungi due corsi: nel registro compaiono due POST con header Content-Type, body e risposta { name }. 2. Premi sospendo o attivo su un corso: un PATCH con body { attivo: false }. 3. Premi Modifica, cambia le ore e salva: un PUT con l'oggetto completo nel body. 4. Elimina un corso: un DELETE senza body che risponde null. 5. Premi Ricarica: un GET con ?orderBy="\$key" nell'endpoint. 6. Spegni il server finto e premi Ricarica: status 0 e il messaggio Server non raggiungibile. 7. Crea il database su Firebase, incolla l'URL, riavvia ng serve e ripeti: i corsi compaiono anche nella console di Firebase, scheda Dati.

Errori tipici con HTTP e Firebase

NullInjectorError: No provider for HttpClient Manca provideHttpClient() in app.config.ts.

La richiesta non parte Nessun subscribe. Nella scheda Rete del browser non compare niente.

Status 0 Server spento, URL sbagliato, rete assente o blocco CORS. Controlla firebaseUrl e la console del browser.

401 Permission denied Regole di Firebase: modalità test scaduta, oppure regole auth != null senza ?auth=token.

400 Index not defined orderBy su un campo senza ".indexOn" nelle regole.

400 orderBy must be a valid JSON encoded path Mancano le virgolette: orderBy="\$key", non orderBy=\$key.

Firebase risponde HTML o 404 Manca .json in fondo all'endpoint.

La lista è un oggetto e @for non funziona Firebase restituisce un oggetto: trasformalo con Object.entries.

L'id finisce duplicato nel database Hai mandato anche id nel body del PUT: separalo con const { id, ...dati }.

Warning NG02801 sul fetch Con SSR usa provideHttpClient(withFetch()).

ng build si blocca o fallisce su una pagina con HTTP Il prerender la esegue sul server: usa RenderMode.Client per quella rotta.

Errore CORS in console Il server non permette richieste dal dominio di sviluppo. Va configurato sul server, oppure si usa un proxy di ng serve (proxy.conf.json).

16. Autenticazione

Come funziona il login in un'app Angular

Angular gira nel browser, quindi non può custodire segreti né decidere da solo chi è autorizzato. Il vero controllo lo fa il server. Il flusso più diffuso usa un token JWT:

1. L'utente scrive email e password nel form di login.
2. Il servizio le invia al server con una POST.
3. Il server le verifica e risponde con un token e i dati dell'utente.
4. L'app salva il token e l'utente.
5. Un interceptor aggiunge il token in ogni richiesta successiva, nell'intestazione Authorization: Bearer token.
6. Il server legge il token a ogni richiesta e decide se rispondere.
7. Le guardie nascondono le pagine a chi non è loggato o non ha il ruolo giusto.
8. Se il server risponde 401 (token mancante o scaduto), l'app fa logout e manda al login.

Un JWT è una stringa in tre parti separate da punti: intestazione, contenuto (payload) e firma. Il payload è in base64, quindi chiunque può leggerlo: contiene per esempio l'id dell'utente e la scadenza exp. Nessuno può modificarlo senza invalidare la firma, che solo il server sa verificare.

File che costruiremo:

`auth/modelli.ts` Interfacce Utente e RispostaLogin.
`auth/auth-servizio.ts` Stato del login, login, logout, ruoli.
`auth/auth-interceptor.ts` Aggiunge il token, gestisce il 401.
`auth/auth-guard.ts` `authGuard`, `ruoloGuard`, `ospiteGuard`.
`auth/backend-finto-interceptor.ts` Server finto per provare tutto senza backend.
`pagine/login/login.ts` e `.html` Il form.
`app.routes.ts` e `app.config.ts` Collegamento di tutto.

Modelli e lettura del token

`auth/modelli.ts`

```
export interface Utente {
  id: number;
  nome: string;
  email: string;
  ruoli: string[];
}

export interface RispostaLogin {
  token: string;
  utente: Utente;
}

export function tokenScaduto(token: string): boolean {
  try {
    const base64 = token.split('.')[1].replace(/-/g, '+').replace(/_/g, '/');
    const payload = JSON.parse(atob(base64));
    return typeof payload.exp === 'number' && payload.exp * 1000 < Date.now();
  } catch {
    return true;
  }
}
```

`exp` è in secondi dal 1970, `Date.now()` in millisecondi: per questo `* 1000`. Se il token non si legge lo consideriamo scaduto. Questa verifica serve solo all'interfaccia, per non mostrare l'utente come loggato con un token vecchio: la firma la controlla il server.

Il servizio di autenticazione

auth/auth-servizio.ts

```
import { HttpClient } from '@angular/common/http';
import { Injectable, PLATFORM_ID, inject, signal } from '@angular/core';
import { isPlatformBrowser } from '@angular/common';
import { Router } from '@angular/router';
import { Observable, map, tap } from 'rxjs';
import { environment } from '../../environments/environment';
import { RispostaLogin, Utente, tokenScaduto } from './modelli';

const CHIAVE_TOKEN = 'auth_token';
const CHIAVE_UTENTE = 'auth_utente';

@Injectable({ providedIn: 'root' })
export class AuthService {
  private http = inject(HttpClient);
  private router = inject(Router);
  private browser = isPlatformBrowser(inject(PLATFORM_ID));

  private readonly _token = signal<string | null>(this.leggi(CHIAVE_TOKEN));
  private readonly _utente = signal<Utente | null>(
    JSON.parse(this.leggi(CHIAVE_UTENTE) ?? 'null'),
  );

  readonly token = this._token.asReadonly();
  readonly utente = this._utente.asReadonly();

  isLoggato(): boolean {
    const t = this._token();
    return t !== null && !tokenScaduto(t);
  }

  haRuolo(ruolo: string): boolean {
    return this._utente()?.ruoli.includes(ruolo) ?? false;
  }

  login(email: string, password: string): Observable<Utente> {
    return this.http
      .post<RispostaLogin>(`${environment.apiUrl}/auth/login`, { email, password })
      .pipe(
        tap((r) => this.salvaSessione(r)),
        map((r) => r.utente),
      );
  }

  logout(): void {
    this._token.set(null);
    this._utente.set(null);
    if (this.browser) {
      localStorage.removeItem(CHIAVE_TOKEN);
      localStorage.removeItem(CHIAVE_UTENTE);
    }
    this.router.navigate(['/login']);
  }

  private salvaSessione(r: RispostaLogin): void {
    this._token.set(r.token);
    this._utente.set(r.utente);
    if (this.browser) {
      localStorage.setItem(CHIAVE_TOKEN, r.token);
      localStorage.setItem(CHIAVE_UTENTE, JSON.stringify(r.utente));
    }
  }

  private leggi(chiave: string): string | null {
    return this.browser ? localStorage.getItem(chiave) : null;
  }
}
```

signal privati Lo stato del login è in un posto solo. Header, guardie e interceptor leggono lo stesso servizio.

localStorage Il token sopravvive al ricaricamento della pagina. All'avvio i signal ripartono da lì.

isPlatformBrowser I tuoi progetti usano SSR: sul server localStorage non esiste e darebbe errore. Leggiamo e scriviamo solo nel browser.

isLoggato() è un metodo Non un computed. Un computed ricorda il risultato finché il token non cambia, e non si accorgerebbe che il tempo è passato e il token è scaduto. Il metodo legge comunque il signal, quindi nel template resta reattivo.

tap Esegue salvaSessione quando la risposta arriva, senza cambiarla. map poi restituisce al componente solo l'utente.

Ordine dei campi browser è dichiarato prima di _token perché leggi() lo usa durante l'inizializzazione. I campi si inizializzano dall'alto in basso.

L'interceptor che aggiunge il token

auth/auth-interceptor.ts

```
import { HttpResponse, HttpInterceptorFn } from '@angular/common/http';
import { inject } from '@angular/core';
import { catchError, throwError } from 'rxjs';
import { environment } from '../../environments/environment';
import { AuthService } from '../auth-servizio';

export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const auth = inject(AuthServizio);
  const token = auth.token();
  const versoLaNostraApi = req.url.startsWith(environment.apiUrl);

  const richiesta =
    token && versoLaNostraApi
      ? req.clone({ setHeaders: { Authorization: `Bearer ${token}` } })
      : req;

  return next(richiesta).pipe(
    catchError((err: HttpResponse) => {
      if (err.status === 401 && !req.url.endsWith('/auth/login')) {
        auth.logout();
      }
      return throwError(() => err);
    })
  );
};
```

Il token va solo verso la nostra API: mandarlo a un sito esterno significherebbe consegnargli le chiavi. Sul 401 del login non facciamo logout, perché lì 401 vuol dire solo password sbagliata e l'errore lo gestisce il form.

throwError fa arrivare comunque l'errore a chi ha fatto la richiesta.

Le guardie

auth/auth-guard.ts

```
import { inject } from '@angular/core';
import { CanActivateFn, Router } from '@angular/router';
import { AuthService } from '../auth-servizio';

export const authGuard: CanActivateFn = (route, state) => {
  const auth = inject(AuthServizio);
  if (auth.isLoggato()) return true;
  return inject(Router).createUrlTree(['/login'], {
    queryParams: { returnUrl: state.url },
  });
};

export const ruoloGuard: CanActivateFn = (route) => {
  const auth = inject(AuthServizio);
  const ammessi = (route.data['ruoli'] as string[] | undefined) ?? [];
  if (ammessi.length === 0 || ammessi.some((r) => auth.haRuolo(r))) return true;
  return inject(Router).createUrlTree(['/non-autorizzato']);
};

export const ospiteGuard: CanActivateFn = () => {
  const auth = inject(AuthServizio);
  return auth.isLoggato() ? inject(Router).createUrlTree(['/']) : true;
};
```

state.url L'indirizzo che l'utente voleva aprire. Lo passiamo al login come returnUrl, così dopo l'accesso torna lì.

createUrlTree Invece di false restituisce un indirizzo: il router annulla la navigazione e va lì. È meglio di chiamare router.navigate dentro la guardia.

route.data['ruoli'] I ruoli ammessi si scrivono nella rotta, la guardia resta una sola per tutte le pagine.

ospiteGuard Chi è già loggato non deve rivedere il form di login.

La pagina di login

pagine/login/login.ts

```
import { Component, inject, input, signal } from '@angular/core';
import { FormBuilder, ReactiveFormsModule, Validators } from '@angular/forms';
import { Router } from '@angular/router';
import { AuthService } from '../../auth/auth-servizio';

@Component({
  selector: 'app-login',
  imports: [ReactiveFormsModule],
  templateUrl: './login.html',
})
export class Login {
  private auth = inject(AuthServizio);
  private router = inject(Router);

  returnUrl = input<string>(); // da ?returnUrl=
  errore = signal('');
  invio = signal(false);

  form = inject(FormBuilder).nonNullable.group({
    email: ['', [Validators.required, Validators.email]],
    password: ['', [Validators.required, Validators.minLength(6)]],
  });

  accedi() {
    if (this.form.invalid) {
      this.form.markAllAsTouched();
      return;
    }
    const { email, password } = this.form.getRawValue();
    this.invio.set(true);
    this.errore.set('');
    this.auth.login(email, password).subscribe({
      next: () => this.router.navigateByUrl(this.returnUrl() ?? '/'),
      error: () => {
        this.errore.set('Email o password non corretti');
        this.invio.set(false);
      },
    });
  },
```

```

    });
  }
}

```

pagine/login/login.html

```

<form [formGroup]="form" (ngSubmit)="accedi()">
  <label>Email
    <input type="email" formControlName="email" autocomplete="username">
  </label>
  @if (form.controls.email.touched && form.controls.email.invalid) {
    <small>Inserisci un'email valida</small>
  }
  <label>Password
    <input type="password" formControlName="password" autocomplete="current-password">
  </label>
  @if (form.controls.password.touched && form.controls.password.hasError('minlength')) {
    <small>Almeno 6 caratteri</small>
  }
  @if (errore()) {
    <p class="errore">{{ errore() }}</p>
  }
  <button type="submit" [disabled]="invio()">
    {{ invio() ? 'Accesso in corso...' : 'Accedi' }}
  </button>
</form>

```

ReactiveFormsModule Serve per [formGroup] e formControlName. Il form è definito nella classe, non nel template.

nonNullable I campi tornano al valore iniziale con reset, e il tipo è string invece di string | null.

markAllAsTouched Mostra subito tutti gli errori se l'utente preme Accedi con il form vuoto.

ngSubmit Parte premendo Invio o il bottone type="submit", e impedisce il ricaricamento della pagina.

invio() Disattiva il bottone durante la richiesta, per evitare doppi invii.

Collegare tutto: rotte, configurazione, intestazione

app.routes.ts

```

export const routes: Routes = [
  { path: '', pathMatch: 'full', redirectTo: 'eroi' },
  { path: 'login', component: Login, canActivate: [ospiteGuard], title: 'Accedi' },
  { path: 'eroi', component: Eroi, title: 'Eroi' },
  { path: 'profilo', component: Profilo, canActivate: [authGuard], title: 'Profilo' },
  {
    path: 'admin',
    canActivate: [authGuard, ruoloGuard],
    data: { ruoli: ['admin'] },
    loadChildren: () => import('./admin/admin.routes').then((m) => m.ADMIN_ROUTES),
  },
  { path: 'non-autorizzato', component: NonAutorizzato },
  { path: '**', component: NonTrovata },
];

```

app.config.ts

```

export const appConfig: ApplicationConfig = {
  providers: [
    provideBrowserGlobalErrorListeners(),
    provideRouter(routes, withComponentInputBinding()),
    provideHttpClient(
      withFetch(),
      withInterceptors([authInterceptor, backendFintoInterceptor]),
    ),
    provideClientHydration(withEventReplay()),
  ],
};

```

app.ts e app.html: l'intestazione cambia in base al login.

```

export class App {
  protected auth = inject(AuthServizio);
}

<header>
  <a routerLink="/eroi">Eroi</a>
  @if (auth.isLoggato()) {
    <a routerLink="/profilo">Profilo</a>
    @if (auth.haRuolo('admin')) {
      <a routerLink="/admin">Admin</a>
    }
    <span>Ciao {{ auth.utente()?.nome }}</span>
    <button type="button" (click)="auth.logout()">Esci</button>
  } @else {
    <a routerLink="/login">Accedi</a>
  }
</header>
<router-outlet />

```

Nascondere il link Admin è comodità per l'utente, non sicurezza: la guardia blocca la pagina, e il server deve rifiutare le richieste di chi non è admin.

Un server finto per provare subito

Finché non hai un backend, un interceptor può fingere di essere il server. Deve stare per ultimo nell'array, così riceve la richiesta con il token già aggiunto. Quando arriva il server vero, lo toglie da `withInterceptors` e non cambia altro.

`auth/backend-finto-interceptor.ts`

```

import { HttpErrorResponse, HttpInterceptorFn, HttpResponse } from '@angular/common/http';
import { delay, of, throwError } from 'rxjs';
import { Utente } from './modelli';

const UTENTI_FINTI = [
  { id: 1, nome: 'Mario', email: 'mario@esempio.it', password: 'mario123', ruoli: ['utente'] },
  { id: 2, nome: 'Anna', email: 'admin@esempio.it', password: 'admin123', ruoli: ['utente', 'admin'] },
];

function creaTokenFinto(utente: Utente): string {
  const payload = { sub: utente.id, ruoli: utente.ruoli, exp: Math.floor(Date.now() / 1000) + 3600 };
  return `finto.${btoa(JSON.stringify(payload))}.firma`;
}

export const backendFintoInterceptor: HttpInterceptorFn = (req, next) => {
  if (req.method === 'POST' && req.url.endsWith('/auth/login')) {
    const { email, password } = req.body as { email: string; password: string };
    const trovato = UTENTI_FINTI.find((u) => u.email === email && u.password === password);
    if (!trovato) {
      return throwError(() => new HttpErrorResponse({ status: 401, url: req.url }));
    }
    const { password: _, ...utente } = trovato;
    return of(new HttpResponse({ status: 200, body: { token: creaTokenFinto(utente), utente } })))
      .pipe(delay(500));
  }

  if (req.url.endsWith('/profilo')) {
    if (!req.headers.has('Authorization')) {
      return throwError(() => new HttpErrorResponse({ status: 401, url: req.url }));
    }
    return of(new HttpResponse({ status: 200, body: { messaggio: 'Dati riservati' } }));
  }

  return next(req);
};

```

Prova: apri `/profilo` senza login e finisci su `/login?returnUrl=%2Fprofilo`. Accedi con l'utente Mario del file e torni al profilo. Apri `/admin` e finisci su `/non-autorizzato`. Esci, accedi con l'utente admin e `/admin` si apre. Ricarica la pagina: resti loggato grazie a `localStorage`. Nella scheda Rete degli strumenti del browser vedi l'intestazione `Authorization` nelle richieste.

`delay(500)` Simula la lentezza della rete, così vedi il bottone disattivato.

`const { password: _, ...utente }` Toglie la password dall'oggetto prima di rispondere. Un server vero non la rimanda mai.

`return next(req)` Tutte le altre richieste passano oltre, verso il server vero o json-server.

La pagina protetta che usa il token

```
export class Profilo {
  private http = inject(HttpClient);
  protected auth = inject(AuthServizio);
  dati = toSignal(this.http.get<{ messaggio: string }>(`${environment.apiUrl}/profilo`));
}

<h2>Profilo di {{ auth.utente()?.nome }}</h2>
<p>Email: {{ auth.utente()?.email }}</p>
<p>Dal server: {{ dati()?.messaggio }}</p>
```

Il componente non sa niente del token: lo aggiunge l'interceptor. È questo il vantaggio di averlo in un punto solo.

Login e SSR

corso-angular e tour-degli-eroi usano il rendering lato server (app.routes.server.ts). Sul server localStorage non c'è, quindi il server vede sempre l'utente come non loggato: una pagina protetta renderizzata lì farebbe scattare la guardia e il redirect al login, anche se nel browser il token c'è. Soluzione semplice: le pagine protette si disegnano solo nel browser.

```
// app.routes.server.ts
export const serverRoutes: ServerRoute[] = [
  { path: 'profilo', renderMode: RenderMode.Client },
  { path: 'admin/**', renderMode: RenderMode.Client },
  { path: '**', renderMode: RenderMode.Prenderer },
];
```

Le rotte più specifiche vanno prima di **. L'alternativa professionale è un cookie HttpOnly che il server legge a ogni richiesta, ma richiede un backend che lo gestisca.

Rinnovare il token (refresh token)

Un token di accesso dura poco, per esempio 15 minuti. Il server dà anche un refresh token che dura di più e serve solo a chiedere un nuovo token di accesso. Nell'interceptor, invece di fare subito logout sul 401, si prova a rinnovare e si ripete la richiesta.

```
return next(richiesta).pipe(
  catchError((err: HttpErrorResponse) => {
    if (err.status !== 401 || req.url.includes('/auth/')) {
      return throwError(() => err);
    }
    return auth.rinnova().pipe(
      // POST /auth/refresh, salva il nuovo token
      switchMap((nuovoToken) =>
        next(req.clone({ setHeaders: { Authorization: `Bearer ${nuovoToken}` } }))),
      catchError((e) => {
        auth.logout();
        return throwError(() => e);
      })
    );
  })
);
```

Se più richieste falliscono insieme, in un'app vera si fa partire un solo rinnovo e le altre aspettano il suo risultato, per esempio con shareReplay. È un dettaglio da affrontare quando serve davvero.

Sicurezza: cosa ricordare

Le guardie non proteggono i dati Il codice di Angular è nel browser e si può modificare. Una guardia evita di mostrare una pagina, ma solo il server può rifiutare i dati.

Mai password nel codice o in localStorage Si salva solo il token. Gli utenti finti del backend finto sono solo per provare in locale.

localStorage e XSS Uno script estraneo iniettato nella pagina può leggere localStorage. Angular protegge già i `{{ }}` e i binding; evita innerHTML con testo non tuo e `bypassSecurityTrust`.

Cookie HttpOnly Il token non è leggibile da JavaScript. È più sicuro, ma va gestito dal server e richiede protezione CSRF.

HTTPS sempre In produzione il token viaggia solo su connessioni cifrate.

Scadenza breve Un token rubato vale poco se scade presto. Il logout lato server invalida il refresh token.

PARTE TERZA

Qualità e pubblicazione

17. Angular Material

Cos'è

Angular Material è la libreria ufficiale di componenti grafici di Angular, secondo le regole Material Design di Google: campi di testo, bottoni, tabelle, finestre di dialogo, menu, avvisi. Sono accessibili (tastiera, lettori di schermo), funzionano con i form di Angular e si personalizzano con un tema. In corso-angular è già installata: lo vedi da `@angular/material` in `package.json` e dal tema in `styles.scss`.

Esempio completo nel progetto: `src/app/pagine/material-esempio`, rotta `/material`, link Angular Material nel menu.

Installazione e tema

```
ng add @angular/material
```

`ng add` installa la libreria, chiede un tema di colori e configura tutto: aggiunge il tema in `styles.scss` e i font Roboto e Material Icons in `index.html`. In corso-angular il tema è questo:

```
// src/styles.scss
@use '@angular/material' as mat;

html {
  @include mat.theme((
    color: (primary: mat.$cyan-palette, tertiary: mat.$orange-palette),
    typography: Roboto,
    density: 0,
  ));
}

body {
  color-scheme: light; // 'dark' per il tema scuro, 'light dark' per seguire il sistema
  background-color: var(--mat-sys-surface);
  color: var(--mat-sys-on-surface);
  font: var(--mat-sys-body-medium);
}
```

Il tema crea variabili CSS come `--mat-sys-primary`, `--mat-sys-surface`, `--mat-sys-outline`: usale anche nei tuoi stili per restare coerente con i colori di Material.

Usare un componente: import e tag

Ogni componente di Material si importa dal suo pacchetto e si mette negli imports del tuo componente, come una pipe o una direttiva.

```
import { MatButtonModule } from '@angular/material/button';
import { MatCardModule } from '@angular/material/card';
import { MatCheckboxModule } from '@angular/material/checkbox';
import { MatFormFieldModule } from '@angular/material/form-field';
import { MatIconModule } from '@angular/material/icon';
import { MatInputModule } from '@angular/material/input';
import { MatSelectModule } from '@angular/material/select';
import { MatSnackBar } from '@angular/material/snack-bar';
import { MatTableModule } from '@angular/material/table';

@Component({
  selector: 'app-material-esempio',
  imports: [ReactiveFormsModule, MatButtonModule, MatCardModule, MatCheckboxModule,
    MatFormFieldModule, MatIconModule, MatInputModule, MatSelectModule, MatTableModule, Troncato],
  templateUrl: './material-esempio.html',
})
```

Campi di un form: mat-form-field

```
<form [formGroup]="form" (ngSubmit)="aggiungi()" id="form-lezione">
  <mat-form-field appearance="outline">
    <mat-label>Titolo</mat-label>
    <input matInput formControlName="titolo">
    <mat-icon matSuffix>edit</mat-icon>
    @if (form.controls.titolo.hasError('required')) {
      <mat-error>Il titolo è obbligatorio</mat-error>
    } @else if (form.controls.titolo.hasError('minlength')) {
      <mat-error>Almeno 3 caratteri</mat-error>
    }
  </mat-form-field>

  <mat-form-field appearance="outline">
    <mat-label>Livello</mat-label>
    <mat-select formControlName="livello">
      @for (l of livelli; track l) { <mat-option [value]="l">{{ l }}</mat-option> }
    </mat-select>
  </mat-form-field>

  <mat-form-field appearance="outline">
    <mat-label>Descrizione</mat-label>
    <textarea matInput formControlName="descrizione" rows="3"></textarea>
    <mat-hint align="end">{{ form.controls.descrizione.value.length }}/120</mat-hint>
  </mat-form-field>
</form>
```

mat-form-field Il contenitore: bordo, etichetta che si sposta, messaggi. appearance="outline" o "fill".

matInput Direttiva da mettere su <input> e <textarea> dentro mat-form-field.

mat-label L'etichetta del campo. Con required Material aggiunge l'asterisco da solo.

mat-error Messaggio d'errore. Compare da solo quando il controllo è invalid e touched, oppure dopo il submit. Non serve il tuo @if su touched.

mat-hint Testo di aiuto sotto il campo.

matSuffix / matPrefix Icona o testo dentro il campo, a destra o a sinistra.

mat-select, mat-option Il select di Material. Funziona con formControlName e ngModel come un select normale.

Material funziona sia con i form reactive sia con i template-driven: le direttive formControlName e ngModel si mettono sugli stessi tag.

Bottoni e icone

```
<button matButton type="button" (click)="svuota()">Svuota</button>
<button matButton="filled" type="submit" form="form-lezione">
  <mat-icon>add</mat-icon> Aggiungi
</button>
<button matIconButton type="button" (click)="elimina(1)" aria-label="Elimina lezione">
  <mat-icon>delete</mat-icon>
</button>
```

matButton Bottone di testo. Con un valore cambia stile: matButton="filled", "outlined", "elevated", "tonal".

mat-flat-button, mat-raised-button, mat-stroked-button La forma vecchia degli stessi bottoni. Funziona ancora e la trovi nel codice esistente.

matIconButton Bottone rotondo con sola icona. Mettigli sempre aria-label: senza testo, un lettore di schermo non sa cosa fa.

<mat-icon>nome</mat-icon> Un'icona del font Material Icons. I nomi sono su fonts.google.com/icons.

form="form-lezione" Un bottone submit fuori dal <form> lo invia lo stesso, grazie all'id del form. Serve qui perché i bottoni stanno in mat-card-actions.

Card

```
<mat-card appearance="outlined">
  <mat-card-header>
    <mat-card-title>Nuova lezione</mat-card-title>
    <mat-card-subtitle>mat-form-field, matInput, mat-select</mat-card-subtitle>
  </mat-card-header>
  <mat-card-content> ... </mat-card-content>
  <mat-card-actions align="end"> ... bottoni ... </mat-card-actions>
</mat-card>
```

Tabella: mat-table

```
colonne = ['fatta', 'titolo', 'livello', 'descrizione', 'azioni'];
lezioni = signal<Lezione[]>([...]);

<table mat-table [dataSource]="lezioni()">
  <ng-container matColumnDef="titolo">
    <th mat-header-cell *matHeaderCellDef>Titolo</th>
    <td mat-cell *matCellDef="let l">{{ l.titolo }}</td>
  </ng-container>
  <ng-container matColumnDef="fatta">
    <th mat-header-cell *matHeaderCellDef>Fatta</th>
    <td mat-cell *matCellDef="let l">
      <mat-checkbox [checked]="l.fatta" (change)="segnaFatta(l, $event.checked)" />
    </td>
  </ng-container>
  <!-- ... una ng-container per ogni colonna ... -->

  <tr mat-header-row *matHeaderRowDef="colonne"></tr>
  <tr mat-row *matRowDef="let riga; columns: colonne"></tr>
  <tr class="mat-mdc-row" *matNoDataRow>
    <td class="mat-mdc-cell" [attr.colspan]="colonne.length">Nessuna lezione.</td>
  </tr>
</table>
```

[dataSource] L'array da mostrare. Passando un signal letto (lezioni()), la tabella si aggiorna quando il signal cambia.

matColumnDef="titolo" Definisce una colonna: intestazione (*matHeaderCellDef) e cella (*matCellDef, con let l per la riga).

colonne L'elenco e l'ordine delle colonne da mostrare. Togliendo un nome, la colonna sparisce.

***matNoDataRow** La riga mostrata quando l'array è vuoto.

Per ordinamento e pagine ci sono MatSortModule e MatPaginatorModule, da usare con la classe MatTableDataSource.

Avvisi: snack bar

```
private snackBar = inject(MatSnackBar);

this.snackBar.open(`Lezione "${nuova.titolo}" aggiunta`, 'OK', { duration: 3000 });

const avviso = this.snackBar.open(`Eliminata "${lezione.titolo}"`, 'Annulla', { duration: 5000 });
avviso.onAction().subscribe(() => this.lezioni.update((l) => [...l, lezione]));
```

MatSnackBar è un servizio: non va negli imports, si prende con inject. open mostra un avviso in basso; il secondo argomento è il testo del bottone; onAction avvisa quando l'utente lo preme. Nell'esempio Annulla rimette la lezione eliminata.

Per le finestre di dialogo il meccanismo è simile: MatDialog.open(ComponenteDialogo, { data }) apre un tuo componente in una finestra, e afterClosed() restituisce il valore scelto.

Reset di un form con Material

Dopo aver aggiunto una lezione, il form va svuotato. Con `this.form.reset()` il campo Titolo, vuoto e obbligatorio, diventava subito rosso con il messaggio d'errore. Il motivo: `mat-error` compare anche quando il form risulta inviato, e `reset()` azzerava valori e `touched` ma non lo stato "inviato", che appartiene alla direttiva `[formGroup]` del template. Soluzione:

```
private direttivaForm = viewChild.required(FormGroupDirective);

svuota() {
  this.direttivaForm().resetForm(); // valori iniziali, touched e submitted azzerati
}
```

`resetForm()` della direttiva azzerava anche `submitted`. È un errore che si incontra in quasi tutti i form con Material.

Altri componenti utili

MatDialog Finestre di dialogo modali.

MatMenuModule Menu a tendina su un bottone.

MatToolbarModule, MatSidenavModule Barra in alto e menu laterale per la struttura dell'app.

MatDatepickerModule Selettore di date. Richiede un adapter di date: `provideNativeDateAdapter()`.

MatAutocompleteModule Campo con suggerimenti mentre scrivi.

MatProgressSpinnerModule, MatProgressBarModule Indicatori di caricamento.

MatTabsModule, MatExpansionModule Schede e pannelli apribili.

MatTooltipModule Suggerimento al passaggio del mouse: `matTooltip="testo"`.

Il catalogo completo, con esempi da copiare, è su material.angular.dev.

18. Test

Perché e quali test

Un test è codice che controlla altro codice: chiama una funzione o crea un componente, e verifica che il risultato sia quello atteso. Si scrive una volta e si esegue sempre, in pochi secondi: se una modifica rompe qualcosa, lo scopri subito invece che dall'utente.

Test unitari Provano un pezzo alla volta: una pipe, un servizio, un componente. Sono i file `.spec.ts`. Si eseguono con `ng test`.

Test end-to-end (e2e) Aprono l'app in un browser vero e cliccano come un utente. Si fanno con strumenti come Playwright o Cypress, da aggiungere al progetto.

Angular 21 usa Vitest per i test unitari: lo vedi in `package.json` (`vitest`, `jsdom`) e in `angular.json` (`builder @angular/build:unit-test`). Nei progetti più vecchi trovi Karma e Jasmine: la sintassi dei test è quasi identica.

Eseguire i test

```
ng test                # esegue tutti i test e resta in ascolto: li rifà a ogni salvataggio
ng test --watch=false  # una volta sola, poi termina
ng test --include src/app/pipes/troncato.spec.ts # solo un file
```

Il risultato in corso-angular dopo le correzioni di questo capitolo:

```
Test Files  20 passed (20)
Tests      30 passed (30)
```

Se anche un solo file `.spec.ts` non compila, non parte nessun test: l'errore di compilazione blocca tutto. Va sistemato per primo.

La struttura di un test

```
describe('Troncato', () => {                // un gruppo di test, di solito uno per file
  const pipe = new Troncato();

  it('lascia invariato un testo corto', () => { // un singolo caso
    expect(pipe.transform('Angular')).toBe('Angular');
  });
});
```

describe('nome', () => ...) Raggruppa i test di una stessa cosa.

it('cosa deve fare', () => ...) Un caso. Il nome è una frase: quando fallisce, dice cosa non va.

expect(valore) Il valore da controllare, seguito da un confronto.

.toBe(x) Uguale, con `===`. Per numeri, stringhe, booleani.

.toEqual(x) Uguale nel contenuto. Per oggetti e array.

.toBeTruthy() / **.toBeFalsy()** Vero o falso in senso largo.

.toContain(x) La stringa o l'array contiene x.

beforeEach(() => ...) Codice eseguito prima di ogni it: prepara un ambiente pulito.

afterEach(() => ...) Codice eseguito dopo ogni it.

Testare una pipe

Una pipe è una classe normale: non serve Angular, basta crearla e chiamare transform.

src/app/pipes/troncato.spec.ts:

```
import { Troncato } from './troncato';

describe('Troncato', () => {
  const pipe = new Troncato();

  it('lascia invariato un testo corto', () => {
    expect(pipe.transform('Angular')).toBe('Angular');
  });

  it('accorcia un testo lungo e aggiunge i puntini', () => {
    expect(pipe.transform('Corso completo di Angular 21', 10)).toBe('Corso comp...');
  });

  it('usa la fine scelta e toglie lo spazio prima', () => {
    expect(pipe.transform('Signal e zoneless', 7, '...')).toBe('Signal...');
  });

  it('restituisce stringa vuota per null e undefined', () => {
    expect(pipe.transform(null)).toBe('');
    expect(pipe.transform(undefined)).toBe('');
  });
});
```

Testare una direttiva

Il test generato da ng g directive per Highlight faceva new Highlight(), ma il constructor della direttiva vuole un ElementRef: il file non compilava, e bloccava tutti i test del progetto (errore TS2554: Expected 1 arguments, but got 0). Versione corretta, src/app/direttive/highlight.spec.ts:

```
import { ElementRef } from '@angular/core';
import { Highlight } from './highlight';

describe('Highlight', () => {
  function creaDirettiva() {
    const elemento = document.createElement('p');
    return { elemento, direttiva: new Highlight(new ElementRef(elemento)) };
  }

  it('colora lo sfondo con il colore scelto', () => {
    const { elemento, direttiva } = creaDirettiva();
    direttiva.appHighlight = 'yellow';
    expect(elemento.style.backgroundColor).toBe('yellow');
  });

  it('torna al colore predefinito quando il mouse esce', () => {
    const { elemento, direttiva } = creaDirettiva();
    direttiva.defaultColor = 'pink';
    direttiva.appHighlight = 'yellow';
    direttiva.onMouseLeave();
    expect(elemento.style.backgroundColor).toBe('pink');
  });
});
```

document.createElement funziona anche nei test perché Vitest usa jsdom, un finto browser in memoria.

Testare un componente con TestBed

TestBed crea un piccolo ambiente Angular solo per il test: il componente, i suoi imports e i provider che gli servono.

```
import { ComponentFixture, TestBed } from '@angular/core/testing';
import { provideRouter } from '@angular/router';
import { Pagina0 } from './pagina0';

describe('Pagina0', () => {
  let component: Pagina0;
  let fixture: ComponentFixture<Pagina0>;

  beforeEach(async () => {
    await TestBed.configureTestingModule({
      imports: [Pagina0],
      providers: [provideRouter([])], // il template usa routerLink: serve il router
    }).compileComponents();

    fixture = TestBed.createComponent(Pagina0);
    component = fixture.componentInstance;
    await fixture.whenStable();
  });

  it('should create', () => {
    expect(component).toBeTruthy();
  });
});
```

TestBed.configureTestingModule Come app.config.ts, ma per il test: imports e providers.

TestBed.createComponent Crea il componente. fixture è il suo contenitore.

fixture.componentInstance L'istanza della classe: puoi chiamarne i metodi e leggerne i signal.

fixture.nativeElement L'HTML disegnato: ci cerchi elementi con querySelector.

await fixture.whenStable() Aspetta che Angular abbia finito di aggiornare la vista.

provideRouter([]) Un router vuoto. Senza, un componente con routerLink fallisce con NG0201: No provider found for ActivatedRoute.

Un test che controlla il contenuto della pagina, app.spec.ts:

```
it('should render the menu', async () => {
  const fixture = TestBed.createComponent(App);
  await fixture.whenStable();
  const compiled = fixture.nativeElement as HTMLElement;
  expect(compiled.querySelector('menu a[href="/pagina0"]')?.textContent).toContain('Pagina 0');
  expect(compiled.querySelector('router-outlet')).toBeTruthy();
});
```

Questo test prima cercava <app-prova>, un tag tolto da app.html in una modifica precedente: il test era rimasto indietro rispetto al codice. È normale: quando cambi un componente, aggiorna anche il suo test.

Per simulare un'azione: trova il bottone, chiamane click(), aspetta whenStable e controlla il risultato.

```
const bottone = fixture.nativeElement.querySelector('button') as HTMLButtonElement;
bottone.click();
await fixture.whenStable();
expect(fixture.nativeElement.textContent).toContain('Salvato');
```

Testare un servizio HTTP

Nei test non partono richieste vere. provideHttpClientTesting sostituisce il server con HttpTestingController: il test controlla quale richiesta è partita e risponde a mano. src/app/servizi/corsi-firebase.spec.ts:

```
import { TestBed } from '@angular/core/testing';
import { provideHttpClient } from '@angular/common/http';
import { HttpTestingController, provideHttpClientTesting } from '@angular/common/http/testing';
import { CorsiFirebase } from './corsi-firebase';
import { environment } from '../../environments/environment';

describe('CorsiFirebase', () => {
  let servizio: CorsiFirebase;
  let http: HttpTestingController;
  const base = `${environment.firebaseUrl}/corsi`;

  beforeEach(() => {
```

```

TestBed.configureTestingModule({
  providers: [provideHttpClient(), provideHttpClientTesting()],
});
servizio = TestBed.inject(CorsiFirebase);
http = TestBed.inject(HttpTestingController);
});

afterEach(() => http.verify()); // fallisce se è rimasta una richiesta senza risposta

it('lista: fa GET e trasforma l\'oggetto di Firebase in un array', () => {
  let risultato: unknown;
  servizio.lista().subscribe((corsi) => (risultato = corsi));

  const req = http.expectOne((r) => r.url === `${base}.json`);
  expect(req.request.method).toBe('GET');
  expect(req.request.params.get('orderBy')).toBe('$key');
  req.flush({ '-a1': { titolo: 'RxJS', docente: 'Bianchi', ore: 8, attivo: true } });

  expect(risultato).toEqual([
    { id: '-a1', titolo: 'RxJS', docente: 'Bianchi', ore: 8, attivo: true },
  ]);
});

it('crea: fa POST con il body e usa name come id', () => {
  const nuovo = { titolo: 'Angular', docente: 'Rossi', ore: 16, attivo: true };
  let risultato: unknown;
  servizio.crea(nuovo).subscribe((corso) => (risultato = corso));

  const req = http.expectOne(`${base}.json`);
  expect(req.request.method).toBe('POST');
  expect(req.request.body).toEqual(nuovo);
  req.flush({ name: '-b2' });

  expect(risultato).toEqual({ id: '-b2', ...nuovo });
});

it('trasforma un 401 in un messaggio leggibile', () => {
  let messaggio = '';
  servizio.lista().subscribe({ error: (e: Error) => (messaggio = e.message) });
  http.expectOne((r) => r.url === `${base}.json`)
    .flush({ error: 'Permission denied' }, { status: 401, statusText: 'Unauthorized' });
  expect(messaggio).toContain('Permesso negato');
});
});

```

TestBed.inject(Classe) Prende il servizio, come inject() nell'app.

http.expectOne(url o funzione) Controlla che sia partita una sola richiesta corrispondente e la restituisce. Con i query params si confronta r.url, che non li contiene.

req.request La richiesta: method, body, headers, params.

req.flush(dati, opzioni) Risponde con quei dati. Con status 401 simula un errore.

http.verify() In afterEach: nessuna richiesta deve restare senza risposta.

Servizi finti nei test di componente

Per testare un componente senza il servizio vero, gli si passa una versione finta con useValue:

```

const apiFinta = { lista: () => of([{ id: '1', titolo: 'Test', docente: 'X', ore: 1, attivo: true }]) };

TestBed.configureTestingModule({
  imports: [CorsiFirebasePagina],
  providers: [{ provide: CorsiFirebase, useValue: apiFinta }],
});

```

Il componente chiede CorsiFirebase con inject e riceve l'oggetto finto: il test non dipende dal server e controlla solo il comportamento del componente.

Errori tipici nei test

TS2554 Expected 1 arguments, but got 0 Il test crea una classe con new senza i parametri del constructor. Passa i parametri o usa TestBed.

NG0201 No provider found for ActivatedRoute / Router Il componente usa il router: aggiungi `provideRouter([])`.

NG0201 No provider found for HttpClient Aggiungi `provideHttpClient()` e `provideHttpClientTesting()`.

NG0701 Missing locale data for the locale "it" Una pipe usa la lingua italiana non registrata: `registerLocaleData(localeIt)`. Nel progetto il test di `pipes.spec.ts` ha scoperto questo errore, che sarebbe comparso anche nell'app.

Expected one matching request, found none Il servizio non ha fatto la richiesta attesa, o l'URL è diverso: controlla query params e barre.

Il test passa da solo ma fallisce con gli altri Stato condiviso tra test: preparalo in `beforeEach`, non fuori.

expect(...) dopo subscribe non vede il valore Con `HttpTestingController` il valore arriva solo dopo `flush`: metti `expect` dopo `flush`.

19. Build, SSR e pubblicazione

ng build

```
ng build
```

ng build prepara l'app per la pubblicazione: traduce TypeScript in JavaScript, unisce e comprime i file, toglie il codice non usato, sostituisce environment.development.ts con environment.ts. Il risultato va nella cartella dist/.

In corso-angular, con SSR attivo, la cartella è fatta così:

dist/corso-angular/browser/ I file da mandare al browser: JavaScript, CSS, immagini, e le pagine prerenderizzate.

dist/corso-angular/browser/index.csr.html La pagina di partenza per il rendering solo lato browser.

dist/corso-angular/browser/pagina0/index.html Una pagina già disegnata durante la build (prerender).

dist/corso-angular/server/ Il codice del server Node che disegna le pagine su richiesta.

prerendered-routes.json L'elenco delle pagine prerenderizzate.

ng build usa la configurazione production di angular.json: ottimizzazioni attive e controllo dei budget, cioè limiti di dimensione. Se il bundle supera il budget, la build avvisa o fallisce: è un segnale che stai importando troppo. ng build --configuration development fa una build veloce, non ottimizzata.

Rendering lato server (SSR) e prerender

Senza SSR il server manda una pagina quasi vuota, e il browser la costruisce eseguendo il JavaScript. Con SSR la pagina arriva già disegnata: si vede prima ed è leggibile dai motori di ricerca. Poi Angular prende il controllo dell'HTML esistente senza ridisegnarlo: è l'hydration (provideClientHydration in app.config.ts).

Per ogni rotta si sceglie come disegnarla, in app.routes.server.ts:

```
export const serverRoutes: ServerRoute[] = [
  { path: 'dettaglio/:id', renderMode: RenderMode.Server },
  { path: 'corsi-firebase', renderMode: RenderMode.Client },
  { path: '**', renderMode: RenderMode.Prerender },
];
```

RenderMode.Prerender La pagina viene disegnata una volta durante ng build e salvata come HTML statico. Velocissima. Per pagine uguali per tutti.

RenderMode.Server Disegnata dal server Node a ogni richiesta. Per pagine con dati che cambiano, come un dettaglio.

RenderMode.Client Disegnata solo nel browser, come senza SSR. Per pagine protette dal login o che chiamano API durante il caricamento.

Il codice che gira anche sul server non può usare window, document, localStorage o alert: sul server non esistono. Controlla con isPlatformBrowser(inject(PLATFORM_ID)), oppure metti il codice in afterNextRender. L'HTML disegnato sul server e quello nel browser devono coincidere: una <table> senza <tbody> fa fallire l'hydration con NG0500, come è successo nella pagina /iscrizione.

Provare la build con SSR in locale:

```
npm run build
npm run serve:ssr:corso-angular
```

Pubblicare su Firebase Hosting

Firebase Hosting pubblica un sito statico su un indirizzo https, gratis per progetti piccoli. Va bene per le pagine prerenderizzate e per quelle Client; le pagine RenderMode.Server invece richiedono un server Node, che su Firebase si ottiene con App Hosting, un servizio a parte.

1) Installa gli strumenti di Firebase e accedi con il tuo account Google:

```
npm install -g firebase-tools
firebase login
```

2) Nella cartella del progetto, collega il progetto Firebase (lo stesso del capitolo 15) e configura l'hosting:

```
firebase init hosting
```

Rispondi così: usa un progetto esistente e scegli il tuo; come cartella pubblica scrivi dist/corso-angular/browser; alla domanda se è un'app a pagina singola (single-page app) rispondi sì; alla domanda su GitHub rispondi no; non sovrascrivere index.html se te lo chiede.

3) Controlla firebase.json. Poiché con SSR la pagina di partenza si chiama index.csr.html, tutte le rotte non prerenderizzate devono puntare lì:

```
{
  "hosting": {
    "public": "dist/corso-angular/browser",
    "ignore": ["firebase.json", "**/.*", "**/node_modules/**"],
    "rewrites": [{ "source": "**", "destination": "/index.csr.html" }]
  }
}
```

4) Fai la build e pubblica:

```
ng build
firebase deploy --only hosting
```

Alla fine il comando stampa l'indirizzo pubblico, del tipo https://corso-angular-12345.web.app. Prima di pubblicare, metti in environment.ts l'URL vero del database e controlla le regole di sicurezza: in produzione non si lascia la modalità test.

Il rewrite è indispensabile per un'app con router: se l'utente apre direttamente https://.../corsi-firebase o ricarica la pagina, il server non ha un file con quel nome, e senza rewrite risponderebbe 404. Con il rewrite consegna l'app, e il router di Angular mostra la pagina giusta.

Altri modi di pubblicare

Hosting statico (Netlify, Vercel, GitHub Pages, un server web) Pubblica la cartella browser con la stessa regola: ogni percorso sconosciuto va alla pagina di partenza.

Server Node Per le rotte RenderMode.Server: si avvia node dist/corso-angular/server/server.mjs su un server o un servizio cloud.

Senza SSR Se non ti serve, crea il progetto con ng new --ssr=false: la build produce solo file statici e la pagina di partenza è index.html.

Sottocartella Se l'app non sta alla radice del dominio: ng build --base-href /nome-cartella/.

Prima di pubblicare: controllo finale

- ng test --watch=false passa senza errori.
- ng build termina senza errori né avvisi di budget.
- environment.ts contiene gli indirizzi di produzione, non localhost.
- Nessun segreto nel codice: tutto ciò che è in src/ finisce nel browser.

- Le regole del database non sono in modalità test.
- Provata la build in locale, anche ricaricando una pagina interna.

20. Buone pratiche ed errori frequenti

Organizzare le cartelle

Nei progetti piccoli va bene una cartella per tipo: pagine, servizi, pipe, direttive. Quando l'app cresce, conviene una cartella per funzionalità, con dentro tutto ciò che la riguarda:

```
src/app/
  core/                servizi usati ovunque: auth, interceptor, registro http
  shared/              componenti, pipe e direttive riusabili: card, troncato, highlight
  corsi/               una funzionalità: pagine, componenti e servizio dei corsi
    corsi.routes.ts    le sue rotte, caricate con loadChildren
    lista-corsi/
      dettaglio-corso/
        corsi-api.ts
  iscrizioni/
  app.config.ts
  app.routes.ts
```

Così una funzionalità si trova, si sposta e si carica in lazy loading tutta insieme.

Regole di stile

- Un file per ogni componente, servizio, pipe o direttiva. Il nome del file è il nome della classe in minuscolo con i trattini: dettaglio-corso.ts contiene DettaglioCorso.
- Prefisso app- nei selector dei componenti e app nelle direttive: app-card-eroe, appHighlight.
- Nomi che dicono cosa fa: caricaCorsi, eroiFiltrati, isLoggato. Un booleano risponde a una domanda: attivo, inModifica, haErrori.
- Tipi espliciti per i dati: un'interfaccia per ogni oggetto che arriva dal server. Niente any.
- strict: true in tsconfig.json, sempre.
- Formatta il codice in modo automatico: il progetto ha Prettier (.prettierrc). In VS Code attiva Formatta al salvataggio.

Componenti

- Componenti piccoli: se un template supera un centinaio di righe o fa due cose, dividilo.
- Separa i componenti che caricano dati (pagine, "smart") da quelli che mostrano soltanto (card, righe, "presentational"). I secondi ricevono tutto con input e comunicano con output: sono facili da riusare e da testare.
- Niente logica complicata nel template: spostala in un computed con un nome chiaro.
- Il template legge; le modifiche passano da metodi della classe.
- inject() nei campi, non constructor lunghi. protected per ciò che usa il template, private per il resto.
- track con un id vero in ogni @for.

Stato e dati

- Signal per lo stato che cambia; computed per i valori derivati; mai modificare un array o un oggetto dentro un signal: sempre una copia nuova.
- Lo stato condiviso da più componenti sta in un servizio, con signal privati ed esposti in sola lettura.
- Le chiamate HTTP stanno nei servizi, non nei componenti. Il componente chiama un metodo con un nome chiaro: corsi.lista(), non http.get(url).

- Gestisci sempre tre stati: caricamento, errore, dati. L'utente deve sapere cosa sta succedendo.
- Ogni subscribe che può restare aperto si chiude: `takeUntilDestroyed`, `async` o `toSignal`.
- URL e configurazioni negli environment, non scritti nel codice.

Prestazioni

- Lazy loading delle sezioni con `loadComponent` e `loadChildren`: il primo caricamento resta leggero.
- `@defer` per i pezzi pesanti non visibili subito.
- `NgOptimizedImage` (`ngSrc`) per le immagini.
- Pipe pure e computed invece di chiamare funzioni costose nel template.
- `shareReplay` o un servizio con `signal` per non ripetere la stessa richiesta.

Accessibilità

- Usa i tag giusti: `button` per le azioni, `a` per i link, `label` per ogni campo, un solo `h1` per pagina.
- Ogni immagine ha `alt`; ogni bottone con sola icona ha `aria-label`.
- Tutto deve funzionare da tastiera: prova la pagina con `Tab` e `Invio`.
- Non affidare un'informazione solo al colore: accanto al bordo rosso metti il messaggio d'errore.
- I componenti di Angular Material sono già accessibili: usarli aiuta.

Sicurezza

- Il codice Angular è nel browser e chiunque può leggerlo: nessun segreto, nessuna password, nessuna chiave privata in `src/`.
- Le guardie migliorano l'esperienza, ma i dati li protegge solo il server.
- Angular protegge da solo `{{ }}` e i binding dagli script iniettati. Non disattivare la protezione con `bypassSecurityTrust` e non usare `innerHTML` con testo che viene dall'utente.
- HTTPS in produzione, token con scadenza breve.

Errori già visti

Pipe, `ngModel`, `ngFor` o un componente usati nel template ma assenti da `imports` La compilazione si ferma: aggiungi la classe all'array `imports` del componente.

`{{ title }}` stampa il signal `{{ title() }}` stampa il testo.

`[(ngModel)]` non accetta `?.` Angular deve poter scrivere in quel campo: niente optional chaining dentro `[( )]`.

Leggere `.nome` quando l'eroe non c'è ancora Avvolgi quel pezzo in `@if`.

`ng serve` che si chiude o non vede le modifiche dopo un `import` o un file nuovo È la cache di Vite: riavvia il comando, il codice può essere a posto. Se non basta, `ng cache clean`.

La pagina non si aggiorna dopo una risposta HTTP App zoneless e proprietà normale modificata nel `subscribe`: usa un `signal`.

NG0500 During hydration Angular expected... Con SSR l'HTML del server e del browser non coincidono. Classico: `<table>` senza `<tbody>`.

NG0203 inject() must be called from an injection context `inject`, `toSignal`, `effect` o `takeUntilDestroyed()` usati in un metodo o in `ngOnInit`. Spostali in un campo o nel constructor.

NG0201 / NullInjectorError: No provider for X Manca il provider: provideHttpClient, provideRouter, oppure il servizio non ha providedIn: 'root'.

NG0701 Missing locale data Registra la lingua con registerLocaleData.

Il bottone ricarica la pagina Un bottone dentro un form senza type="button", oppure un form senza FormsModule o ReactiveFormsModule.

EADDRINUSE: address already in use La porta è occupata da un altro programma: ng serve --port 4300, o un'altra porta per il server finto.

La porta 4200 è già in uso C'è ancora un ng serve aperto in un altro terminale.

Come cercare la soluzione a un errore

- 1) Leggi il messaggio per intero, dall'inizio: la prima riga dice cosa, le righe con il nome dei tuoi file dicono dove.
- 2) Se c'è un codice come NG0203, cercalo: il link nel messaggio porta alla pagina di spiegazione su angular.dev.
- 3) Controlla la console del browser (F12) e il terminale di ng serve: gli errori di compilazione stanno nel terminale, quelli durante l'esecuzione nella console.
- 4) Per le chiamate HTTP guarda la scheda Rete degli strumenti del browser: URL, metodo, status, risposta.
- 5) Riduci il problema: commenta metà del template e vedi se l'errore resta.
- 6) La documentazione ufficiale è su angular.dev: guide, riferimento delle API e tutorial interattivi.

Ogni giorno

ng serve Avvia l'app su http://localhost:4200 e ricompila quando salvi.

ng g component nome Crea un componente.

ng g service nome Crea un servizio.

ng test Esegue i test e li ripete a ogni salvataggio.

ng build Prepara l'app per la pubblicazione.

Ordine mentale: i dati stanno nella classe o in un servizio. Il template li mostra. Un click chiama un metodo. Il figlio riceve input e, se deve rispondere, manda un output.

PARTE QUARTA

Esercizi e appendici

21. Esercizi

Falli in ordine nel progetto corso-angular: ognuno usa quello che hai imparato prima. Prova senza guardare il codice degli esempi; quando hai finito, confronta la tua soluzione con il capitolo. Il numero tra parentesi è il capitolo di riferimento.

Basi (capitoli 1-8)

- 1. (1) Crea un progetto nuovo con `ng new prova-angular`, avvialo, cambia il titolo in `app.html` e guarda la pagina aggiornarsi da sola.
- 2. (2) Scrivi un'interfaccia `Libro` con titolo, autore, anno e un campo facoltativo prestato. Crea un array di tre libri e usa `filter`, `map` e `find` per ottenere i libri dopo il 2000, i soli titoli, il libro di un autore.
- 3. (4) Crea un componente `ListaLibri` che mostri i libri con `@for`, evidenzi con `[class.selezionato]` quello cliccato e ne mostri i dettagli sotto con `@if`.
- 4. (4) Nel componente aggiungi un contatore di secondi con `setInterval` avviato in `ngOnInit` e fermato in `ngOnDestroy`. Verifica con `console.log` che si fermi quando cambi pagina.
- 5. (5) Mostra un messaggio diverso con `@switch` in base al numero di libri: nessuno, uno, molti. Aggiungi `@empty` al `@for`.
- 6. (6) Aggiungi un campo di ricerca e un computed `libriFiltrati`. Aggiungi un effect che salvi la ricerca in `localStorage`.
- 7. (7) Crea una pipe iniziale che trasformi "Mario Rossi" in "M.R.". Scrivi il suo test.
- 8. (7) Registra la lingua italiana con `LOCALE_ID` e mostra una data con `date:'fullDate'` e un prezzo con `currency:'EUR'`.
- 9. (8) Crea una direttiva `appMaiuscolo` che al blur di un input trasformi il testo in maiuscolo, usando `host`.

Comunicazione e servizi (capitoli 9-11)

- 10. (9) Crea un componente `CardPersona` con `input.required<Persona>()` e `output<number>()` per l'eliminazione. Usalo nella lista.
- 11. (9) Crea un Contatore con `model()` e collegalo a un signal del padre con `[(valore)]`.
- 12. (9) Crea un `PreferitiServizio` con un signal privato, `asReadOnly` e `computed` per il numero. Un bottone nella card aggiunge ai preferiti, un badge in `app.html` mostra il totale.
- 13. (10) Crea un `InjectionToken NOME_APP` con il nome dell'applicazione e mostralo nell'intestazione.
- 14. (11) Crea un campo di ricerca con `FormControl`, `debounceTime` e `switchMap` che filtri un elenco finto restituito con `of(...).pipe(delay(500))`. Verifica nel log che le ricerche precedenti vengano annullate.

Routing (capitolo 12)

- 15. Aggiungi `withComponentInputBinding` in `app.config.ts` e trasforma `Dettaglio` perché legga id con `input.required({ transform: numberAttribute })` al posto di `route.snapshot`.
- 16. Aggiungi una pagina `NonTrovata` e la rotta `**` in fondo. Prova un indirizzo inventato.
- 17. Metti `routerLinkActive` sui link di `app.html` con una classe attivo nello scss.

- 18. Nella lista persone aggiungi un campo di ricerca che scrive ?cerca= nell'indirizzo. Ricarica la pagina: il filtro deve restare.
- 19. Carica una pagina con loadComponent e verifica nella scheda Rete del browser che il suo file venga scaricato solo quando la apri.

Form (capitoli 13-14)

- 20. Nella pagina /ngmodel aggiungi un campo email con [(ngModel)] su un signal e mostra con computed se contiene una chiocciola.
- 21. Crea un form template-driven Contatti con nome, email e messaggio: required, email, minlength="10" sul messaggio, errori mostrati con touched o submitted, e un riepilogo dopo l'invio.
- 22. Scrivi un validatore personalizzato appSoloLettere che rifiuti numeri nel nome e usalo nel form Contatti.
- 23. Prendi il form Contatti con viewChild, iscriviti a statusChanges e mostra quante volte è passato da INVALID a VALID.
- 24. Rifai il form Contatti in versione reactive con new FormGroup e new FormControl, e confronta i due file.
- 25. Aggiungi al form reactive un campo codice sconto facoltativo: con un checkbox Ho un codice il campo si abilita e diventa obbligatorio, con un pattern di 8 caratteri maiuscoli.
- 26. Trasforma i telefoni della pagina /iscrizione-reactive in un FormArray di FormGroup con tipo (casa, lavoro) e numero.
- 27. Riscrivi il form reactive con FormBuilder.nonNullable e verifica che il comportamento resti identico.
- 28. Aggiungi modificheGuard a un form di modifica e verifica che chieda conferma prima di uscire.

HTTP e autenticazione (capitoli 15-16)

- 29. Installa json-server, crea db.json con le persone e scrivi un servizio CRUD. Mostra caricamento ed errore; spegni json-server e verifica il messaggio Server non raggiungibile.
- 30. Crea il database su Firebase, incolla l'URL in environment.development.ts e usa la pagina /corsi-firebase con il database vero. Guarda i dati comparire nella console di Firebase.
- 31. Nella pagina /corsi-firebase aggiungi una ricerca per docente con orderBy="docente" ed equalTo. Aggiungi .indexOn nelle regole e verifica l'errore 400 senza indice.
- 32. Aggiungi un GET di un solo corso (/corsi/ID.json) e una pagina di dettaglio con rotta corsi-firebase/:id.
- 33. Leggi la risposta completa con observe: 'response' e mostra status e Content-Type nel registro.
- 34. Aggiungi l'autenticazione completa del capitolo 16 con il backend finto. Proteggi pagina2 con authGuard e crea una sezione admin con rotte figlie e ruoloGuard.
- 35. Attiva Firebase Authentication con Email/Password, cambia le regole in auth != null e fai funzionare /corsi-firebase con il login e firebaseAuthInterceptor.

Qualità e pubblicazione (capitoli 17-20)

- 36. (17) Rifai il form Contatti con mat-form-field, mat-error e un matButton="filled". Dopo l'invio mostra uno snack bar e svuota il form con resetForm.

- 37. (17) Mostra i corsi di Firebase in una mat-table con una colonna azioni e un MatDialog di conferma prima di eliminare.
- 38. (18) Scrivi il test del PreferitiServizio: aggiungi due elementi, toglie uno, controlla il computed del numero.
- 39. (18) Scrivi il test di CardPersona: passa un input con `fixture.componentRef.setInput('persona', ...)`, clicca Elimina e verifica che l'output emetta l'id.
- 40. (19) Pubblica corso-angular su Firebase Hosting. Apri direttamente l'indirizzo di una pagina interna e ricaricala: deve funzionare grazie al rewrite.

Appendice A. Glossario

- Angular CLI** Il comando ng: crea progetti e file, avvia, testa e compila l'app.
- AsyncPipe** La pipe async: si iscrive a un Observable nel template e si stacca da sola.
- Binding** Collegamento tra dati della classe e template: {{ }}, [], (), [()].
- Bootstrap** L'avvio dell'app: bootstrapApplication(App, appConfig) in main.ts.
- Change detection** Il meccanismo che aggiorna la pagina quando i dati cambiano.
- CLI** Command Line Interface: un programma che si usa scrivendo comandi nel terminale.
- Componente** Classe con template e stile, usata come tag: la parte base di ogni app Angular.
- computed** Signal calcolato da altri signal, aggiornato da solo.
- Control flow** I blocchi @if, @for, @switch, @defer, @let del template.
- CORS** Regola del browser che blocca le chiamate verso un altro dominio se il server non le permette.
- CRUD** Create, Read, Update, Delete: le quattro operazioni sui dati.
- Decoratore** La @ sopra una classe o una proprietà: @Component, @Injectable, @Input.
- Dependency injection (DI)** Angular crea i servizi e li consegna a chi li chiede con inject().
- Direttiva** Classe che aggiunge comportamento a un elemento: di attributo o strutturale.
- effect** Funzione che si riesegue quando cambiano i signal che legge.
- Endpoint** L'indirizzo di una risorsa di un'API: https://server/api/corsi/3.
- Environment** File con i valori diversi per sviluppo e produzione.
- FormArray** Lista di controlli di lunghezza variabile in un form reactive.
- FormControl** Un campo di un form: valore, stato, errori.
- FormGroup** Un gruppo di controlli con un nome ciascuno.
- Guardia (guard)** Funzione che permette o blocca l'accesso a una rotta.
- Header** Informazione di una richiesta o risposta HTTP: Content-Type, Authorization.
- Hook del ciclo di vita** Metodi chiamati da Angular in momenti precisi: ngOnInit, ngOnDestroy...
- HttpClient** Il servizio di Angular per fare chiamate HTTP.
- Hydration** Angular riusa l'HTML disegnato dal server invece di ridisegnarlo.
- InjectionToken** Una chiave per iniettare un valore che non è una classe.
- input() / @Input** Proprietà di un componente che riceve un valore dal padre.
- Interceptor** Funzione che vede passare ogni richiesta HTTP e la sua risposta.
- JSON** Formato di testo per i dati: { "nome": "Mario" }.
- JWT** JSON Web Token: token firmato che identifica l'utente.
- Lazy loading** Scaricare il codice di una pagina solo quando serve.
- model()** Input modificabile dal figlio: legame nei due sensi con [()].
- NgModule** Il vecchio sistema di moduli di Angular, prima dei componenti standalone.
- Observable** Flusso di valori nel tempo, della libreria RxJS.
- output() / @Output** Evento con cui il figlio avvisa il padre.
- Pipe** Trasformazione di un valore nel template: {{ data | date }}.
- Prerender** Disegnare una pagina durante la build e salvarla come HTML.
- Provider** Istruzione che dice ad Angular come creare un servizio o un valore.

Query params Parte dell'indirizzo dopo ?: /eroi?cerca=man.

Reactive form Form costruito nella classe con FormGroup e FormControl.

Resolver Funzione che carica dati prima di mostrare una rotta.

REST Stile di API: la risorsa nell'indirizzo, l'azione nel metodo HTTP.

Rotta (route) Associazione tra un indirizzo e un componente.

router-outlet Il punto del template dove il router mostra la pagina.

RxJS La libreria degli Observable e degli operatori.

Selector Il nome del tag o dell'attributo di un componente o di una direttiva.

Servizio Classe condivisa con dati e logica, presa con inject().

Signal Contenitore di un valore che avvisa quando cambia.

SPA Single Page Application: una pagina sola, contenuto cambiato da JavaScript.

SSR Server-Side Rendering: pagine disegnate sul server.

Standalone Componente che dichiara da solo i suoi imports, senza NgModule.

Status code Il numero della risposta HTTP: 200, 404, 500.

Subscribe Iniziare ad ascoltare un Observable.

Template L'HTML di un componente, con la sintassi di Angular.

Template-driven form Form descritto nel template con ngModel.

TestBed L'ambiente Angular per i test dei componenti.

track Nel @for, come riconoscere ogni elemento della lista.

TypeScript JavaScript con i tipi, il linguaggio di Angular.

Validatore Funzione o direttiva che controlla il valore di un campo.

Vite Lo strumento che compila e serve l'app durante lo sviluppo.

Vitest Il programma che esegue i test in Angular 21.

Zoneless App senza zone.js: la pagina si aggiorna con signal ed eventi del template.

Appendice B. Gli esempi nel progetto corso-angular

src/app/pagine/iscrizione Form template-driven completo: validatori pronti e personalizzati, gruppi, stati, viewChild. Rotta /iscrizione.

src/app/pagine/esempi-ngmodel Nove varianti di ngModel. Rotta /ngmodel.

src/app/pagine/iscrizione-reactive Form reactive con FormArray, campo condizionale, validatori a funzione. Rotta /iscrizione-reactive.

src/app/pagine/corsi-firebase CRUD con HttpClient verso Firebase, registro delle chiamate. Rotta /corsi-firebase.

src/app/pagine/material-esempio Angular Material: form field, select, bottoni, card, tabella, snack bar. Rotta /material.

src/app/servizi/corsi-firebase.ts Servizio HTTP con GET, POST, PUT, PATCH, DELETE.

src/app/servizi/registro-http.ts Interceptor che registra le chiamate.

src/app/pipes/troncato.ts Pipe personalizzata.

src/app/direttive/highlight.ts Direttiva di attributo.

***.spec.ts** 30 test: pipe, direttiva, servizio HTTP con HttpTestingController, componenti.

firebase-finto.mjs Server locale che imita Firebase: npm run firebase-finto.

src/environments Indirizzo del database per sviluppo e produzione.

Avvio completo per provare tutto:

```
npm run firebase-finto
npx ng serve
```

In due terminali separati. Poi apri <http://localhost:4200>.